بايثون للذكاء الاصطناعي: من الصفر إلى البناء

كتاب تأسيسي لطلاب مسار B في برنامج LLM4Yemen

Python for AI: From Zero to Building

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

2026

English · عربي

بايثون للذكاء الاصطناعي: من الصفر إلى البناء

Python for AI: From Zero to Building

كتاب تأسيسي لطلاب مسار B في برنامج LLM4Yemen

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

الطبعة الأولى — 2026 · First Edition

جميع الحقوق محفوظة للمؤلف

بسم الله الرحمن الرحيم

إهداء

إلى طلاب اليمن الذين يحفرون في الصخر بأظفارهم، فيُعلّمون العالم أن الإرادة أقوى من الظلام. إلى كل عقلٍ شابٍّ يقرأ هذا الكتاب على ضوء شمعة، أو على شاشةٍ تومض مع كل انقطاع للتيار: لكم كُتب هذا الكتاب، ولكم سيُكتب الذي بعده. ﴿وَقُل رَّبِّ زِدْنِي عِلْمًا﴾

تنبيه

هذا الكتاب أُنتج بمساعدة نموذج Claude Opus 4.7 من شركة Anthropic، تحت إشراف وتوجيه المؤلف. التوجيه الفكري واختيار الموضوع والمراجعة والمسؤولية النهائية للمؤلف الدكتور فضل محمد الأكوع. الكتاب لأغراض تثقيفية وإثرائية، ولا يُغني عن استشارة المتخصصين في الموضوعات القانونية والطبية والمالية والشرعية.

Notice

This book was produced with the assistance of Anthropic's Claude Opus 4.7 model, under the author's direction and supervision. Intellectual direction, topic selection, review, and final responsibility rest with the author, Dr. Fadhl Mohammed Alakwaa. The book is for educational purposes and does not substitute for consultation with specialists in legal, medical, financial, or religious matters.

المحتويات

المقدمة: لماذا بايثون، ولماذا الآن؟	1
الفصل الثاني: إعداد بيئة العمل	2
الفصل الثالث: برنامجك الأول وكيف يفكر بايثون	3
الفصل الرابع: المتغيرات والأنواع	4
الفصل الخامس: التحكم في تدفق البرنامج	5
الفصل السادس: الدوال	6
الفصل السابع: هياكل البيانات	7
الفصل الثامن: قراءة الملفات وكتابتها	8
الفصل التاسع: التعامل مع JSON و CSV	9
الفصل العاشر: طلبات HTTP ومكتبة requests	10

الفصل الحادي عشر: معالجة الأخطاء واستراتيجيات الـ Debugging 11

الفصل الثاني عشر: متغيرات البيئة وحفظ مفاتيح API بأمان 12

الفصل الثالث عشر: استدعاء نماذج اللغة الكبيرة من بايثون 13

الفصل الرابع عشر: المشروع الختامي — مُلَخِّص ملف CSV عربي بالذكاء الاصطناعي 14

الفصل الخامس عشر: حلول التمارين 15

الفصل السادس عشر: الخاتمة والجسر إلى عالم RAG 16

المراجع 17

المقدمة: لماذا بايثون، ولماذا الآن؟

﴿وَقُل رَّبِّ زِدْنِي عِلْمًا﴾

سورة طه — الآية 114

كلمة إلى طالب اليمن

عزيزي الطالب، عزيزتي الطالبة في برنامج LLM4Yemen — مسار B،

إن كنت تفتح هذا الكتاب وأنت لم تكتب سطر برمجة واحدًا في حياتك، فاعلم أنك في المكان الصحيح. هذا الكتاب لم يُكتب لمهندسي البرمجيات في وادي السيليكون، بل كُتب لك أنت: الطالب اليمني الذي يدرس على حاسوب بذاكرة 8 جيجا بايت، الذي قد ينقطع عنه التيار الكهربائي مرتين في اليوم، الذي يريد أن يتعلم الذكاء الاصطناعي ولكنه يحتاج أولًا إلى أدواته الأساسية.

البرمجة بلغة بايثون هي بوابتك إلى عالم الذكاء الاصطناعي. كل نموذج لغوي كبير تستخدمه، وكل تطبيق يستخدم الذكاء الاصطناعي، يعتمد في جوهره على

بايثون. وفي الأسبوعين القادمين، ستتعلم ما يكفي من بايثون لتبني نظامك الخاص في الكتاب التالي عن هندسة الاسترجاع المعزز (RAG).

لماذا بايثون بالذات؟

تخيل أنك تريد أن تتعلم لغة جديدة لتتحدث مع شعبٍ ما. هل ستختار لغة يتحدثها مليون شخص، أم لغة يتحدثها عشرات الملايين؟ بايثون هي اللغة الرسمية لمجتمع الذكاء الاصطناعي عالميًا. وعندما يصدر باحث في جامعة ستانفورد ورقة علمية جديدة، أو يطلق فريق Anthropic نموذجًا جديدًا، فإن أول كود يصدرونه يكون بلغة بايثون.

أربعة أسباب تجعل بايثون اختيارك الأول:

أولًا: السهولة. بايثون أقرب لغة برمجة إلى اللغة الإنجليزية المنطوقة. السطر التالي يعمل فعلًا:

```
if temperature > 30:
    print("الجو حار جدًا")
```

لا يحتاج هذا السطر إلى شرح طويل. تقرؤه فتفهم: "إذا كانت الحرارة أكبر من 30، اطبع: الجو حار جدًا".

ثانيًا: المكتبات الجاهزة. كل ما تحتاجه تقريبًا في الذكاء الاصطناعي مكتوب بالفعل في بايثون. تريد قراءة ملف CSV؟ سطر واحد. تريد استدعاء نموذج لغوي ضخم من Anthropic؟ خمسة أسطر. تريد بناء قاعدة بيانات متجهية لنظام RAG؟ كل المكتبات جاهزة.

ثالثًا: المجتمع. عندما تواجه مشكلة في كودك، فإن آلاف المبرمجين قد واجهوها قبلك. ستجد الإجابة على Stack Overflow وعلى مواقع GitHub. وستجد دروسًا على YouTube بالعربية وبالإنجليزية.

رابعًا: المجانية. بايثون مجانية تمامًا. لا تحتاج إلى رخصة، ولا إلى دفع رسوم سنوية. كل ما ستتعلمه في هذا الكتاب يمكنك تشغيله بدون أن تدفع ريالًا واحدًا.

كيف يربط هذا الكتاب بكتاب RAG التالي؟

كتابك التالي في برنامج LLM4Yemen هو "هندسة نظم RAG: من المفهوم إلى الإنتاج". وكل مهارة تتعلمها هنا ستستخدمها هناك مباشرةً. الجدول التالي يوضح الجسر بين الكتابين:

ما تتعلمه في هذا الكتاب	كيف ستستخدمه في كتاب RAG
قراءة وكتابة الملفات (الفصل 8)	تحميل المستندات (PDF، نصوص، Word) لإدخالها في نظام RAG
القواميس والقوائم (الفصل 7)	

ما تتعلمه في هذا الكتاب	كيف ستستخدمه في كتاب RAG
	تخزين البيانات الوصفية (metadata) للقطع النصية المتجهة
طلبات HTTP والمكتبة `requests` (الفصل 10)	استدعاء نماذج التضمين (embedding APIs) من OpenAI و Voyage
التعامل مع JSON و CSV (الفصل 9)	استلام نتائج البحث وتنسيقها للنموذج اللغوي
متغيرات البيئة و `.env` (الفصل 12)	إخفاء مفاتيح API الخاصة بقواعد بيانات Pinecone و Weaviate
استدعاء واجهات النماذج اللغوية (الفصل 13)	بناء طبقة التوليد في نظام RAG (Generation Layer)
معالجة الأخطاء (الفصل 11)	جعل نظامك يستمر في العمل عند فشل أي خطوة في خط الأنابيب
المشروع الختامي (الفصل 14)	الأساس الكامل لبناء أول نظام RAG تجريبي لك

تأمل في هذا الجدول. إنه يخبرك أن كل دقيقة تقضيها في تعلم بايثون هي استثمار مباشر في قدرتك على بناء أنظمة الذكاء الاصطناعي لاحقًا.

كيف تستخدم هذا الكتاب؟

هذا الكتاب صُمم لأسبوعين من الدراسة المكثفة، بمعدل فصل كامل أو فصلين في اليوم. ولكن السرعة ليست هي الهدف؛ الهدف هو أن **تكتب كل سطر كود بيدك**.

لا تنسخ الكود من الكتاب وتلصقه مباشرةً. اكتبه. اكتبه بأصابعك حرفًا حرفًا. سيخطئ إصبعك في البداية، وسيخبرك بايثون أنك نسيت قوسًا أو فاصلة، وستتعلم من هذا الخطأ أكثر مما تتعلم من قراءة الكود مكتوبًا أمامك. هذه هي القاعدة الذهبية للمبتدئين.

كل فصل من فصول هذا الكتاب يحتوي على:

أولًا، شرح بلغة بسيطة لمفهوم جديد، مع أمثلة من حياتك اليومية. ثانيًا، أمثلة كود قصيرة، كل سطر منها مشروح. ثالثًا، تمارين في نهاية الفصل لتختبر فهمك بنفسك. رابعًا، إجابات هذه التمارين موجودة في فصل خاص في آخر الكتاب — لكن لا تنظر إليها قبل أن تحاول بنفسك ساعة كاملة على الأقل. الصبر على المحاولة هو ما يصنع المبرمج.

ما تحتاجه قبل أن تبدأ

لا تحتاج إلى أي معرفة سابقة بالبرمجة. تحتاج فقط إلى:

حاسوب يعمل بنظام Windows أو macOS أو Linux، بذاكرة لا تقل عن 4 جيجا بايت (الكتاب صُمم خصيصًا ليعمل على حواسيب 8 جيجا بايت). اتصال بالإنترنت لتنزيل الأدوات في البداية وللوصول إلى Google Colab. صبر لمدة أسبوعين. شغف لأن تكون من أوائل بناة الذكاء الاصطناعي في اليمن.

إن كان حاسوبك ضعيفًا جدًا، أو إذا كان التيار الكهربائي يقطع عنك كثيرًا، فلا تقلق. سنعلمك كيف تستخدم Google Colab، وهي بيئة بايثون كاملة تعمل عبر متصفح الإنترنت، ولا تحتاج إلى تثبيت أي شيء على حاسوبك.

نصيحة أخيرة قبل البداية

البرمجة ليست موهبة فطرية، بل مهارة تُكتسب بالتكرار. لن تفهم كل شيء من القراءة الأولى. هذا طبيعي. سيظن عقلك في اليوم الأول أنك لن تستطيع أبدًا. ثم في اليوم الخامس ستُحل أمامك المسائل التي حيرتك في اليوم الأول. هذه هي رحلة كل مبرمج في العالم. ثق بالعملية، وثق بنفسك، وابدأ.

﴿وَقُل رَّبِّ زِدْنِي عِلْمًا﴾.

والآن، انتقل إلى الفصل الثاني، وابدأ بإعداد بيئة العمل.

تمارين الفصل الأول

1. اكتب فقرة قصيرة (5 أسطر) عن سبب رغبتك في تعلم الذكاء الاصطناعي. احتفظ بهذه الورقة. اقرأها بعد أسبوعين عندما تنهي الكتاب.
2. ابحث في الإنترنت عن "ما الفرق بين بايثون 2 و بايثون 3؟" واكتب الإجابة بخمسة أسطر.
3. اذكر ثلاثة منتجات أو تطبيقات تستخدمها يوميًا، وتعتمد على الذكاء الاصطناعي.

الفصل الثاني: إعداد بيئة العمل

> "العالم بلا أداة كالفارس بلا فرس."
>
> مثل عربي

قبل أن تكتب أول سطر بايثون، عليك أن تجهز ورشتك. كما لا يبدأ النجار عمله قبل أن يُحضر منشاره ومطرقته، فإنك ستحتاج إلى أدوات أساسية: مفسر بايثون نفسه، ومحرر نصوص ذكي يساعدك على كتابة الكود، وبيئة سحابية احتياطية حين ينقطع التيار.

سنغطي في هذا الفصل ثلاثة مسارات: المسار المحلي على حاسوبك باستخدام VS Code، والمسار السحابي باستخدام Google Colab، وتنظيم البيئات الافتراضية (virtual environments) لمنع التعارض بين المشاريع.

أولًا: تثبيت بايثون على حاسوبك

ابدأ بزيارة الموقع الرسمي للغة بايثون: `python.org`. اضغط على زر "Downloads" واختر أحدث إصدار مستقر — في وقت كتابة هذا الكتاب هو

الإصدار 3.12. تجنب الإصدارات التي تحمل الحرف (a) أو (b) أو (rc) لأنها تجريبية.

عند تثبيت بايثون على Windows، توقف عند الشاشة الأولى للمثبّت، وتأكد من تفعيل خيار **Add Python to PATH**. هذا الخيار حاسم — بدونه لن يتعرف نظامك على بايثون من سطر الأوامر، وستقضي ساعات في حل مشكلة كان حلها ضغطة واحدة. إن نسيت هذا الخيار، أزل بايثون وأعد التثبيت.

بعد انتهاء التثبيت، افتح "موجه الأوامر" (Command Prompt) على Windows أو "Terminal" على macOS و Linux، واكتب:

```
python --version
```

يجب أن يطبع لك سطرًا يشبه: `Python 3.12.3`. إن ظهر هذا السطر، فقد نجحت. إن ظهرت رسالة خطأ، عد إلى خطوات التثبيت وتأكد من تفعيل خيار PATH.

ثانيًا: تثبيت محرر VS Code

بايثون نفسها لا تحتوي على محرر نصوص. أنت بحاجة إلى أداة تساعدك على كتابة الكود، وتلون الكلمات المهمة، وتنبهك إلى الأخطاء قبل تشغيل البرنامج. أفضل خيار مجاني هو Visual Studio Code من ميكروسوفت.

زر `code.visualstudio.com` ونزّل النسخة المناسبة لنظامك. بعد التثبيت، افتح VS Code وانتقل إلى أيقونة الإضافات (Extensions) في الشريط الجانبي. ابحث عن "Python" وثبّت الإضافة الرسمية من Microsoft. هذه الإضافة تضيف لك تشغيل الكود بزر واحد، وتلوين الكلمات المفتاحية، واقتراح الإكمال التلقائي.

أنصحك أيضًا بتثبيت الإضافات التالية:

Pylance — يوفر مساعدة أكثر ذكاءً في الكتابة، ويكشف الأخطاء قبل التشغيل.

Jupyter — يتيح لك فتح ملفات الدفاتر التفاعلية (notebooks) بصيغة `.ipynb` داخل VS Code مباشرةً.

Arabic Language Pack — لو أردت واجهة VS Code بالعربية. (شخصيًا أنصحك بإبقائها بالإنجليزية، فأغلب الموارد التعليمية تستخدم المصطلحات الإنجليزية، وستجد المساعدة على الإنترنت أسرع).

ثالثًا: مشروعك الأول — اختبار البيئة

أنشئ مجلدًا جديدًا في مكان مناسب على حاسوبك، مثلًا `C:\python-projects` على Windows أو `~/python-projects` على macOS. افتح VS Code، ومن قائمة File اختر Open Folder، ثم اختر هذا المجلد.

داخل VS Code، أنشئ ملفًا جديدًا باسم `hello.py`. (الامتداد `.py` مهم — هو الذي يخبر VS Code أن هذا ملف بايثون). الصق فيه السطر التالي:

```
print("مرحبًا أيها العالم! أنا أتعلم بايثون.")
```

احفظ الملف بـ `Ctrl+S`. الآن افتح "Terminal" داخل VS Code من قائمة View → Terminal. اكتب:

```
python hello.py
```

سيظهر السطر مطبوعًا في نافذة Terminal:

```
مرحبًا أيها العالم! أنا أتعلم بايثون.
```

تهانينا! لقد شغّلت أول برنامج بايثون في حياتك.

رابعًا: Google Colab — البديل السحابي

قد لا يعمل حاسوبك دائمًا. قد ينقطع التيار، أو قد تكون في الجامعة بعيدًا عن حاسوبك. هنا يأتي دور Google Colab — بيئة بايثون كاملة تعمل عبر المتصفح، مجانية، ولا تحتاج إلى تثبيت أي شيء.

زر colab.research.google.com. سجل الدخول بحساب Gmail. اضغط على "New Notebook". ستفتح صفحة فيها صندوق فارغ يسمى "خلية" (cell). اكتب فيه:

```
print("مرحبًا من السحابة")
2 + 2
```

اضغط على زر التشغيل (المثلث الأيمن) أو Shift+Enter. ستظهر النتيجة تحت الخلية مباشرة. هذا هو سحر Colab: لا حاجة إلى ملفات منفصلة، ولا حاجة إلى Terminal — كل شيء في صفحة واحدة.

ميزات Colab الإضافية: تحصل على معالج رسومي (GPU) مجاني لتدريب نماذج الذكاء الاصطناعي. تحصل على ذاكرة 12 جيجا بايت مجانًا. ملفاتك محفوظة على Google Drive تلقائيًا. وأهم من ذلك كله: عملك لا يضيع إذا انقطع التيار في حاسوبك.

عيوب Colab: يحتاج إلى اتصال إنترنت مستمر. الجلسة تنتهي بعد 12 ساعة من عدم النشاط. لا يصلح للتطبيقات التي تحتاج إلى نوافذ رسومية.

نصيحتي العملية: استخدم VS Code للمشاريع الجدية الطويلة، واستخدم Colab للتجارب السريعة وللعمل عند انقطاع التيار.

خامسًا: البيئات الافتراضية (Virtual Environments)

تخيل أن لديك مشروعين: مشروع قديم يستخدم نسخة معينة من مكتبة `requests`، ومشروع جديد يحتاج نسخة أحدث منها. إن ثبّت النسختين معًا في بايثون، فستتعارض المكتبتان ولن يعمل أي مشروع. هنا تأتي فكرة "البيئة الافتراضية": صندوق منفصل لكل مشروع، فيه نسخه الخاصة من المكتبات.

لإنشاء بيئة افتراضية، انتقل إلى مجلد مشروعك في Terminal، ثم اكتب:

```
python -m venv venv
```

هذا الأمر ينشئ مجلدًا جديدًا اسمه `venv` فيه نسخة معزولة من بايثون. لتفعيل البيئة:

على Windows:

```
venv\Scripts\activate
```

على macOS و Linux:

```
source venv/bin/activate
```

ستلاحظ ظهور كلمة `(venv)` في بداية سطر Terminal. هذا يعني أنك الآن داخل البيئة الافتراضية. أي مكتبة تثبتها بأمر `pip install` ستذهب إلى هذه البيئة فقط، ولن تؤثر على بقية حاسوبك.

لتثبيت أول مكتبة، جرب:

```
pip install requests
```

ستثبَّت مكتبة `requests` (التي سنستخدمها في الفصل العاشر) داخل بيئتك.

للتأكد:

```
pip list
```

سيعرض قائمة بكل المكتبات المثبتة. لإلغاء تفعيل البيئة عند انتهاء العمل:

```
deactivate
```

نصيحة ذهبية: أنشئ بيئة افتراضية جديدة لكل مشروع. اعتد على هذه العادة من الآن. سترى لاحقًا — حين تعمل على نظام RAG وتحتاج إلى مكتبات متخصصة — أن البيئة الافتراضية تحميك من فوضى لا تتخيلها.

سادسًا: ملف requirements.txt

عند انتهاء مشروعك، يمكنك حفظ قائمة بكل المكتبات التي استخدمتها في ملف نصي:

```
pip freeze > requirements.txt
```

هذا الملف يصبح "وصفة" مشروعك. عندما يأتي زميل لاحقًا ويريد تشغيل كودك، فإنه يكتب فقط:

```
pip install -r requirements.txt
```

وكل المكتبات تثبَّت تلقائيًا بنفس النسخ التي استخدمتها أنت. هذا ما يسميه المبرمجون "إعادة الإنتاج" (reproducibility) — قدرة شخص آخر على تشغيل كودك بنفس الطريقة التي يعمل بها لديك.

خلاصة الفصل

أصبحت ورشتك جاهزة. بايثون مثبتة، VS Code يعمل، Google Colab متاح كاحتياطي، والبيئة الافتراضية تحمي مشاريعك من التعارض. في الفصل القادم، ستكتب برنامجك الأول الحقيقي، وتفهم كيف يفكر بايثون.

تمارين الفصل الثاني

1. ثبّت بايثون على حاسوبك واكتب `python --version` للتأكد.
   صور الناتج إن أردت أن تشاركه مع زملائك.
2. أنشئ مجلدًا اسمه `learning-python`، وأنشئ بداخله بيئة افتراضية، وفعّلها، ثم ثبّت مكتبة `requests`، واكتب `pip list` لتتأكد من تثبيتها.
3. سجل دخولك إلى Google Colab، أنشئ دفترًا جديدًا، واكتب فيه برنامجًا يطبع اسمك بالعربية ثلاث مرات.
4. أنشئ ملف `requirements.txt` لمشروعك، وافتحه في VS Code وانظر إلى محتواه.

الفصل الثالث: برنامجك الأول وكيف يفكر بايثون

> "إن أحسنت الجلوس مع نفسك ساعةً، أحسنت العمل ساعتين."
>
> حكمة قديمة

في هذا الفصل، ستفهم كيف يقرأ بايثون الكود، وكيف ينفّذ الأوامر سطرًا سطرًا، وكيف تتحدث إليه عبر الطباعة (`print`) وعبر الإدخال (`input`). هذه هي اللبنات الأولى لأي برنامج، وستراها في كل سطر تكتبه لاحقًا.

كيف يفكر بايثون؟

بايثون مفسّر (interpreter)، وليس مترجمًا (compiler). الفرق المهم لك كمبتدئ: بايثون يقرأ كودك سطرًا سطرًا، من أعلى الملف إلى أسفله، ويُنفّذ كل سطر بمفرده. إن وصل إلى سطر فيه خطأ، فإنه يتوقف عن التنفيذ ويخبرك بالخطأ. هذا يعني أنك تستطيع كتابة سطر، تشغيله، رؤية النتيجة، ثم كتابة السطر التالي. هذه دورة سحرية تسمى "الحلقة التفاعلية" (REPL: Read-Eval-Print Loop).

افتح Terminal واكتب الأمر python فقط، بدون اسم ملف. ستظهر علامات >>> تنتظر منك أن تكتب. هذه هي الحلقة التفاعلية. جرب:

```
>>> 2 + 2
4
>>> print("سلام")
سلام
>>> name = "فضل"
>>> print("مرحبًا يا ", name)
مرحبًا يا فضل
>>> exit()
```

في Colab، كل خلية بحد ذاتها تعمل كحلقة تفاعلية صغيرة. تكتب فيها كودًا، تضغط Shift+Enter، وتظهر النتيجة فورًا.

دالة print — كيف تتحدث إلى الشاشة

print هي أهم دالة ستتعلمها. كل ما تكتبه بين قوسيها سيظهر على الشاشة.

أمثلة:

```
print("مرحبًا")          # يطبع: مرحبًا
print("اسمي فضل")        # يطبع: اسمي فضل
```

```
print(2 + 3)                    # يطبع: 5
print("النتيجة هي:", 2 + 3)  # يطبع: النتيجة هي: 5
```

لاحظ ثلاثة أشياء مهمة: - النصوص توضع داخل علامتي تنصيص: "..." أو '...'. أي اقتباس صحيح طالما تطابق البداية والنهاية. - الأرقام تُكتب بدون تنصيص. لو وضعت الرقم بين تنصيص أصبح نصًا، لا رقمًا. - يمكنك تمرير عدة قيم مفصولة بفواصل. سيضع بايثون مسافة تلقائية بينها.

الفواصل والمسافات

إن أردت تغيير الفاصل بين القيم، استخدم المعامل sep:

```
print("أحمد", "علي", "خالد", sep=" و ")
# يطبع: أحمد و علي و خالد
```

وإن أردت ألا يضع بايثون سطرًا جديدًا في النهاية، استخدم end:

```
print("جزء أول،", end="")
print("جزء ثانٍ")
# يطبع: جزء أول، جزء ثانٍ
```

دالة input — كيف يتحدث المستخدم إليك

input تتيح لك أن تطلب من المستخدم أن يكتب شيئًا. مثلًا:

```
name = input("ما اسمك؟ ")
print("مرحبًا يا ", name)
```

عند تشغيل هذا البرنامج، سيظهر السؤال "ما اسمك؟"، وسينتظر بايثون. حين تكتب اسمك وتضغط Enter، يحفظ الاسم في المتغير name، ثم يستخدمه في السطر التالي.

ملاحظة مهمة جدًا: كل ما يُدخله المستخدم عبر input هو **نص** (string)، حتى لو كتب رقمًا. لو سألت المستخدم عن عمره وأراد أن تجمعه على عدد، فعليك تحويل النص إلى رقم:

```
age = input("كم عمرك؟ ")
print("في العام القادم سيكون عمرك:", int(age) + 1)
```

int(age) تحوّل النص إلى عدد صحيح. لو نسيت هذه الخطوة وكتبت age + 1 مباشرة، لظهر خطأ غامض اسمه TypeError: can only concatenate str (not "int") to str.

التعليقات

أي شيء يبدأ بـ # يتجاهله بايثون. هذه تسمى "تعليقات" (comments)، وتستخدم لشرح ما يفعله الكود لمن يقرؤه لاحقًا — بما في ذلك أنت بعد شهرين:

```
# هذا برنامج يحسب مساحة دائرة
radius = 5          # نصف القطر بالأمتار
pi = 3.14159        # ثابت باي
area = pi * radius * radius
print("المساحة:", area)
```

عوّد نفسك على التعليق منذ اليوم الأول. كود بدون تعليقات هو لغز محبوك بإحكام، حتى صاحبه ينسى مفاتيحه.

المسافات والإزاحة (Indentation)

بايثون مختلفة عن أغلب اللغات في نقطة حاسمة: المسافات في بداية السطر **لها معنى**. لا يستخدم بايثون أقواسًا مجعدة { } ليُحدد كتل الكود، بل يستخدم الإزاحة (Indentation) — أي عدد المسافات في بداية كل سطر.

سترى في الفصل القادم كيف يعمل هذا مع `if` و `for`. للآن فقط احفظ القاعدة: عندما يطلب منك الكود إزاحة، استخدم **4 مسافات** أو زر `Tab`. لا تخلط بين

المسافات والـ Tab — اختر أحدهما والتزم به في الملف كله. VS Code سيدير هذا تلقائيًا لك إذا فعّلت إضافة Python.

برنامج كامل: حاسبة بسيطة

دعنا نطبق ما تعلمناه. أنشئ ملفًا اسمه `calculator.py` واكتب فيه:

```
# حاسبة بسيطة لشخصين
print("=== حاسبة جمع بسيطة ===")

first = input("اكتب الرقم الأول: ")
second = input("اكتب الرقم الثاني: ")

# نحوّل النصوص إلى أرقام
num1 = float(first)
num2 = float(second)

result = num1 + num2

print("الناتج هو:", result)
```

شغّله بـ `python calculator.py`. أدخل رقمين. تأمل النتيجة. الآن جرب أن تُدخل كلمة بدلًا من رقم. ماذا يحدث؟ ستحصل على رسالة خطأ. هذه هي بداية رحلتك مع معالجة الأخطاء — وسنتعلم في الفصل 11 كيف نمنع البرنامج من الانهيار.

`float` تحوّل النص إلى عدد عشري (مثل 3.14). و `int` تحوّل النص إلى عدد صحيح (مثل 5). الفرق سنغطيه بالتفصيل في الفصل القادم.

رسائل الخطأ — صديقك لا عدوّك

كثير من المبتدئين يُصابون بالذعر حين يرون رسالة خطأ بالأحمر. لكنّ رسائل خطأ بايثون هي من بين أوضح الرسائل في عالم البرمجة. تعلّم قراءتها. مثال:

```
:Traceback (most recent call last)
<File "calculator.py", line 7, in <module
num1 = float(first)
'مرح :ValueError: could not convert string to float
```

تخبرك هذه الرسالة بثلاثة أشياء مهمة: اسم الملف الذي حدث فيه الخطأ، رقم السطر (7)، ونوع الخطأ (`ValueError`)، ولماذا حدث (لا يستطيع تحويل النص "مرحبا" إلى رقم). كل ما عليك أن تفعل هو الذهاب إلى السطر 7 وتفهم لماذا وصل إليه نص لا يصلح.

نصيحة عملية: شغّل الكود كثيرًا

لا تنتظر حتى تنتهي من كتابة 50 سطرًا قبل أن تجرّب البرنامج. اكتب 3 أسطر، شغّل، تأكد، ثم أضف 3 أسطر أخرى. هذه عادة المبرمجين المحترفين. إن انتظرت كثيرًا قبل التشغيل، ستجمع 10 أخطاء معًا، ولن تعرف أيها سبب الآخر.

خلاصة الفصل

تعرفت على دالتي `print` و `input`، وكيف تكتب التعليقات، وكيف تقرأ رسائل الخطأ، وأن المسافات في بداية السطر مهمة في بايثون. أنت الآن قادر على كتابة برامج بسيطة تتفاعل مع المستخدم. في الفصل القادم، سنتعمق في المتغيرات والأنواع — كيف يحفظ بايثون البيانات، وكيف تختار النوع المناسب لكل بيانة.

تمارين الفصل الثالث

1. اكتب برنامجًا يسأل المستخدم عن اسمه وبلده، ثم يطبع جملة ترحيبية تتضمن الاسم والبلد. مثال: "أهلًا أحمد من صنعاء".
2. اكتب برنامجًا يحسب مساحة مستطيل. اطلب من المستخدم الطول والعرض، ثم اطبع المساحة.
3. عدّل برنامج الحاسبة ليحسب الفرق (-) والضرب (*) والقسمة (/) أيضًا، ويطبع كل النتائج الأربع.

4. اكتب برنامجًا يسأل عن سنة الميلاد ويحسب عمر المستخدم تقريبيًا (افترض أن العام الحالي 2026).

الفصل الرابع: المتغيرات والأنواع

> "الكلمات تحمل المعاني، والأنواع تحمل البيانات."

كل برنامج يحفظ بيانات: اسم مستخدم، عمره، نص رسالة، قائمة منتجات. الطريقة التي يحفظ بها بايثون هذه البيانات تسمى "متغيرات" (variables)، وكل متغير له "نوع" (type) يحدد ما يستطيع أن يحويه وما يستطيع أن يفعله.

ما هو المتغير؟

تخيل أن لديك صناديق على رف، وعلى كل صندوق ملصق باسم. تضع داخل الصندوق "أحمد" قيمته نص "محمد". تضع داخل صندوق "العمر" قيمته رقم 25. متى ما أردت قيمة، تنادي اسم الصندوق:

```
name = "محمد"
age = 25
city = "صنعاء"
```

```
print(name)          # محمد
print(age)           # 25
print(city)          # صنعاء
```

علامة = ليست "يساوي" بمعنى الرياضيات، بل "اجعل قيمة المتغير الأيسر مساويةً لما على اليمين". هذا يعني أن السطر `x = x + 1` ليس عبثًا — بل يقول "خذ قيمة x الحالية، أضف لها 1، وأعد حفظ النتيجة في x".

قواعد تسمية المتغيرات

اسم المتغير في بايثون يخضع لقواعد بسيطة: يبدأ بحرف أو شَرطة سفلية _، ولا يبدأ برقم. لا يحتوي على مسافات (استخدم _ بدلًا منها). حساس لحالة الأحرف (`Name` و `name` متغيران مختلفان). لا يمكن استخدام كلمات بايثون المحجوزة مثل `if` و `for` و `class`.

أمثلة جيدة: `student_name`, `total_score`, `is_active`, `book_count`.

أمثلة سيئة: `1name` (يبدأ برقم)، `student name` (مسافة)، `class` (كلمة محجوزة)، `x` (غير وصفي).

النصيحة الذهبية: اختر أسماءً وصفية. سيقرأ كودك زميلٌ ما، أو ستقرؤه أنت بعد شهرين. اسم `total_revenue_2026` أوضح بألف مرة من اسم `t`.

الأنواع الأساسية

بايثون يميّز بين عدة أنواع من البيانات. الأهم منها:

النص (String): متغير يحوي حروفًا أو كلمات أو جمل. يُكتب بين علامتي تنصيص.

```
greeting = "السلام عليكم"
city = 'صنعاء'
multiline = """نص يمتد
على عدة
أسطر"""
```

العدد الصحيح (Integer): عدد بدون كسور.

```
age = 25
year = 2026
temperature = -5
```

العدد العشري (Float): عدد فيه كسور.

```
price = 19.99
pi = 3.14159
weight = 75.5
```

القيمة المنطقية (Boolean): قيمة من قيمتين فقط: True أو False. (لاحظ أن الحرف الأول كبير).

```
is_student = True
has_passport = False
```

القيمة الفارغة (None): قيمة خاصة تمثل "لا شيء" أو "غير محدد".

```
result = None
```

سترى None كثيرًا حين تستدعي دالة لا تُرجع شيئًا.

معرفة نوع المتغير

استخدم دالة type() لمعرفة نوع أي متغير:

```
print(type("نص"))  # <class 'str>
print(type(25))  # <class 'int>
print(type(3.14))  # <class 'float>
```

```
<'print(type(True))        # <class 'bool
<'print(type(None))        # <class 'NoneType
```

هذه الدالة منقذة عند تتبع الأخطاء. لو لم يعمل برنامجك كما تتوقع، اطبع نوع المتغير وستفهم.

التحويل بين الأنواع

أحيانًا تحتاج إلى تحويل قيمة من نوع إلى آخر. بايثون يوفر دوال تحويل:

```
# نص إلى رقم
"age_text = "25
age_number = int(age_text)
print(age_number + 1)        # 26

# رقم إلى نص
score = 95
message = "نتيجتك: " + str(score)
print(message)               # نتيجتك: 95

# عدد عشري إلى عدد صحيح (يقطع الكسر، لا يقرّب)
price = 19.99
```

```
whole = int(price)
print(whole)                    # 19

# عدد صحيح إلى عشري
count = 5
ratio = float(count)
print(ratio)                    # 5.0
```

تنبيه: التحويل قد يفشل. `int("مرحبا")` سيعطي خطأ `ValueError`.

سنتعلم في الفصل 11 كيف نحمي برنامجنا من هذه الأخطاء.

العمليات على الأرقام

بايثون يدعم كل العمليات الحسابية المعتادة:

```
a = 10
b = 3

print(a + b)        # 13   (جمع)
print(a - b)        # 7    (طرح)
print(a * b)        # 30   (ضرب)
print(a / b)        # 3.333 ...(قسمة عادية، النتيجة
```

```
print(a // b)      # 3    (قسمة صحيحة، يهمل الكسر)
print(a % b)       # 1    (الباقي)
print(a ** b)      # 1000 (أس: 10 مرفوعة للقوة 3)
```

اعرف الفرق بين / و // جيدًا. / تُرجع دائمًا قيمة عشرية، حتى لو كانت 4/2 = 2.0. // تُرجع قيمة صحيحة بقطع الكسر.

العمليات على النصوص

النصوص في بايثون قوية جدًا. تستطيع جمعها، تكرارها، الوصول إلى حروفها، تغيير حالتها:

```
first = "محمد"
last = "علي"

# الجمع (concatenation)
full = first + " " + last
print(full)          # محمد علي

# التكرار
line = "-" * 20
print(line)          # ----------
```

```
# الطول
print(len(full))  # 9 (مع المسافة)

# الوصول إلى حرف معين (يبدأ من 0)
print(full[0])  # م
print(full[-1])  # ي (آخر حرف)

# تقطيع (slicing)
print(full[0:4])  # محمد

# تغيير الحالة (للنصوص اللاتينية)
greeting = "Hello World"
print(greeting.upper())  # HELLO WORLD
print(greeting.lower())  # hello world
print(greeting.title())  # Hello World

# استبدال
text = "أنا أحب القهوة"
new_text = text.replace("القهوة", "الشاي")
print(new_text)  # أنا أحب الشاي
```

```
# تقسيم نص إلى قائمة
sentence = "أحمد,محمد,خالد"
names = sentence.split(",")
print(names)  # ['أحمد', 'محمد', 'خ

# دمج قائمة في نص
joined = " - ".join(names)
print(joined)  # أحمد - محمد - خالد
```

النصوص المنسقة (f-strings)

من أهم ما تعلّمته بايثون مؤخرًا: f-strings. طريقة أنيقة لتركيب النصوص مع المتغيرات:

```
name = "فضل"
age = 35

# الطريقة القديمة
print("اسمي " + name + " وعمري " + str(age)
```

```
# طريقة f-string (الأفضل)
print(f"اسمي {name} وعمري {age}")

# يمكنك حتى وضع تعبيرات داخل {}
print(f"بعد 5 سنوات سيكون عمري {age + 5}")

# تحديد عدد الخانات العشرية
price = 19.998765
print(f"السعر: {price:.2f}")      # السعر: 20.00
```

استخدم f-strings دائمًا. هي الطريقة الموصى بها رسميًا في بايثون الحديثة، وستجدها في كل الكود الذي ستراه في الذكاء الاصطناعي.

الثوابت

بايثون لا يدعم "الثوابت" بشكل رسمي مثل بعض اللغات، لكن العرف هو أن تكتب اسم المتغير بأحرف كبيرة لتُشير إلى أنه ثابت لا يجب تعديله:

```
PI = 3.14159
MAX_RETRIES = 3
API_BASE_URL = "https://api.anthropic.com/v1"
```

أي مبرمج يقرأ كودك سيفهم أن هذه قيم ثابتة، لا تُعدّل أثناء تشغيل البرنامج.

المتغيرات في بايثون مرنة (Dynamic Typing)

في بايثون، يمكنك تغيير نوع المتغير في أي وقت:

```
x = "نص"          # x الآن str
x = [1, 2, 3]     # x الآن list
x = 10            # x الآن int
```

هذه ميزة وعيب في الوقت نفسه. ميزة لأنها مرنة. وعيب لأنها تُنتج أخطاء صعبة الاكتشاف. لتجنب المشاكل، اتفق مع نفسك أن المتغير يحتفظ بنوعه طوال البرنامج. إن أردت تخزين شيء جديد، أعطه اسمًا جديدًا.

خلاصة الفصل

تعلمت أنواع البيانات الأساسية في بايثون: النصوص، الأرقام الصحيحة والعشرية، القيم المنطقية، وقيمة `None`. تعلمت كيف تحوّل بين الأنواع، وكيف تجري العمليات على الأرقام والنصوص، وكيف تستخدم f-strings للطباعة المنسقة. هذه اللبنات ستظهر في كل سطر بايثون ستكتبه.

تمارين الفصل الرابع

1. اكتب برنامجًا يطلب من المستخدم سعرًا ونسبة خصم، ويحسب السعر بعد الخصم. مثال: السعر 100، الخصم 20%، يطبع: السعر بعد الخصم: 80.
2. اكتب برنامجًا يأخذ نصًا من المستخدم ويطبع: عدد حروفه، النص بعكس ترتيبه، النص بأحرف كبيرة (إن كان إنجليزيًا).
3. اكتب برنامجًا يحسب متوسط ثلاث درجات يدخلها المستخدم، ويطبع المتوسط بخانتين عشريتين باستخدام f-string.
4. ما الفرق بين / و // و %؟ اكتب 3 أمثلة لكل واحدة منها.

الفصل الخامس: التحكم في تدفق البرنامج

> "العقل يميّز، والمنطق يقرر."

كل ما كتبناه حتى الآن يسير في خط مستقيم: سطر تلو الآخر من أعلى الملف إلى أسفله. لكن البرامج الحقيقية تحتاج إلى **قرارات** و **تكرار**. أن نقول لبايثون: "إن كانت درجة الطالب فوق 90 فاطبع ممتاز، وإلا فاطبع جيد". أو: "كرر هذه العملية على كل اسم في القائمة".

هذا ما يسمى "التحكم في التدفق" (control flow)، وهو قلب أي برنامج.

الجملة الشرطية: if / elif / else

أبسط قرار في بايثون يبدأ بكلمة `if`:

```
age = 18
```

```
if age >= 18:
    print("بالغ، يحق له التصويت")
```

لاحظ ثلاثة أشياء: - نهاية سطر `if` بنقطتين رأسيتين `:`. - السطر التالي مزاح بأربع مسافات (هذا ضروري — الإزاحة هي ما يجعل بايثون يعرف أن السطر تابع للشرط). - علامة == للمقارنة (تختلف عن = التي تُسند قيمة).

عمليات المقارنة في بايثون: == (يساوي)، ! = (لا يساوي)، < (أصغر من)، > (أكبر من)، <= (أصغر من أو يساوي)، >= (أكبر من أو يساوي).

else — ماذا لو لم يتحقق الشرط؟

```
age = 15

if age >= 18:
    print("بالغ")
else:
    print("قاصر")
```

`else` لا تأخذ شرطًا، فهي تعني "في كل الحالات الأخرى".

elif — شروط متعددة

عندما يكون عندك أكثر من حالتين، استخدم elif (اختصار لـ else if):

```
score = 75

if score >= 90:
    print("ممتاز")
elif score >= 80:
    print("جيد جدًا")
elif score >= 70:
    print("جيد")
elif score >= 60:
    print("مقبول")
else:
    print("راسب")
```

ينفّذ بايثون أول شرط يتحقق، ويتجاهل الباقي. إن كان الشرط الأول صحيحًا، لن يفحص بايثون الشروط التالية.

الجمع بين الشروط: and / or / not

```
age = 25
has_passport = True

if age >= 18 and has_passport:
    print("يمكنك السفر")

if age < 18 or not has_passport:
    print("لا يمكنك السفر")
```

and يتطلب أن يكون الشرطان صحيحين. or يتحقق إن كان أحد الشرطين صحيحًا. not يقلب القيمة المنطقية.

التحقق من العضوية: in

عملية مفيدة جدًا — تخبرك إن كان شيء موجودًا في نص أو في قائمة:

```
name = "محمد"
if "م" in name:
    print("الاسم يحتوي على حرف الميم")
```

```
cities = ["صنعاء", "عدن", "تعز", "الحديدة"]
city = input("اكتب اسم مدينتك: ")
if city in cities:
    print("نعرف مدينتك")
else:
    print("لا تظهر في قائمتنا")
```

القيم "الكاذبة" (Falsy values)

في بايثون، القيم التالية تعتبر False عند فحصها كشرط: - False نفسها - None - الصفر (0 و 0.0) - النص الفارغ "" - القائمة الفارغة [] - القاموس الفارغ {}

كل ما عدا ذلك يُعتبر True. هذا يبسط كثيرًا من الفحوصات:

```
name = input("اكتب اسمك: ")

if name:
    print(f"مرحبًا يا {name}")
else:
    print("لم تكتب اسمًا")
```

`if name:` تعني "إن كان name ليس فارغًا".

حلقة for — التكرار على عناصر

أكثر حلقة ستستخدمها هي for. هي تتكرر على عناصر "مجموعة" (نص، قائمة، قاموس...).

```
names = ["أحمد", "محمد", "خالد"]
for name in names:
    print(f"مرحبًا يا {name}")
```

يقرأ بايثون: "خذ كل عنصر من names، سمّه name مؤقتًا، ونفّذ السطور المزاحة تحته".

يمكنك التكرار على نص أيضًا (سيتكرر على كل حرف):

```
for letter in "بايثون":
    print(letter)
```

range — التكرار عددًا معينًا من المرات

range(n) تنتج سلسلة أرقام من 0 إلى n-1.

```
for i in range(5):
    print(i)
# يطبع: 0, 1, 2, 3, 4
```

range(start, stop) تبدأ من start وتنتهي قبل stop:

```
for i in range(1, 11):
    print(i)
# يطبع: 1, 2, 3, ..., 10
```

range(start, stop, step) تأخذ خطوة معينة:

```
for i in range(0, 21, 5):
    print(i)
# يطبع: 0, 5, 10, 15, 20
```

حلقة while — كرر طالما الشرط صحيح

```
balance = 1000
while balance > 0:
    spend = int(input("كم تنفق؟ "))
    balance = balance - spend
    print(f"المتبقي: {balance}")
```

```
print("نفد الرصيد!")
```

تستمر `while` في التكرار طالما الشرط صحيح. **انتبه:** إن لم يتغير الشرط داخل الحلقة، ستحصل على حلقة لا نهائية. اضغط `Ctrl+C` لإيقاف برنامج علق في حلقة لا نهائية.

break و continue

`break` تخرج من الحلقة فورًا:

```
for number in range(1, 100):
    if number == 7:
        print("وجدت السبعة!")
        break
    print(number)
```

`continue` تتخطى ما تبقى من الدورة الحالية وتذهب للدورة التالية:

```
for number in range(1, 11):
    if number % 2 == 0:
        continue          # تجاهل الأرقام الزوجية
    print(number)         # سيطبع: 1, 3, 5, 7, 9
```

else بعد الحلقة

ميزة فريدة في بايثون: يمكنك إضافة else بعد حلقة for أو while. ينفَّذ هذا الـelse إن انتهت الحلقة بشكل طبيعي (لم تُكسر بـ break):

```
numbers = [2, 4, 6, 8]
for n in numbers:
    if n % 2 == 1:
        print("وجدت رقمًا فرديًا")
        break
else:
    print("كل الأرقام زوجية")
```

القائمة الاستيعابية (List Comprehension)

طريقة أنيقة لإنشاء قائمة من حلقة في سطر واحد. مثلًا، لإنشاء قائمة بمربعات الأرقام من 1 إلى 10:

```
# الطريقة الطويلة
squares = []
for i in range(1, 11):
    squares.append(i * i)
```

```
# الطريقة المختصرة
squares = [i * i for i in range(1, 11)]
(squares)  # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

يمكنك إضافة شرط:

```
# مربعات الأرقام الزوجية فقط
uares = [i * i for i in range(1, 11) if i % 2 == 0]
print(even_squares)  # [4, 16, 36, 64, 100]
```

ستراها كثيرًا في كود الذكاء الاصطناعي. اعتد عليها مبكرًا.

الحلقات المتداخلة (Nested Loops)

يمكنك وضع حلقة داخل حلقة:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} × {j} = {i * j}")
```

سيطبع جدول الضرب من 1 إلى 3. الحلقة الخارجية تعمل 3 مرات، والداخلية تعمل 3 مرات في كل دورة، فيكون المجموع 9 طبعات.

تحذير: الحلقات المتداخلة تُبطئ البرنامج بسرعة. حلقتان كل منهما 1000 عنصر تنتج مليون عملية. سترى في الفصل 11 كيف نقيس أداء البرنامج.

مثال متكامل: لعبة تخمين

```
import random

secret = random.randint(1, 100)
attempts = 0
max_attempts = 7

print("فكرت برقم بين 1 و 100. لديك 7 محاولات.")

while attempts < max_attempts:
    guess_text = input("خمّن: ")
    guess = int(guess_text)
    attempts += 1

    if guess == secret:
        print(f"أحسنت! خمنت في المحاولة رقم {attempts}")
        break
```

```
        elif guess < secret:
            print("الرقم أكبر")
        else:
            print("الرقم أصغر")
    else:
    print(f"انتهت محاولاتك. الرقم كان {secret}")
```

تأمل البرنامج جيدًا. فيه `while`، شرط `if/elif/else`، استخدام `break`، وحتى `else` على الحلقة. هذا يلخص الفصل كله في 15 سطرًا.

خلاصة الفصل

تعلمت كيف تتخذ قرارات بـ `if/elif/else`، وكيف تكرر بـ `for` و `while`، وكيف تتحكم في الحلقة بـ `break` و `continue`. هذه الأدوات كافية لكتابة معظم منطق البرامج. في الفصل القادم، سنتعلم الدوال — كيف نُغلّف الكود في وحدات قابلة لإعادة الاستخدام.

تمارين الفصل الخامس

1. اكتب برنامجًا يطبع الأرقام من 1 إلى 100، ولكن يستبدل الأرقام التي تقبل القسمة على 3 بكلمة "Fizz"، والأرقام التي تقبل القسمة على 5 بكلمة

"Buzz"، والأرقام التي تقبل القسمة على الاثنين بـ "FizzBuzz" (هذا تمرين كلاسيكي شهير).

2. اكتب برنامجًا يطلب من المستخدم 5 أرقام ويطبع أكبرها وأصغرها.
3. اكتب برنامجًا يطلب من المستخدم كلمة سرّية، ويسمح له بـ 3 محاولات فقط لإدخالها صحيحةً (الكلمة الصحيحة: "yemen2026").
4. باستخدام list comprehension، أنشئ قائمة بمكعّبات (الأس 3) الأرقام من 1 إلى 20 التي تقبل القسمة على 4.

الفصل السادس: الدوال

> "خير الكلام ما قلّ ودلّ. وخير الكود ما تكرر استخدامه."

تخيل أنك تكتب برنامجًا يحسب ضريبة قيمة مضافة على عشر منتجات. هل ستكتب صيغة الضريبة عشر مرات؟ بالتأكيد لا. ستضع الصيغة في "دالة" (function)، وستستدعيها عشر مرات.

الدالة هي قطعة كود لها اسم، تأخذ مدخلات، وتُرجع نتيجة. هذا هو حجر الأساس لأي برنامج جدي.

تعريف دالة

تبدأ الدالة بكلمة `def`، يتبعها اسم الدالة، أقواس، ونقطتان رأسيتان:

```
def greet():
    print("السلام عليكم")
    print("أهلًا بك في برنامجنا")
```

لاستدعاء الدالة:

```
greet()
greet()
greet()
```

سيطبع التحية ثلاث مرات. لاحظ أن الكتابة `def greet():` لا تشغّل الدالة — هي فقط تعرّفها. التشغيل يحدث عند `greet()`.

دوال تأخذ معطيات (Parameters)

أمتى تكون الدالة مفيدة فعلًا؟ حين تتلقى مدخلات تتغير في كل مرة:

```
def greet(name):
    print(f"السلام عليكم يا {name}")

greet("أحمد")      # السلام عليكم يا أحمد
greet("محمد")      # السلام عليكم يا محمد
greet("خالد")      # السلام عليكم يا خالد
```

الكلمة `name` بين القوسين تسمى "معطىً" (parameter). والقيمة "أحمد" التي مررتها عند الاستدعاء تسمى "وسيطًا" (argument). هذا الفرق مهم في الكتب المتقدمة، لكن في الحياة العملية الناس يستخدمون الكلمتين بالتبادل.

دوال تُرجع قيمة

الدالة الأقوى هي التي **تُرجع** نتيجة، فيمكنك استخدامها في عمليات أخرى:

```
:def calculate_tax(price)
    tax_rate = 0.15
    tax = price * tax_rate
    return tax

product_price = 200
tax_amount = calculate_tax(product_price)
total = product_price + tax_amount
print(f"السعر بعد الضريبة: {total}")
```

كلمة `return` تُنهي تنفيذ الدالة وتُعطي القيمة لمن استدعاها. أي سطر بعد `return` لن ينفَّذ.

معطيات متعددة

```
:def calculate_total(price, quantity, tax_rate)
    subtotal = price * quantity
    tax = subtotal * tax_rate
```

```
return subtotal + tax

result = calculate_total(50, 3, 0.15)
print(result)        # 172.5
```

ترتيب الوسائط مهم: السعر أولًا، ثم الكمية، ثم نسبة الضريبة.

معطيات بقيم افتراضية

يمكنك إعطاء معطى قيمة افتراضية:

```
def calculate_total(price, quantity, tax_rate=0.15)
    subtotal = price * quantity
    tax = subtotal * tax_rate
    return subtotal + tax

# يمكنك تجاهل tax_rate (سيستخدم 0.15)
print(calculate_total(50, 3))        # 172.5

# أو تعديلها
print(calculate_total(50, 3, 0.20))   # 180.0
```

وسائط مسماة (Keyword Arguments)

عند الاستدعاء، يمكنك تسمية الوسائط لزيادة الوضوح:

```
alculate_total(price=50, quantity=3, tax_rate=0.10)
```

هذا أوضح بكثير من `calculate_total(50, 3, 0.10)`، خاصة حين تكون الدالة كبيرة وذات معطيات كثيرة. اعتد على هذا في الدوال المهمة.

إرجاع عدة قيم

يمكن لدالة بايثون أن تُرجع عدة قيم في آن واحد، يفصل بينها بفواصل:

```
:def stats(numbers)
ers), max(numbers), sum(numbers) / len(numbers)

scores = [85, 90, 78, 92, 88]
lowest, highest, average = stats(scores)

print(f"الأدنى: {lowest}")
```

```
print(f"الأعلى: {highest}")
print(f"المتوسط: {average:.2f}")
```

ما تُرجعه الدالة هنا هو "tuple" (زوج/ثلاثي مرتب)، وستتعلم عنه في الفصل القادم. الفك على عدة متغيرات يسمى "tuple unpacking".

الـ docstring — توثيق الدالة

الممارسة الجيدة تتطلب أن تشرح كل دالة بنص قصير في أولها. هذا يسمى "docstring":

```
def calculate_tax(price, rate=0.15):
    """احسب مقدار الضريبة على سعر معين.

    المعطيات:
        price: السعر الأصلي بالريال.
        rate: نسبة الضريبة (افتراضيًا 0.15).

    تُرجع:
        مقدار الضريبة بالريال.
    """
    return price * rate
```

استخدم النصوص بثلاثة تنصيصات """. أي محرر بايثون سيعرض هذا التوثيق حين تستخدم الدالة لاحقًا.

النطاق (Scope)

المتغيرات داخل الدالة "محلّية" — لا تراها بقية البرنامج. والمتغيرات خارج الدوال "عامّة" — تراها كل الدوال.

```
total = 100          # متغير عام

def add(x):
    result = x + total      # تستطيع رؤية total
    return result

print(add(5))                # 105
print(result)  # هذا سيعطي خطأ: result
```

نصيحة: لا تعتمد على المتغيرات العامة داخل الدوال. مرّر كل ما تحتاجه عبر المعطيات. هذا يجعل دوالك "نقية" (pure) وسهلة الاختبار.

دوال كقيم — لمحة عن البرمجة الوظيفية

في بايثون، الدوال نفسها قيم — يمكنك تمريرها لدوال أخرى، وحفظها في متغيرات:

```
:def shout(text)
()return text.upper

:def whisper(text)
()return text.lower

# نحفظ الدالة في متغير
my_function = shout
print(my_function("hello"))      # HELLO

# نمرر الدالة لدالة أخرى
:def process(text, transformer)
return transformer(text)

print(process("Hello", shout))     # HELLO
print(process("Hello", whisper))   # hello
```

هذا النمط أساسي حين نستخدم مكتبات الذكاء الاصطناعي، حيث نمرر "callbacks" لمعالجة كل قطعة من ردّ النموذج.

الدوال المجهولة (Lambda)

أحيانًا تحتاج إلى دالة صغيرة جدًا لا تستحق `def` كاملاً. استخدم `lambda`:

```
# الطريقة العادية
def square(x):
    return x * x

# الطريقة المجهولة
square = lambda x: x * x

print(square(5))     # 25
```

الاستخدام الشائع جدًا للـ lambda هو في دوال التصنيف:

```
people = [
{"name": "أحمد", "age": 30},
{"name": "محمد", "age": 25},
{"name": "خالد", "age": 35},
```

```
[

# رتّب حسب العمر
people.sort(key=lambda p: p["age"])
print(people)
# [{'name': 'محمد', 'age': 25} ,{'name': 'أحمد', '0
```

args و kwargs*

أحيانًا لا تعرف عدد الوسائط مسبقًا. استخدم *args لاستقبال عدد متغير من الوسائط الترتيبية، و **kwargs للوسائط المسماة:

```
def sum_all(*numbers):
    total = 0
    for n in numbers:
        total += n
    return total

print(sum_all(1, 2, 3))          # 6
print(sum_all(1, 2, 3, 4, 5))    # 15
```

```
:def show_info(**details)
:()for key, value in details.items
print(f"{key}: {value}")

("صنعاء"=age=30, city ,"أحمد"=name)show_info
```

ستراها كثيرًا في توثيق المكتبات، خاصة في وظائف مثل `requests.get()` التي تقبل عشرات الوسائط الاختيارية.

استيراد الدوال من ملفات أخرى

أنت لست مضطرًا لكتابة كل الدوال في ملف واحد. ضع دوالك المهمة في ملف منفصل، ثم استورده:

ملف `math_helpers.py`:

```
:def calculate_tax(price, rate=0.15)
return price * rate

:def calculate_total(price, quantity)
return price * quantity
```

ملف `main.py`:

```
math_helpers import calculate_tax, calculate_total

t = calculate_total(50, 3)
tx = calculate_tax(t)
print(t + tx)
```

أو استورد الكل دفعةً واحدةً:

```
import math_helpers

t = math_helpers.calculate_total(50, 3)
tx = math_helpers.calculate_tax(t)
```

هذا ما يسمح ببناء مشاريع كبيرة من قطع صغيرة قابلة لإعادة الاستخدام.

مثال متكامل: مكتبة دوال نصية

```
def word_count(text):
    """يحسب عدد الكلمات في نص."""
    return len(text.split())

def is_palindrome(text):
```

```
    """يتحقق إن كان النص متناظرًا."""
    cleaned = text.replace(" ", "").lower()
    return cleaned == cleaned[::-1]

def truncate(text, max_length=50):
    """يقتطع النص ويضع نقاطًا في النهاية."""
    if len(text) <= max_length:
        return text
    return text[:max_length] + "..."

# استخدام
sentence = "بايثون لغة برمجة سهلة وقوية"
print(word_count(sentence))  # 5
print(is_palindrome("سلسل"))  # True
print(truncate("نص طويل جدًا " * 10))  # نص طويل جدً
```

خلاصة الفصل

الدوال هي حجر الأساس للكود القابل لإعادة الاستخدام. تعلمت كيف تعرّف دالة، كيف تمرر لها معطيات، كيف ترجّع نتائج، وكيف تنظمها في ملفات منفصلة. كل

سطر بايثون جدي ستكتبه لاحقًا سيكون داخل دالة. في الفصل القادم، سنغوص في هياكل البيانات الأساسية — القوائم والقواميس والمجموعات.

تمارين الفصل السادس

1. اكتب دالة `temperature_to_fahrenheit(c)` تأخذ درجة حرارة مئوية وتُرجع المكافئ بالفهرنهايت (الصيغة: F = C × 9/5 + 32).
2. اكتب دالة `arabic_letter_count(text)` تحسب عدد الحروف العربية فقط في نص (تجاهل الأرقام والمسافات وعلامات الترقيم).
3. اكتب دالة `safe_divide(a, b)` تقسم رقمين، وترجّع `None` إذا كان `b` صفرًا.
4. اكتب دالة `format_name(first, last, title="السيد")` تركّب اسمًا كاملاً مع لقب اختياري.

الفصل السابع: هياكل البيانات

> "نظّم بياناتك يَنتَظِم تفكيرك."

كل برنامج جدي يتعامل مع مجموعات من البيانات، لا قِيَم منفردة. قائمة طلاب، قاموس درجاتهم، مجموعة بدول مختلفة من الكتب. هذا الفصل يغطي أربع هياكل أساسية في بايثون: القائمة، القاموس، المجموعة، والـ tuple. كلها ستظهر في كل سطر من كود الذكاء الاصطناعي ستكتبه لاحقًا.

القائمة (List)

القائمة هي مجموعة مرتبة من العناصر، يُمكن تعديلها. تُكتب بين أقواس مربعة:

```
fruits = ["تفاح", "موز", "برتقال"]
numbers = [10, 20, 30, 40]
mixed = [1, "نص", 3.14, True]      # يمكن خلط الأنو
empty = []
```

الوصول إلى عنصر

العناصر مفهرسة من 0:

```
fruits = ["تفاح", "موز", "برتقال"]
print(fruits[0])      # تفاح
print(fruits[1])      # موز
print(fruits[2])      # برتقال
print(fruits[-1])     # برتقال (آخر عنصر)
print(fruits[-2])     # موز
```

التقطيع (Slicing)

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[0:3])      # [10, 20, 30]  من 0 إلى ما
print(nums[2:])       # [30, 40, 50, 60]  من 2 إلى
print(nums[:4])       # [10, 20, 30, 40]  من البداية
print(nums[::2])      # [10, 30, 50]  خطوة 2
print(nums[::-1])     # [60, 50, 40, 30, 20, 10]  مع
```

تعديل القائمة

```
fruits = ["تفاح", "موز", "برتقال"]

# تغيير عنصر
fruits[1] = "مانجو"
print(fruits)          # ['تفاح', 'مانجو', 'برتقال']

# إضافة في النهاية
fruits.append("عنب")
print(fruits)          # ['تفاح', 'مانجو', 'برتقال', 'عنب']

# إضافة في موقع معين
fruits.insert(0, "فراولة")
print(fruits)          # ['فراولة', 'تفاح', 'مانجو', 'برتقال', 'عنب']

# حذف بالقيمة
fruits.remove("مانجو")
print(fruits)          # ['فراولة', 'تفاح', 'برتقال', 'عنب']

# حذف بالموقع
```

```
removed = fruits.pop(0)
print(removed)          # فراولة
print(fruits)           # ['تفاح', 'برتقال', 'عنب']

# إضافة قائمة كاملة
more = ["خوخ", "كرز"]
fruits.extend(more)
print(fruits)           # ['تفاح', 'برتقال', 'عنب', '
```

دوال مفيدة على القوائم

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

print(len(nums))        # 8 (عدد العناصر)
print(sum(nums))        # 31 (المجموع)
print(min(nums))        # 1
print(max(nums))        # 9
print(sorted(nums))     # [1, 1, 2, 3, 4, 5, 6, 9] (ن

nums.sort()             # ترتيب القائمة الأصلية
nums.reverse()          # عكس الترتيب
```

```
print(nums.count(1))  # 2 (عدد مرات تكرار 1)
print(nums.index(5))  # 4 (موقع 5)
```

التحقق من العضوية

```
fruits = ["تفاح", "موز", "برتقال"]
if "تفاح" in fruits:
    print("نعم لدينا تفاح")
```

القاموس (Dictionary)

القاموس هو مجموعة من أزواج "مفتاح: قيمة" (key: value). كأنه قاموس عربي حقيقي: تبحث عن كلمة (المفتاح) فتجد معناها (القيمة).

```
student = {
    "name": "أحمد",
    "age": 22,
    "city": "صنعاء",
    "grade": 88.5,
}
```

```
# الوصول
print(student["name"])          # أحمد
print(student["age"])           # 22

# التعديل
student["grade"] = 92.0
student["email"] = "ahmed@example.com"      # إضافة

# الحذف
del student["city"]

# التحقق من المفتاح
if "name" in student:
    print("لدينا اسمه")

# الوصول الآمن
phone = student.get("phone")            # None إن ل
phone = student.get("phone", "غير معروف")  # قيمة
```

الدوران على القاموس

```
student = {"name": "أحمد", "city": "صنعـ" ,"age": 22}

# المفاتيح فقط
for key in student:
    print(key)

# المفاتيح والقيم
for key, value in student.items():
    print(f"{key}: {value}")

# القيم فقط
for value in student.values():
    print(value)
```

القواميس المتداخلة

في الذكاء الاصطناعي، ستراها كثيرًا — ردود APIs غالبًا ما تكون قواميس متداخلة:

```
} = response
,"model": "claude-sonnet-4-6"
} :"usage"
,input_tokens": 150"
,output_tokens": 320"
,{
] :"content"
,{"مرحبًا" :"type": "text", "text"}
,[
{
```

```
print(response["usage"]["input_tokens"])    # 150
print(response["content"][0]["text"])        # مرحبً
```

اعتد على هذا التداخل. كل API لنموذج لغوي يُعيد بنية مشابهة.

المجموعة (Set)

المجموعة هي مجموعة من العناصر **الفريدة**، غير مرتبة. تُكتب بين أقواس مجعدة:

```
fruits = {"تفاح", "موز", "برتقال", "تفاح"}    # الـ
print(fruits)          # {'تفاح', 'موز', 'برتقال'}

# الإضافة والحذف
fruits.add("عنب")
fruits.remove("موز")

# التحقق
if "تفاح" in fruits:
    print("نعم")
```

عمليات المجموعات الرياضية

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)          # الاتحاد: {1, 2, 3, 4, 5, 6}
print(a & b)          # التقاطع: {3, 4}
print(a - b)          # الفرق: {1, 2}
print(a ^ b)          # الفرق المتماثل: {1, 2, 5, 6
```

استخدام شائع جدًا للمجموعة: حذف التكرارات من قائمة:

```
words = ["نعم", "لا", "نعم", "ربما", "نعم", "لا"]
unique_words = list(set(words))
print(unique_words)  # ['نعم', 'لا', 'ربما']
```

الـ tuple

شبيهة بالقائمة، لكن **لا يمكن تعديلها** بعد إنشائها. تُكتب بأقواس عادية:

```
point = (3, 5)
colors = ("أحمر", "أخضر", "أزرق")
single = (42,)  # tuple بعنصر واحد، لاحظ الفا
empty = ()
```

تستخدم الـ tuple حين تريد بيانات لا تتغير، مثل إحداثيات نقطة، أو مكوّنات لون RGB. وكذلك تُرجع كثير من دوال بايثون tuples (مثل divmod التي تُعيد الناتج والباقي معًا).

```
result = divmod(17, 5)
print(result)  # (3, 2)

# tuple unpacking
quotient, remainder = divmod(17, 5)
```

```
print(quotient)        # 3
print(remainder)       # 2
```

أيهما أستخدم متى؟

اختيار الهيكل الصحيح أهم من تعلم استخدامه:

استخدم list حين تحتاج إلى قائمة مرتبة من العناصر يمكنك تعديلها (إضافة، حذف، تغيير ترتيب).

استخدم dict حين تحتاج إلى ربط مفاتيح بقيم (اسم بعمر، كلمة بمعنى، رقم منتج بسعره).

استخدم set حين تهتم بوجود عنصر فقط، ولا يهمك الترتيب، وتريد منع التكرار.

استخدم tuple حين تريد بيانات ثابتة لا يجب تعديلها (إحداثيات، أبعاد، تواريخ).

التداخل والدمج

الواقع غير مرتب. ستحتاج كثيرًا إلى قائمة من القواميس، أو قاموس قيمه قوائم:

```
# قائمة من القواميس (كل صف بيانات)
students = [
```

```
,{grade": 88" ,"أحمد" :"name"}
,{grade": 92" ,"محمد" :"name"}
,{grade": 75" ,"خالد" :"name"}
[

# الدوران
:for s in students
print(f"{s['name']}: {s['grade']}")

# الترتيب حسب الدرجة (تنازليًا)
d(students, key=lambda s: s["grade"], reverse=True)
print(sorted_students)

# قاموس قيمه قوائم
} = courses
"بايثون": ["أحمد", "محمد"],
"ذكاء اصطناعي": ["خالد", "أحمد"],
"قواعد بيانات": ["فاطمة"],
{

# طباعة كل دورة وطلابها
```

```
:()for course, names in courses.items
print(f"{course}: {', '.join(names)}")
```

القواميس الاستيعابية (Dict Comprehension)

كما لدينا list comprehension، لدينا dict comprehension:

```
# قاموس يربط الرقم بمربعه
squares = {n: n * n for n in range(1, 6)}
(squares)        # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# قاموس مفاتيحه نصوص وقيمه أعمارها
names = ["أحمد", "محمد", "خالد"]
lengths = {name: len(name) for name in names}
print(lengths)        # {'أحمد': 4, 'محمد': 4, 'خال
```

مثال متكامل: محلل إحصائي بسيط

```
} = scores
"أحمد": 88,
"محمد": 92,
```

```
    "خالد": 75,
    "فاطمة": 95,
    "مريم": 80,
}

# حساب الإحصائيات
all_scores = list(scores.values())
average = sum(all_scores) / len(all_scores)
top_student = max(scores, key=scores.get)
{name: s for name, s in scores.items() if s >= 80}

print(f"عدد الطلاب: {len(scores)}")
print(f"المتوسط: {average:.2f}")
print(f"الأفضل: {top_student} بدرجة {scores[top_student]}
print(f"الناجحون: {passing}")
```

خلاصة الفصل

أربع هياكل أساسية تشكّل أساس كل تطبيق بايثون: القائمة للترتيب، القاموس للربط، المجموعة للتفرّد، الـ tuple للثبات. إتقانها سيرفعك من مبتدئ إلى متمكن.

في الفصل القادم، سنخرج من ذاكرة البرنامج إلى الملفات على القرص، ونتعلم كيف نقرأ ونكتب — بما في ذلك الملفات العربية بترميز UTF-8.

تمارين الفصل السابع

1. أنشئ قائمة فيها 10 درجات لطلاب. اكتب كودًا يطبع: المتوسط، أعلى درجة، أقل درجة، عدد الناجحين (>= 60).
2. أنشئ قاموسًا يضم 5 طلاب وأعمارهم. اكتب كودًا يجد الطالب الأكبر سنًا.
3. خذ قائمتين فيهما طلاب، واكتب كودًا يطبع: الأسماء المشتركة بينهما، الأسماء في الأولى فقط، كل الأسماء بدون تكرار.
4. حوّل القاموس التالي إلى قائمة من tuples: `{"أحمد": 88, "محمد": 92, "خالد": 75}`.

الفصل الثامن: قراءة الملفات وكتابتها

> "الذاكرة تنسى، والملف يحفظ."

كل ما تعلمناه حتى الآن يعيش في ذاكرة البرنامج، ويختفي عند إغلاقه. لكن البرامج الحقيقية تتعامل مع ملفات: تقرأ مستندًا، تستخرج بيانات منه، تكتب نتائج جديدة. في الذكاء الاصطناعي خاصةً، ستحمّل آلاف المستندات النصية، وتعالجها، وتحفظ النتائج. هذا الفصل يعلمك الأساسيات.

فتح ملف وقراءته

أبسط طريقة لقراءة ملف نصي:

```
file = open("students.txt", "r", encoding="utf-8")
content = file.read()
print(content)
file.close()
```

شرح كل جزء: - open() : تفتح الملف. - "students.txt": اسم الملف. يمكن أن يكون مسارًا كاملًا. - "r": وضع الفتح (r للقراءة). - encoding="utf-8": ترميز الملف. **حاسم** للنصوص العربية. لا تنسَه أبدًا. - file.read() : تقرأ كل محتوى الملف كنص واحد. - file.close() : تُغلق الملف. ضرورية لتحرير الموارد.

الطريقة الموصى بها: with

نسيان close() يسبب مشاكل. الطريقة الأنيقة هي استخدام with، الذي يُغلق الملف تلقائيًا:

```
open("students.txt", "r", encoding="utf-8") as file
content = file.read()

print(content)          # هنا الملف مُغلق بالفعل
```

استخدم with دائمًا. هي الطريقة الموصى بها رسميًا.

أوضاع فتح الملف

الوضع المعنى

"r" قراءة فقط (افتراضي). يفشل إن لم يكن الملف موجودًا.

"w" كتابة. **ينشئ ملفًا جديدًا أو يمحو الموجود.**

"a" إلحاق. يكتب في نهاية الملف بدون مسح.

"r+" قراءة وكتابة.

"rb" قراءة ثنائية (للصور والملفات غير النصية).

تحذير: "w" خطير. يُهلك الملف الموجود فورًا بدون سؤال. تحقق دائمًا من اسم الملف قبل استخدام هذا الوضع.

كتابة ملف

```
n open("output.txt", "w", encoding="utf-8") as file
    file.write("السطر الأول\n")
    file.write("السطر الثاني\n")
    file.write("السطر الثالث\n")
```

\n هي رمز "السطر الجديد". أي محرر نصوص سيعرضها كانتقال سطر فعلي.

لكتابة عدة سطور دفعةً واحدةً، استخدم writelines:

```
["n\خالد" ,"n\محمد" ,"n\أحمد"] = lines
th open("names.txt", "w", encoding="utf-8") as file
                    file.writelines(lines)
```

قراءة سطرًا سطرًا

ملفات الذكاء الاصطناعي قد تكون ضخمة (مليون سطر). تحميلها كلها بـ read() يستهلك ذاكرة هائلة. الأفضل قراءتها سطرًا سطرًا:

```
open("big_file.txt", "r", encoding="utf-8") as file
                    :for line in file
 من n\ تحذف print(line.strip())     # strip
```

هذا الكود يعمل حتى على ملف بحجم 10 جيجابايت، لأنه يحمّل سطرًا واحدًا في الذاكرة في كل مرة.

القراءة في قائمة

إذا أردت كل السطور كقائمة:

```
open("students.txt", "r", encoding="utf-8") as file
lines = file.readlines()

# lines الآن قائمة، كل عنصر سطر (مع \n في النهاية)
for line in lines:
    print(line.strip())
```

الإلحاق على ملف موجود

```
with open("log.txt", "a", encoding="utf-8") as file
    file.write("حدث جديد: تسجيل دخول\n")
```

يضيف هذا الكود سطرًا جديدًا في نهاية `log.txt` بدون مسح ما قبله. هذه عادة محبوبة لملفات السجل (logs).

النصوص العربية والترميز UTF-8

النصوص العربية تحتاج ترميز خاص لتُكتب وتُقرأ بشكل صحيح. الترميز المعياري الذي يدعم العربية وكل لغات العالم هو **UTF-8**. كل أمر `open` يجب أن يحتوي `encoding="utf-8"`.

ماذا يحدث إن نسيت؟ قد يعمل الكود على Linux و macOS (التي تستخدم UTF-8 افتراضيًا)، لكنه يفشل على Windows (الذي يستخدم cp1252 افتراضيًا) وستحصل على خطأ مرعب اسمه `UnicodeDecodeError`.

اكتب `encoding="utf-8"` في كل مرة. اعتد عليه عادةً ثابتة.

```
# مثال شامل لقراءة ملف عربي
th open("poem.txt", "r", encoding="utf-8") as file
()poem = file.read

("{len(poem)} :عدد الحروف"f)print
("{poem.count(chr(10)) + 1} :عدد الأسطر"f)print
("{len(poem.split())} :عدد الكلمات"f)print
```

فحص وجود الملف

قبل القراءة، قد تريد التأكد من وجود الملف:

```
import os

:if os.path.exists("data.txt")
th open("data.txt", "r", encoding="utf-8") as f
print(f.read())
```

```
:else
    print("الملف غير موجود")
```

أو الطريقة الحديثة بـ pathlib:

```
from pathlib import Path

file = Path("data.txt")
if file.exists():
    print(file.read_text(encoding="utf-8"))
```

pathlib أنظف وأقوى. سترى كثيرًا في الكود الحديث:

```
from pathlib import Path

# قراءة بسطر واحد
text = Path("poem.txt").read_text(encoding="utf-8")

# كتابة بسطر واحد
Path("output.txt").write_text("محتوى جديد", encoding="utf-8")

# الدوران على كل ملف نصي في مجلد
```

```
:for file in Path("documents").glob("*.txt")
print(file.name)
print(file.read_text(encoding="utf-8")[:100])
```

التعامل مع المسارات

المسارات تختلف بين Windows و Linux: Windows يستخدم \،
والآخرون /. pathlib يحل هذا تلقائيًا:

```
from pathlib import Path

# أنت تكتب هكذا
path = Path("documents") / "students" / "ahmed.txt"

# pathlib يحوّلها للنظام الصحيح
print(path)
# على Linux: documents/students/ahmed.txt
# على Windows: documents\students\ahmed.txt
```

إنشاء مجلدات

```
from pathlib import Path

# إنشاء مجلد جديد
Path("output").mkdir(exist_ok=True)

# إنشاء سلسلة مجلدات
t/2026/january").mkdir(parents=True, exist_ok=True)
```

`exist_ok=True` تمنع الخطأ إن كان المجلد موجودًا.

`parents=True` تنشئ كل المجلدات الأبوية.

نسخ، حذف، نقل

```
import shutil
from pathlib import Path

# نسخ
shutil.copy("source.txt", "destination.txt")
```

```
# نقل
shutil.move("old_name.txt", "new_name.txt")

# حذف
()Path("file_to_delete.txt").unlink
```

تحذير: `unlink` تحذف الملف فورًا، بدون سؤال، وبدون سلة محذوفات. تأكد جيدًا قبل تشغيلها.

مثال متكامل: تنظيف ملف نصي عربي

تخيّل أن لديك ملفًا فيه آلاف الأسطر، بعضها فارغ، وبعضها يحتوي مسافات زائدة. اكتب برنامجًا ينظفه:

```
from pathlib import Path

input_file = Path("messy.txt")
output_file = Path("clean.txt")

# قراءة
text = input_file.read_text(encoding="utf-8")
```

```
# تـنظيف
lines = text.split("\n")
[] = clean_lines
:for line in lines
    ()line = line.strip        # احذف الـمسافـات مـن ال
    :if line                   # تـجاهل الأسطر الـفـارغة
        clean_lines.append(line)

# كـتـابـة
output_text = "\n".join(clean_lines)
tput_file.write_text(output_text, encoding="utf-8")

print(f"حُذفـت {len(lines) - len(clean_lines)} سطرًا ف
print(f"الـمـتـبـقـي: {len(clean_lines)} سطرًا")
```

هذا النوع من التنظيف هو أول خطوة في كل خط أنابيب RAG. تحتاج إلى تنظيف الملفات قبل تقطيعها إلى قطع وتحويلها إلى متجهات.

الجسر إلى كتاب RAG

في الكتاب القادم، ستتعلم بناء "خطوط أنابيب الاستيعاب" (ingestion pipelines). كل خط أنابيب يبدأ بـ:

1. قراءة ملفات من مجلد (هذا الفصل).
2. استخراج النص (سواء كان txt، PDF، Word).
3. تنظيفه (هذا الفصل).
4. تقطيعه إلى قطع (chunks).
5. تحويل كل قطعة إلى متجه عبر API (الفصل 13).
6. حفظ المتجهات في قاعدة بيانات.

كل سطر تعلمته هنا سيكون لبنة في تلك الخطوط. اعتد على `pathlib` و `with open` الآن، وستجدها صديقاتك القريبات لاحقًا.

خلاصة الفصل

تعلمت قراءة الملفات وكتابتها بطريقتين: التقليدية (`open` مع `with`) والحديثة (`pathlib`). تعلمت أهمية الترميز UTF-8 للنصوص العربية. تعلمت كيف تتعامل مع المسارات بطريقة محايدة عن نظام التشغيل. كل هذه مهارات أساسية

لأي عمل بياني. في الفصل القادم، سنتعمق في صيغتي JSON و CSV — أهم صيغتين ستتعامل معهما في الذكاء الاصطناعي.

تمارين الفصل الثامن

1. اكتب برنامجًا يقرأ ملفًا نصيًا عربيًا، ويطبع: عدد الأسطر، عدد الكلمات، أطول كلمة فيه.
2. اكتب برنامجًا يأخذ ملفًا نصيًا، ويُنشئ نسخة منه بكل الأسطر معكوسة الترتيب.
3. اكتب برنامجًا يبحث في كل ملفات `.txt` في مجلد عن كلمة معينة، ويطبع أسماء الملفات التي تحتوي عليها.
4. اكتب برنامجًا "ملف سجل" (logger): دالة تستقبل نصًا، وتضيفه إلى ملف `log.txt` مع طابع الوقت في بدايته.

الفصل التاسع: التعامل مع JSON و CSV

> "البيانات بدون شكل فوضى. والشكل بدون بيانات هيكل أجوف."

في عالم الذكاء الاصطناعي، 90٪ من البيانات التي ستتعامل معها ستكون بإحدى صيغتين: JSON أو CSV. كل API لنموذج لغوي يُعيد JSON. كل ملف بيانات تنزّله من Kaggle أو من إدارة جامعتك على الأرجح CSV. إتقان هاتين الصيغتين ضروري.

ما هو JSON؟

JSON (JavaScript Object Notation) صيغة لكتابة البيانات المهيكلة كنص. تشبه قواميس بايثون كثيرًا:

```
}
"name": "أحمد",
"age": 25,
"is_student": true,
```

```
"courses": ["بايثون", "ذكاء اصطناعي"],
"address": {
"city": "صنعاء",
"street": "السبعين"
}
}
```

أوجه الشبه مع قاموس بايثون: أزواج مفتاح-قيمة، الأقواس المجعدة، الأقواس المربعة للقوائم.

الاختلافات المهمة: - المفاتيح في JSON يجب أن تكون نصوصًا بين علامتي تنصيص. - `True/False/None` في بايثون تكتب `true/false/null` في JSON (بأحرف صغيرة). - لا فواصل بعد آخر عنصر (بايثون يسمح بها، JSON لا).

مكتبة json في بايثون

بايثون يأتي بمكتبة `json` جاهزة. لا تحتاج إلى تثبيتها:

```
import json
```

الدالتان الأهم: - `json.loads(text)`: تحوّل نص JSON إلى قاموس بايثون. - `json.dumps(obj)`: تحوّل قاموس بايثون إلى نص JSON.

```
import json

# نص JSON إلى قاموس
text = '{"name": "أحمد", "age": 25}'
data = json.loads(text)
print(data["name"])          # أحمد
print(type(data))            # <class 'dict'>

# قاموس إلى نص JSON
person = {"name": "محمد", "city": "عدن", "age": 30}
text = json.dumps(person)
print(text)
# {"name": "محمد", ...}
```

انتبه! بشكل افتراضي، `json.dumps` يُحوّل الحروف العربية إلى ترميز Unicode (م). لمنع ذلك:

```
text = json.dumps(person, ensure_ascii=False)
print(text)
# {"name": "محمد", "city": "عدن", "age": 30}
```

`ensure_ascii=False` تخبر بايثون أن يحتفظ بالحروف العربية كما هي. اعتد على إضافتها في كل JSON عربي.

قراءة وكتابة ملفات JSON

```
import json

# كتابة قاموس إلى ملف
data = {
"students": [
{"name": "أحمد", "grade": 88},
{"name": "محمد", "grade": 92},
{"name": "خالد", "grade": 75},
],
"course": "بايثون للذكاء الاصطناعي",
"year": 2026,
{
```

```
n open("students.json", "w", encoding="utf-8") as f
son.dump(data, f, ensure_ascii=False, indent=2)

# قراءة من ملف
n open("students.json", "r", encoding="utf-8") as f
loaded = json.load(f)

print(loaded["course"])
:for s in loaded["students"]
print(f"{s['name']}: {s['grade']}")
```

فروق مهمة: `json.dumps` و `json.loads` تتعاملان مع **نصوص**.

`json.dump` و `json.load` (بدون s) تتعاملان مع **ملفات مفتوحة**.

`indent=2` تنسّق الناتج بمسافات جميلة، فيصبح قابلًا للقراءة. بدونها، يصبح كل JSON في سطر واحد.

أنواع البيانات المدعومة

JSON تدعم: نصوص، أرقام، قوائم، قواميس، true/false/null. **لا تدعم:** التاريخ والوقت مباشرة، tuple (تتحول لقائمة)، set (تعطي خطأ)، أي صنف مخصص.

لو حاولت حفظ شيء غير مدعوم:

```
import json

data = {"date": datetime.now()}
!json.dumps(data)              # TypeError
```

الحل: تحويلها إلى نص قبل الحفظ.

```
data = {"date": datetime.now().isoformat()}
json.dumps(data)              # ينجح
```

ما هو CSV؟

CSV (Comma-Separated Values) صيغة بسيطة للبيانات الجدولية. كل سطر صف، والقيم مفصولة بفواصل:

```
name,age,city
أحمد,25,صنعاء
محمد,30,عدن
خالد,22,تعز
```

السطر الأول عادةً يحوي أسماء الأعمدة (header). أي ملف Excel يمكن حفظه بصيغة CSV.

مكتبة csv في بايثون

```
import csv

# قراءة CSV
with open("students.csv", "r", encoding="utf-8") as f
    reader = csv.reader(f)
    for row in reader:
        print(row)
# ['name', 'age', 'city']
# ['أحمد', '25', 'صنعاء']
# ...
```

كل صف هو قائمة. الـ header هو الصف الأول.

القراءة كقواميس (الأفضل)

`DictReader` تستخدم سطر الـ header كمفاتيح:

```
import csv

th open("students.csv", "r", encoding="utf-8") as f
reader = csv.DictReader(f)
:for row in reader
print(row["name"], row["age"])
```

أنظف بكثير. لا تحتاج إلى تذكّر ترتيب الأعمدة.

كتابة CSV

```
import csv

] = students
,{"صنعاء" :"age": 25, "city" ,"أحمد" :"name"}
,{"عدن" :"age": 30, "city" ,"محمد" :"name"}
,{"تعز" :"age": 22, "city" ,"خالد" :"name"}
[
```

```
utput.csv", "w", encoding="utf-8", newline="") as f
ctWriter(f, fieldnames=["name", "age", "city"])
()writer.writeheader
writer.writerows(students)
```

ملاحظتان مهمتان: - `newline=""`: ضروري على Windows. بدونه ستظهر أسطر فارغة بين الصفوف. - `fieldnames`: ترتيب الأعمدة في الناتج.

التعامل مع CSV عربي وExcel

Excel على Windows يفتح ملفات CSV بترميز Windows-1256 افتراضيًا، لا UTF-8. ستظهر النصوص العربية كرموز غريبة. الحل:

```
# الكتابة مع BOM لتفتح Excel الملف صحيحًا
t.csv", "w", encoding="utf-8-sig", newline="") as f
ctWriter(f, fieldnames=["name", "age", "city"])
()writer.writeheader
writer.writerows(students)
```

`utf-8-sig` يضيف "BOM" (Byte Order Mark) في بداية الملف، وExcel يستخدمه لمعرفة أن الترميز UTF-8.

أو، انصح المستخدم بفتح الملف عبر "Get Data → From Text" في Excel، ويختار UTF-8 يدويًا.

الفواصل المختلفة

ليست كل ملفات "CSV" تستخدم فواصل. بعضها يستخدم نقطة وفاصلة `;` (شائع في أوروبا)، وبعضها Tab `\t` (يسمى TSV):

```
:with open("data.tsv", "r", encoding="utf-8") as f
        reader = csv.reader(f, delimiter="\t")
                        :for row in reader
                            print(row)
```

استخدام pandas (الأقوى للبيانات)

`pandas` مكتبة قوية لتحليل البيانات. تثبتها بـ `pip install pandas`. ثم:

```
import pandas as pd

# قراءة CSV
df = pd.read_csv("students.csv")
print(df.head())          # أول 5 صفوف
print(df.shape)           # (عدد الصفوف, عدد ال
print(df["age"].mean())   # متوسط العمر

# الترشيح
adults = df[df["age"] >= 25]

# الحفظ
sv("adults.csv", index=False, encoding="utf-8-sig")
```

pandas فائقة في البيانات الكبيرة. pd.read_csv تقرأ ملفًا بمليون صف في ثوانٍ. لو كان مشروعك جديًا، استخدم pandas. للأمور البسيطة، مكتبة csv كافية.

مثال متكامل: قراءة JSON من API ثم حفظه CSV

سترى هذا النمط مرارًا في عملك. APIs تُعيد JSON. أنت تستخرج ما يهمك، وتحفظه CSV لتحلله في Excel.

```
import json
import csv

# تخيل أن هذا جاء من API
api_response = """
{
"results": [
{"id": 1, "name": "أحمد", "city": 88, "score"
{"id": 2, "name": "محمد", "city": 92, "score"
{"id": 3, "name": "خالد", "city": 75, "score"
],
"total": 3
}
"""
```

```
# تحليل JSON
data = json.loads(api_response)
students = data["results"]

# تحويل إلى CSV
s.csv", "w", encoding="utf-8-sig", newline="") as f
(f, fieldnames=["id", "name", "score", "city"])
()writer.writeheader
writer.writerows(students)

("حُفظ {len(students)} صفوف.")print(f
```

مثال عكسي: قراءة CSV ثم تحويله JSON

```
import csv
import json

th open("students.csv", "r", encoding="utf-8") as f
reader = csv.DictReader(f)
students = list(reader)
```

```
# تحويل الأرقام (CSV يقرأها كنصوص)
for s in students:
    s["age"] = int(s["age"])

# حفظ كـ JSON
with open("students.json", "w", encoding="utf-8") as f:
    dump(students, f, ensure_ascii=False, indent=2)
```

ملاحظة مهمة: CSV يقرأ كل شيء كنصوص. عليك تحويل الأرقام يدويًا.

الجسر إلى كتاب RAG

في نظام JSON ،RAG هو لغة التواصل بين كل المكونات. ردّ نموذج OpenAI يأتي JSON. ردّ قاعدة Pinecone JSON. الـ metadata الذي تربطه بكل قطعة نصية تخزنه JSON. يجب أن تكون قراءة وكتابة JSON عملية تلقائية لديك.

CSV ستراها أكثر في "تقييم النظام" — تخزّن أسئلة الاختبار وإجاباتها في CSV، وتقرأها سطرًا سطرًا، وترسل كل سؤال للنظام، وتحفظ الإجابة الصحيحة والإجابة الفعلية في CSV نتائج جديد.

خلاصة الفصل

أتقنت أهم صيغتين للبيانات في الذكاء الاصطناعي. تعلمت كيف تقرأ وتكتب JSON بصورة صحيحة (مع `ensure_ascii=False` للعربية)، وكيف تتعامل مع CSV (مع `utf-8-sig` لـ Excel). تعلمت متى تستخدم مكتبة `csv` ومتى ترتقي إلى pandas. في الفصل القادم، سننفتح على شبكة الإنترنت ونتعلم كيف نطلب البيانات من APIs.

تمارين الفصل التاسع

1. اكتب برنامجًا ينشئ ملف JSON يحوي 5 طلاب (اسم، عمر، درجة، مدينة)، ثم يقرؤه ويطبع المتوسط.
2. اكتب برنامجًا يقرأ ملف CSV فيه أسماء وأعمار، ويفصل الناجحين (=> 60) في ملف، والراسبين في ملف آخر.
3. حوّل ملف JSON يحوي قائمة من الكتب (لكل كتاب: عنوان، مؤلف، سنة) إلى ملف CSV.
4. اقرأ ملف CSV عربي بـ pandas، رتّبه تنازليًا حسب عمود الدرجة، وأظهر أعلى 10 طلاب.

الفصل العاشر: طلبات HTTP ومكتبة requests

> "البرنامج المعزول جزيرة. والبرنامج المتصل قارة."

كل تطبيق ذكاء اصطناعي يتحدث مع خوادم بعيدة. ترسل سؤالًا إلى نموذج Anthropic، تأخذ بيانات من Wikipedia، تستعلم قاعدة بيانات متجهية. هذا التواصل يحدث عبر بروتوكول HTTP، ومكتبة `requests` في بايثون هي أداتك الذهبية للتعامل معه.

ما هو HTTP؟

HTTP هو لغة المحادثة بين الحواسيب على الإنترنت. عميل (مثل برنامجك) يرسل "طلبًا" (request) إلى خادم (server). الخادم يرد بـ "استجابة" (response).

كل طلب له: - **عنوان** (URL): مثل `https://api.example.com/users/123` - **طريقة** (method): GET أو

POST أو PUT أو DELETE. - **رؤوس** (headers): بيانات إضافية مثل المصادقة. - **جسم** (body): البيانات المُرسلة (في POST).

كل استجابة لها: - **رمز حالة** (status code): 200 يعني نجح، 404 يعني غير موجود، 500 يعني خطأ في الخادم. - **رؤوس**. - **محتوى**: غالبًا JSON.

تثبيت requests

```
pip install requests
```

ثم في الكود:

```
import requests
```

أول طلب GET

```
import requests

quests.get("https://api.github.com/users/torvalds")
print(response.status_code)        # 200
print(response.json())             # قاموس بايثون م
```

شرح: - `requests.get(url)`: يرسل طلب GET (لجلب بيانات). -

`response.status_code`: رمز الحالة. 200 يعني نجح. -

`response.json()`: يحوّل الرد JSON إلى قاموس بايثون.

التعامل مع المعطيات (Query Parameters)

أحيانًا تحتاج إلى تمرير معطيات في URL. بدلًا من بنائها يدويًا، استخدم `params`:

```
import requests

# بدلًا من: https://api.example.com/search?q=python&limit=10
response = requests.get(
    "https://api.example.com/search",
    params={"q": "python", "limit": 10}
)
```

`requests` يبني URL لك ويعالج الترميز (encoding) للحروف الخاصة.

رؤوس (Headers)

كثير من APIs تتطلب رؤوسًا للمصادقة:

```
import requests

headers = {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json",
    "User-Agent": "LLM4Yemen-Student/1.0",
}

response = requests.get(
    "https://api.example.com/data",
    headers=headers
)
```

سترى هذا النمط في كل API لنماذج الذكاء الاصطناعي.

طلب POST

POST يُستخدم لإرسال بيانات (مثل إنشاء سجل جديد، أو إرسال سؤال لنموذج لغوي):

```
import requests
```

```
} = data
"name": "أحمد",
"email": "ahmed@example.com",
"age": 25"
{

response = requests.post(
"https://api.example.com/users",
# json=data ترسل البيانات كـ JSON
)

print(response.status_code)
print(response.json())
```

`json=data` يُحوّل القاموس إلى JSON ويضيف الـ Content-Type المناسب تلقائيًا.

التحقق من حالة الاستجابة

```
import requests
```

```
onse = requests.get("https://api.example.com/data")

:if response.status_code == 200
()data = response.json
print(data)
:elif response.status_code == 404
print("المورد غير موجود")
:elif response.status_code == 401
print("غير مصرّح — تحقق من مفتاح API")
:elif response.status_code == 429
print("تجاوزت الحد المسموح — انتظر")
:else
("{response.status_code} :خطأ غير متوقع"f)print
```

أو بطريقة أنظف:

```
response = requests.get(url)
()response.raise_for_status  # يرمي خطأ إن لم
()data = response.json
```

الـ Timeout — حماية برنامجك

إن لم يجب الخادم، سيعلق برنامجك إلى الأبد. ضع دائمًا مهلة زمنية:

```
import requests

try:
    response = requests.get(
        "https://api.example.com/data",
        timeout=10  # 10 ثوانٍ
    )
except requests.Timeout:
    print("الخادم بطيء جدًا. حاول لاحقًا.")
```

اعتد على إضافة timeout= في كل طلب. هذه عادة المبرمجين المحترفين.

التعامل مع أخطاء الشبكة

```
import requests

try:
get("https://api.example.com/data", timeout=10)
```

```
()response.raise_for_status
()data = response.json
:except requests.Timeout
print("انتهت المهلة")
:except requests.ConnectionError
print("لا يمكن الاتصال بالخادم")
:except requests.HTTPError as e
print(f"رمز خطأ {e}: HTTP")
:except ValueError
print("الرد ليس JSON صالحًا")
```

مثال عملي: API الطقس المجاني

open-meteo يوفر API مجاني بدون مفتاح:

```
import requests

# طقس صنعاء (lat=15.35, lon=44.20)
response = requests.get(
"https://api.open-meteo.com/v1/forecast",
params={
```

```
                    ,latitude": 15.35"
                   ,longitude": 44.20"
             ,"current_weather": "true"
                                  ,{
                          timeout=10
                                   (

                 :if response.status_code == 200
                      ()data = response.json
            weather = data["current_weather"]
{weather['temperature']} :درجة الحرارة"f)print
{weather['windspeed']} :سرعة الرياح"f)print كم/
```

شغّل هذا الكود الآن. هذه أول مرة في حياتك ستصل برنامج بايثون مع خادم حقيقي.

مثال: تنزيل ملف

```
import requests

get("https://example.com/document.pdf", timeout=30)
```

```
:with open("document.pdf", "wb") as f
f.write(response.content)
```

`response.content` يُعطي البيانات كثنائية (bytes). ولذا فتحنا الملف بوضع `"wb"` (write binary).

تنزيل ملفات كبيرة (Streaming)

ملف 1 جيجابايت لا يجب تحميله كله في الذاكرة. استخدم `stream=True`:

```
import requests

)response = requests.get
,"https://example.com/big_file.zip"
,stream=True
timeout=60
(

:with open("big_file.zip", "wb") as f
chunk in response.iter_content(chunk_size=8192)
f.write(chunk)
```

يقرأ هذا الكود الملف بقطع 8 كيلوبايت، ويكتبها على القرص فورًا، فلا يستخدم سوى ذاكرة قليلة.

محاولات إعادة (Retries)

الشبكات تفشل أحيانًا. اكتب دالة تعيد المحاولة:

```
import requests
import time

:def fetch_with_retries(url, max_attempts=3)
:for attempt in range(1, max_attempts + 1)
:try
esponse = requests.get(url, timeout=10)
()response.raise_for_status
()return response.json
sts.Timeout, requests.ConnectionError) as e
("{e} :فشلت {attempt} المحاولة"f)print
:if attempt < max_attempts
mpt)  # backoff exponentially
("فشلت كل المحاولات")raise Exception
```

`time.sleep(2 ** attempt)` تنتظر 2 ثانية، ثم 4، ثم 8. هذا "exponential backoff" — نمط معياري حين يفشل الخادم.

محدودية الاستخدام (Rate Limits)

كل API له حد لعدد الطلبات في الدقيقة. في الذكاء الاصطناعي، سترى رمز 429 ("Too Many Requests") كثيرًا. الحل:

```
import time

response = requests.get(url)
:if response.status_code == 429
= int(response.headers.get("Retry-After", 60))
print(f"انتظار {retry_after} ثانية...")
time.sleep(retry_after)
response = requests.get(url)  # حاول مرة
```

مثال متكامل: جلب وحفظ بيانات

```
import requests
import json
```

```
from pathlib import Path

:def fetch_user_data(username)
"""جلب بيانات مستخدم GitHub وحفظها."""
url = f"https://api.github.com/users/{username}

:try
response = requests.get(url, timeout=10)
()response.raise_for_status
()data = response.json
:except requests.RequestException as e
("{e} :فشل"f)print
return None

# حفظ في ملف
output = Path(f"github_{username}.json")
)output.write_text
n.dumps(data, ensure_ascii=False, indent=2)
"encoding="utf-8
(
return data
```

```
# استخدام
user = fetch_user_data("torvalds")
if user:
    print(f"الاسم: {user['name']}")
    print(f"المتابعون: {user['followers']}")
    print(f"المستودعات: {user['public_repos']}")
```

الجسر إلى كتاب RAG

في كتاب RAG، ستستخدم `requests` لـ:

- استدعاء نموذج التضمين (embedding API) لتحويل النص إلى متجهات.
- استدعاء قاعدة بيانات متجهية (Pinecone، Qdrant Cloud) عبر REST API.
- استدعاء النموذج اللغوي للحصول على إجابة نهائية.

كل سطر من تلك الاستدعاءات سيستخدم نفس الأنماط التي تعلمتها هنا: headers للمصادقة، =json للإرسال، response.json() للقراءة، try/except لمعالجة الأخطاء.

خلاصة الفصل

أتقنت `requests` — أهم مكتبة في عمل الذكاء الاصطناعي العملي. تعلمت GET و POST، رؤوس المصادقة، الـ timeouts، معالجة الأخطاء، إعادة المحاولة، محدودية الاستخدام. في الفصل القادم، سنتعمق في معالجة الأخطاء وتقنيات الـ debugging — كيف تكتشف الخطأ وتحلّه بسرعة.

تمارين الفصل العاشر

1. استخدم API الطقس open-meteo لجلب طقس عدن وتعز والحديدة، واطبع جدولًا فيه: المدينة، الحرارة، سرعة الرياح.
2. استدع API GitHub لجلب أحدث 5 مستودعات للمستخدم `tensorflow`، واطبع: الاسم، عدد النجوم، اللغة.
3. اكتب دالة `safe_get(url)` تحاول الطلب 3 مرات، مع فترة انتظار متزايدة، وترجّع JSON أو None.
4. صمم برنامجًا يجلب نص ويكيبيديا لمقالة معينة عبر API:
   `https://ar.wikipedia.org/api/rest_v1/page/summary/<عنوان>`، ويحفظه في ملف نصي.

الفصل الحادي عشر: معالجة الأخطاء واستراتيجيات الـ Debugging

> "الخطأ ليس عيبًا، بل معلّم."

كل مبرمج في العالم — حتى أمهر مهندس في Anthropic — يقضي نصف وقته يصلح أخطاء، لا يكتب كودًا جديدًا. هذه حقيقة مهنية. الفرق بين المبتدئ والخبير ليس في عدد الأخطاء، بل في سرعة اكتشافها وحلها. هذا الفصل يعلمك المهارة الأهم في حياتك البرمجية.

ثلاثة أنواع من الأخطاء

أخطاء النحو (Syntax Errors): تكتب كودًا مخالفًا لقواعد بايثون. لا يستطيع بايثون حتى تشغيله.

```
print("مرحبا"          # نسيت قوس الإغلاق
SyntaxError: '(' was never closed #
```

أخطاء وقت التشغيل (Runtime Errors): الكود صحيح نحويًا، لكنه يفشل أثناء التنفيذ.

```
x = 10 / 0          # ZeroDivisionError
nums = [1, 2, 3]
print(nums[10])     # IndexError
```

الأخطاء المنطقية (Logic Errors): الكود يعمل بدون أخطاء، لكنه ينتج نتيجة خاطئة. هذه أصعب الأخطاء.

```
def average(nums):
    return sum(nums) / len(nums) + 1     # +1 خطأ م
```

try / except — تجنّب انهيار البرنامج

`try/except` تتيح لك "التقاط" الأخطاء ومعالجتها بدلًا من أن يتوقف البرنامج:

```
try:
    x = int(input("اكتب رقمًا: "))
    print(f"المربع: {x * x}")
except ValueError:
    print("ما أدخلته ليس رقمًا!")
```

إن أدخل المستخدم نصًا، يقفز التحكم إلى except، ويتجنب البرنامج الانهيار.

التقاط أخطاء متعددة

```
try:
    x = int(input("الرقم الأول: "))
    y = int(input("الرقم الثاني: "))
    result = x / y
    print(f"الناتج: {result}")
except ValueError:
    print("الإدخال ليس رقمًا")
except ZeroDivisionError:
    print("لا يمكن القسمة على صفر")
```

التقاط كل الأخطاء

```
try:
    risky_operation()
except Exception as e:
    print(f"حدث خطأ: {e}")
```

Exception تلتقط أي خطأ تقريبًا. as e يُعطيك كائن الخطأ لقراءة تفاصيله.

تحذير: تجنب التقاط Exception بشكل عام. الأفضل أن تلتقط الأخطاء التي تتوقعها فقط، وتترك الأخطاء غير المتوقعة تظهر، فهي تكشف عيوبًا حقيقية في كودك.

else و finally

```
:try
file = open("data.txt", "r", encoding="utf-8")
    content = file.read()
:except FileNotFoundError
    print("الملف غير موجود")
:else
    # ينفّذ فقط إن لم يحدث خطأ
    print("نجحت القراءة")
:finally
    # ينفّذ في كل الأحوال
    print("انتهت المحاولة")
```

finally مفيدة لـ "التنظيف" — إغلاق ملفات، إغلاق اتصالات قواعد بيانات. لكن في بايثون الحديث، with تتولى ذلك تلقائيًا.

رمي الأخطاء بنفسك

أحيانًا أنت تريد أن ترمي خطأً صريحًا:

```
:def calculate_age(birth_year)
    :if birth_year < 1900 or birth_year > 2026
        raise ValueError(f"سنة ميلاد غير منطقية: {birth_year}")
    return 2026 - birth_year

:try
    age = calculate_age(1850)
:except ValueError as e
    print(f"خطأ: {e}")
```

raise ترمي خطأً، وأي try/except فوقها يلتقطه.

تخصيص أنواع الأخطاء

يمكنك إنشاء أنواع أخطاء خاصة بمشروعك:

```
class APIError(Exception):
    """خطأ من واجهة API."""
    pass

class RateLimitError(APIError):
    """تم تجاوز الحد المسموح."""
    pass

def call_api():
    response = requests.get(url)
    if response.status_code == 429:
        raise RateLimitError("تجاوزت الحد")
    if response.status_code != 200:
        raise APIError(f"خطأ {response.status_code}")

try:
    call_api()
except RateLimitError:
    print("انتظر قليلًا")
except APIError as e:
    print(f"فشل API: {e}")
```

سترى هذا النمط في كل مكتبة جدية للذكاء الاصطناعي.

استراتيجيات Debugging — الأدوات الخمسة

1. الـ print debugging

أبسط طريقة وأسرعها. أضف `print` لتعرف ما يحدث:

```
def calculate_total(items):
    print(f"DEBUG: عدد العناصر = {len(items)}")
    total = 0
    for item in items:
        print(f"DEBUG: إضافة {item}")
        total += item
    print(f"DEBUG: المجموع = {total}")
    return total
```

احذف الـ `sprint` بعد إصلاح الخطأ. أو استخدم مكتبة `logging` لتتركها وتطفئها بسهولة.

2. قراءة Traceback بعناية

```
Traceback (most recent call last):
  File "main.py", line 15, in <module>
    result = process(data)
  File "main.py", line 8, in process
    return divide(x, y)
  File "main.py", line 3, in divide
    return a / b
ZeroDivisionError: division by zero
```

اقرأ من **الأسفل**. السطر الأخير يخبرك بالخطأ. السطور فوقه تخبرك بمسار الاستدعاء — أين ابتدأ، وأين فشل أخيرًا. هذا "stack trace".

3. عزل الخطأ

عند ظهور خطأ، اسأل: ما أصغر كود يعيد إنتاج الخطأ؟ احذف كل ما لا علاقة له. كلما قل الكود الذي تنظر إليه، أسرع وجدت الخطأ.

4. اختبر فرضية واحدة في المرة

لا تغيّر خمسة أشياء معًا. غيّر شيئًا واحدًا، شغّل، انظر النتيجة. إن نجح، تعرف السبب. إن فشل، تعرف أن السبب في غير ذلك المكان.

5. التحدث للبط (Rubber Duck Debugging)

اشرح كودك بصوت عالٍ، وكأنك تشرحه لزميل (أو لبطة بلاستيكية). كثير من الأخطاء تتضح فقط حين تحاول صياغتها بالكلام. هذه تقنية معروفة في صناعة البرمجيات.

مكتبة logging — الـ print الاحترافي

`print` للتجربة. للمشاريع الجدية استخدم `logging`:

```
import logging

)logging.basicConfig
,level=logging.INFO
rmat="%(asctime)s - %(levelname)s - %(message)s
(
```

```
logging.info("بدء البرنامج")
logging.warning("القيمة قريبة من الحد")
logging.error("فشلت العملية")
```

ميزات `logging` على `print`: - مستويات مختلفة (DEBUG, INFO, WARNING, ERROR). - يمكنك حفظها في ملف. - يمكن إيقافها بدون حذف الكود. - تتضمن طابع الوقت تلقائيًا.

استخدام مفصّل لـ logging

```
import logging

logging.basicConfig(
    filename="app.log",
    filemode="w",
    level=logging.DEBUG,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    encoding="utf-8"
)

def process_user(user_id):
```

```
logging.info(f"بدء معالجة المستخدم {user_id}")
try:
    # عمل ما
    logging.debug(f"المستخدم {user_id} لديه 5 س
    return True
except Exception as e:
    logging.error(f"فشل معالجة {user_id}: {e}")
    return False
```

الـ Assertion — افتراضات صريحة

`assert` يتأكد أن شيئًا صحيح. إن لم يكن، يرمي خطأ:

```
def divide(a, b):
    assert b != 0, "المقام لا يمكن أن يكون صفرًا"
    return a / b

divide(10, 2)        # 5.0
divide(10, 0)        # AssertionError: المقام لا يمك
```

استخدمها للتحقق من الافتراضات الداخلية في كودك. لا تستخدمها للتحقق من إدخال المستخدم — استخدم `if` و `raise ValueError`.

مثال متكامل: API call آمن

```
import requests
import logging

logging.basicConfig(level=logging.INFO)

def safe_api_call(url, headers=None, max_retries=3)
    """يستدعي API مع معالجة كاملة للأخطاء."""
    for attempt in range(1, max_retries + 1):
        try:
s.get(url, headers=headers, timeout=10)
            response.raise_for_status()
            return response.json()
        except requests.Timeout:
out :{attempt} محاولة"f)logging.warning
        except requests.ConnectionError:
 لا :{attempt} محاولة"f)logging.warning
        except requests.HTTPError as e:
        if response.status_code == 429:
:{attempt} محاولة"f)logging.warning
```

```
:else
"HTTP {response.status_code}: {e}")
return None
:except ValueError
("الرد ليس JSON صحيحًا")logging.error
return None

("{url} فشلت كل المحاولات لـ"f)logging.error
return None

# استخدام
e_api_call("https://api.github.com/users/torvalds")
:if data
print(data["name"])
:else
("لم يكن ممكنًا جلب البيانات")print
```

الجسر إلى كتاب RAG

في نظام RAG، عشرات الأشياء قد تفشل: API يبطئ، قاعدة المتجهات تتعطل، النموذج يتجاوز الـ context window. كل خطوة في خط الأنابيب يجب أن

تكون "مقاومة للفشل" (fault-tolerant). أنماط الـ try/except والـ retries التي تعلمتها هنا ستظهر في كل مكان.

`logging` ضروري لتشخيص المشاكل في الإنتاج. حين يخبرك مستخدم أن النظام أعطاه إجابة خاطئة قبل ساعة، ستحتاج إلى سجل تفصيلي لتعرف ما حدث.

خلاصة الفصل

تعلمت كيف تتعامل مع الأخطاء بشكل احترافي — بـ try/except، بـ raise، بأنواع أخطاء مخصصة. تعلمت 5 استراتيجيات Debugging أساسية. تعلمت متى تستخدم `print` ومتى ترتقي إلى `logging`. هذه المهارات تفصل بين البرنامج "الذي يعمل في حاسوبك" والبرنامج "الذي يعمل عند آلاف المستخدمين بدون انقطاع". في الفصل القادم، سنتعلم كيف نخفي مفاتيح API بأمان.

تمارين الفصل الحادي عشر

1. اكتب دالة `safe_int(text)` تحوّل نصًا إلى رقم. إن فشل التحويل، تُرجع 0 وتطبع تحذيرًا.
2. اكتب دالة `read_json_safely(path)` تقرأ ملف JSON وتُرجع قاموسه، أو تُرجع `{}` إن لم يوجد الملف أو إن كان JSON غير صحيح.

3. اكتب فئة خطأ مخصصة `InsufficientBalanceError`، واستخدمها في دالة `withdraw(account, amount)`.
4. أعد كتابة برنامج لعبة التخمين من الفصل 5، لكن مع معالجة كاملة للأخطاء (إن أدخل المستخدم نصًا بدلًا من رقم، اطلب منه مجددًا بدون انهيار).

الفصل الثاني عشر: متغيرات البيئة وحفظ مفاتيح API بأمان

> "السر الذي يعرفه اثنان لم يعد سرًا."

كل API لنماذج الذكاء الاصطناعي يتطلب "مفتاحًا" (API key). هذا المفتاح هو هويتك الرقمية، وهو الذي يحاسبك على الاستخدام. لو وقع المفتاح في يد خاطئة، قد يستخدمه أحدهم بكميات هائلة، وستجد فاتورة بعشرات أو مئات الدولارات.

الخطأ الأشهر للمبتدئين: كتابة المفتاح مباشرة في الكود، ثم رفع الكود على GitHub. ثم في خلال ساعات، يكتشف خوادم الـ scraping المفتاح، ويُسرق الحساب. هذا الفصل يعلمك كيف تتجنب هذه الكارثة.

ما هي متغيرات البيئة؟

متغيرات البيئة (environment variables) قيم محفوظة على مستوى نظام التشغيل، ويستطيع أي برنامج قراءتها. مثلًا، PATH متغير بيئة شهير يحتوي مسارات المجلدات التي يبحث فيها النظام عن البرامج.

نحن سنستخدمها لحفظ المفاتيح السرية، فلا تظهر في الكود.

قراءة متغير بيئة في بايثون

```
import os

api_key = os.environ.get("ANTHROPIC_API_KEY")
print(api_key)        # سيطبع المفتاح إن كان موجودً
```

os.environ.get(name) تُرجع قيمة المتغير أو None إن لم يوجد.

أو يمكنك تحديد قيمة افتراضية:

```
s.environ.get("ANTHROPIC_API_KEY", "default-value")
```

أو الفشل بشكل واضح إن لم يوجد:

```
key = os.environ["ANTHROPIC_API_KEY"]    # KeyError
```

ضبط متغير بيئة على Windows

في PowerShell:

```
"...-env:ANTHROPIC_API_KEY = "sk-ant$
```

ثم شغّل بايثون **في نفس النافذة**. المتغير يختفي عند إغلاق النافذة. للحفظ الدائم: 1. اضغط على Windows + R، اكتب `sysdm.cpl`، اضغط Enter. 2. علامة التبويب "Advanced" → "Environment Variables". 3. أضف متغيرًا جديدًا.

ضبط متغير بيئة على macOS / Linux

في Terminal:

```
"...-export ANTHROPIC_API_KEY="sk-ant
```

للحفظ الدائم، أضف هذا السطر إلى `~/.bashrc` أو `~/.zshrc`.

الطريقة الأنسب: ملف .env

إدارة متغيرات البيئة يدويًا متعبة. الحل المعتمد في صناعة البرمجيات هو ملف `.env`:

أنشئ ملفًا اسمه .env في جذر مشروعك:

```
...-ANTHROPIC_API_KEY=sk-ant-api03
...-OPENAI_API_KEY=sk-proj
..._GROQ_API_KEY=gsk
DATABASE_URL=postgresql://localhost/mydb
```

(لاحظ: لا توجد مسافات حول =، ولا تحتاج إلى علامات تنصيص).

ثم ثبّت مكتبة python-dotenv:

```
pip install python-dotenv
```

ثم في كودك:

```
from dotenv import load_dotenv
import os

load_dotenv()                                  # تـ
api_key = os.environ.get("ANTHROPIC_API_KEY")
print("لـدي مفـتـاح:", api_key is not None)
```

load_dotenv() تقرأ ملف .env وتضع قيمه في os.environ.

الكود الباقي مثل قبل.

حماية المفتاح من Git — أهم خطوة في حياتك

إن كنت تستخدم Git (وستستخدمه)، يجب أن تتأكد أن `.env` **لا يُرفع** أبدًا إلى المستودع.

أنشئ ملفًا اسمه `.gitignore` في جذر مشروعك، واكتب فيه:

```
# مفاتيح وأسرار
.env
.env.local
*.pem
/secrets

# بايثون
/__pycache__
*.pyc
/venv

# نظام التشغيل
.DS_Store
Thumbs.db
```

```
# المحرر
/vscode.
/idea.
```

.gitignore يخبر Git أن يتجاهل هذه الملفات. لن تظهر في git status، ولن تُرفع.

اكتب .env في .gitignore **قبل** أن تُنشئ ملف .env. لو رفعت .env ولو مرة واحدة، يبقى في تاريخ Git إلى الأبد، حتى لو حذفته لاحقًا.

بديل لمشاركة الإعدادات: .env.example

كيف يعرف زميلك الذي يكلون مشروعك بـ git clone أنه يحتاج إلى تعريف ANTHROPIC_API_KEY؟ أنشئ ملفًا اسمه .env.example:

```
# انسخ هذا الملف إلى .env واملأ القيم
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
GROQ_API_KEY=
```

هذا الملف يُرفع إلى Git (لأنه فارغ القيم). ويعرف منه كل من يكلون المشروع ما هي المفاتيح التي يحتاج.

التحقق من تحميل المفتاح

إضافة فحص بسيط ينقذك من ساعات من الحيرة:

```
from dotenv import load_dotenv
import os

()load_dotenv

red_keys = ["ANTHROPIC_API_KEY", "OPENAI_API_KEY"]
[k for k in required_keys if not os.environ.get(k)]

:if missing
    ("env: {missing}. مفاتيح مفقودة في"f)print
    exit(1)

print("كل المفاتيح موجودة. يمكن البدء.")
```

لا تطبع المفتاح أبدًا

```
# سيئ جدًا!
print(f"مفتاحي: {api_key}")

# سيئ أيضًا — قد يصل المفتاح إلى logs
logging.info(f"استخدمت {api_key}")

# جيد
print(f"مفتاحي: {api_key[:8]}...{api_key[-4:]}")
print(f"مفتاحي: ***")
```

عرض جزء صغير من المفتاح (لتعرف أنه محمّل) أمان مقبول. عرض المفتاح كاملًا خطر.

أمن متقدم: مدراء الأسرار

في بيئات الإنتاج الحقيقية، حتى `.env` لا يكفي. الشركات تستخدم:

- **HashiCorp Vault**: نظام مخصص لإدارة الأسرار.
- **AWS Secrets Manager**: من أمازون.
- **Google Secret Manager**: من جوجل.

- **Azure Key Vault**: من ميكروسوفت.

هذه الأنظمة تحفظ المفاتيح مشفّرة، وتعطيها لبرنامجك في وقت التشغيل فقط، وتُسجّل من الذي طلبها ومتى. للآن لست بحاجة إليها — `.env` مع `.gitignore` كافٍ تمامًا للتعلم والمشاريع الشخصية.

التعامل مع المفاتيح في Google Colab

Colab لا يقرأ ملف `.env` مباشرة. الطريقة الأفضل:

```
from google.colab import userdata

api_key = userdata.get("ANTHROPIC_API_KEY")
```

اذهب إلى Colab، أيقونة المفتاح في الشريط الجانبي، وأضف "secret" بنفس الاسم. سيُحفظ في حساب Google الخاص بك، ولن يظهر في الـ notebook.

أو، إن كنت في تجربة سريعة فقط:

```
import getpass

api_key = getpass.getpass("الصق مفتاحك: ")
```

getpass تُخفي ما تكتب (مثل كلمة السر). آمنة من العين المتطفلة على شاشتك.

مثال متكامل: استدعاء Claude مع .env

ملف .env:

```
...-ANTHROPIC_API_KEY=sk-ant-api03
```

ملف app.py:

```
from dotenv import load_dotenv
import os
import requests

()load_dotenv

api_key = os.environ.get("ANTHROPIC_API_KEY")
:if not api_key
raise SystemExit("مفقود ANTHROPIC_API_KEY في .

} = headers
```

```
                    ,x-api-key": api_key"
          ,"anthropic-version": "2023-06-01"
         ,"content-type": "application/json"
                                        {

                                  } = payload
        ,"model": "claude-haiku-4-5-20251001"
                        ,max_tokens": 100"
                             ] :"messages"
 :"role": "user", "content"} "مرحبا، أجبني ب
                                        ,[
                                        {

                     )response = requests.post
    ,"https://api.anthropic.com/v1/messages"
                         ,headers=headers
                            ,json=payload
                              ,timeout=30
                                        (

                 :if response.status_code == 200
```

```
()data = response.json
print(data["content"][0]["text"])
:else
ponse.text} :{response.status_code} خطأ"f)print
```

شغّل هذا، وستكون قد أجريت أول محادثة آمنة مع نموذج لغوي. سنتعمق في تفاصيل الاستدعاءات في الفصل القادم.

قائمة فحص قبل كل commit

قبل أن تكتب `git commit`:

1. هل `.env` موجود في `.gitignore`؟
2. هل `git status` يُظهر `.env`؟ إن كان نعم، شيء خطأ.
3. هل في الكود أي مفتاح ثابت يبدأ بـ `sk-` أو `gsk_` أو غيره؟
4. هل أي `print(api_key)` متبقي؟

اعتد على هذه الأسئلة الأربعة. ستحميك من ندم كبير.

ماذا لو سرّب مفتاح بالخطأ؟

إن نسيت ووضعت مفتاحًا في الكود ورفعته:

1. **فورًا** اذهب إلى لوحة تحكم OpenAI، Anthropic Console) API Platform، إلخ).
2. ألغ المفتاح المسرّب فورًا.
3. أنشئ مفتاحًا جديدًا.
4. ضع الجديد في `.env`.
5. حتى لو حذفت السطر من الكود، قد يبقى في تاريخ Git. للتنظيف الكامل، استخدم `git filter-repo` أو حتى أنشئ مستودعًا جديدًا.

السرعة مهمة. بوتات الـ scraping تكتشف المفاتيح في دقائق.

خلاصة الفصل

تعلمت أمن مفاتيح API — مهارة ستحميك من خسارات حقيقية. تعلمت متغيرات البيئة، ملف `.env`، ومنع رفعها بـ `.gitignore`. تعلمت كيف تتعامل مع المفاتيح في Colab. هذه عادات تتبع الكود الإنتاجي حول العالم. في الفصل القادم، سنطبق كل ما تعلمناه ونستدعي نماذج لغوية حقيقية.

تمارين الفصل الثاني عشر

1. أنشئ مشروعًا جديدًا،

   فيه `.env`، `.env.example`، `.gitignore` صحيحين. تأكد

   من `git status` أن `.env` لا يظهر.

2. اكتب دالة `get_required_env(name)` تقرأ متغير بيئة،

   وترمي خطأ واضح إن لم يوجد.

3. اكتب برنامجًا يطبع كل المفاتيح في ملف `.env` لكن مع عرض أول 4

   أحرف فقط من قيمتها.

4. ابحث عن أداة `git-secrets` على الإنترنت. اكتب فقرة قصيرة (5

   أسطر) عما تفعله.

الفصل الثالث عشر: استدعاء نماذج اللغة الكبيرة من بايثون

> "أعطني نموذجًا، أعطك عالَمًا."

هذا هو الفصل الذي ينتظره كل طالب في برنامج LLM4Yemen. كل ما تعلمناه في الـ 12 فصلًا السابقة كان تحضيرًا لهذه اللحظة: استدعاء نموذج لغوي ضخم، وإجراء محادثة معه، والحصول على رد بالعربية. سنغطي ثلاثة مزودين: OpenAI (GPT)، Anthropic (Claude)، و Groq (مجاني وسريع جدًا).

ملاحظة قبل البداية: مفاتيح وحدود الوصول

طلاب اليمن قد يواجهون عقبتين عند استخدام Anthropic و OpenAI:

1. **بطاقة ائتمان**: كلاهما يتطلب بطاقة ائتمانية صالحة. إن لم يكن لديك، لا تقلق — Groq وOpenRouter يقدمان طبقة مجانية كافية لكل تمارين هذا الكتاب.

2. **عقوبات**: قد لا تتوفر بعض الخدمات في اليمن. جرب استخدام VPN موثوق إن واجهت مشكلة، أو اعتمد على البديل المجاني.

قاعدتي: **اعرف كيف تستدعي Anthropic و OpenAI نظريًا** (لتفهم كل الكود الذي ستراه على الإنترنت)، **وطبّق فعليًا على Groq أو OpenRouter** (لتشغّل التمارين بدون تكلفة).

استدعاء Claude (Anthropic)

تثبيت المكتبة الرسمية

```
pip install anthropic
```

أبسط استدعاء

```
from dotenv import load_dotenv
import os
from anthropic import Anthropic

load_dotenv()

client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))
```

```
)message = client.messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=200
]=messages
{"role": "user", "content": "اكتب قصيدة قصي
[
(

print(message.content[0].text)
```

شرح: - `Anthropic(api_key=...)`: ينشئ "العميل" — كائن مسؤول عن إرسال الطلبات. - `model`: اسم النموذج. `claude-haiku-4-5-20251001` هو الأرخص والأسرع. - `max_tokens`: أقصى عدد من "الرموز" (tokens) في الرد. الرمز ≈ 4 حروف عربية تقريبًا. - `messages`: قائمة من الرسائل. كل رسالة لها دور (`user` أو `assistant`) ومحتوى.

النتيجة المُعادة `message.content[0].text` تحتوي رد النموذج النصي.

رسائل متعددة الأدوار

```
)message = client.messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=300
]=messages
{"role": "user", "content": "ما عاصمة اليمن
{"role": "assistant", "content": "صنعاء هي
{"role": "user", "content": "ومتى أُسست؟"},
[
(

print(message.content[0].text)
```

النموذج يرى المحادثة كاملة، ويعرف أن السؤال الثاني يخص "صنعاء" التي ذكرها قبل قليل.

رسالة النظام (System Prompt)

تحدد شخصية النموذج وأسلوبه:

```
)message = client.messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=200
system="أنت مساعد أكاديمي يمني، تجيب باللغة الع
]=messages
{"role": "user", "content": "ما الفرق بين ب
[
(

print(message.content[0].text)
```

استدعاء عبر REST مباشرةً (بدون مكتبة anthropic)

أحيانًا تريد التحكم الكامل، أو تعمل في بيئة لا تستطيع تثبيت anthropic.

الحل:

```
import requests
import os
from dotenv import load_dotenv

()load_dotenv
```

```
)response = requests.post
,"https://api.anthropic.com/v1/messages"
}=headers
-api-key": os.environ["ANTHROPIC_API_KEY"]"
,"anthropic-version": "2023-06-01"
,"content-type": "application/json"
,{
}=json
,"model": "claude-haiku-4-5-20251001"
,max_tokens": 200"
] :"messages"
أعطني تعري" :"role": "user", "content"}
,[
,{
,timeout=30
(

:if response.status_code == 200
()data = response.json
print(data["content"][0]["text"])
:else
```

```
("{response.status_code} :خطأ"f)print
print(response.text)
```

كل ما تعلمته عن requests ينطبق هنا.

استدعاء GPT (OpenAI)

تثبيت

```
pip install openai
```

أبسط استدعاء

```
from dotenv import load_dotenv
import os
from openai import OpenAI

()load_dotenv

= OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

)response = client.chat.completions.create
```

```
,"model="gpt-4o-mini
]=messages
{"role": "system", "content": "أنت مساعد ذكي"}
{"role": "user", "content": "اشرح ما هو RAG"}
,[
,max_tokens=200
(

print(response.choices[0].message.content)
```

اختلافات ظاهرة عن Anthropic: - في OpenAI، رسالة النظام تأتي ضمن `messages` بدور `"system"`. - النتيجة في `response.choices[0].message.content`. - `gpt-4o-mini` هو نموذج OpenAI الرخيص.

عبر REST

```
import requests
import os
from dotenv import load_dotenv
```

```
()load_dotenv

)response = requests.post
,"https://api.openai.com/v1/chat/completions"
}=headers
": f"Bearer {os.environ['OPENAI_API_KEY']}"
,"Content-Type": "application/json"
,{
}=json
,"model": "gpt-4o-mini"
] :"messages"
اكتب 3 نصـ" :"role": "user", "content"}
,[
,max_tokens": 300"
,{
,timeout=30
(

esponse.json()["choices"][0]["message"]["content"])
```

البديل المجاني الأول: Groq

Groq يقدم نماذج Llama و Mixtral مجانًا، وبسرعة فائقة. لا يتطلب بطاقة.

اذهب إلى console.groq.com، سجّل بحساب Google، احصل على مفتاح فورًا.

تثبيت

```
pip install groq
```

الاستدعاء

```
from dotenv import load_dotenv
import os
from groq import Groq

load_dotenv()

client = Groq(api_key=os.environ.get("GROQ_API_KEY"))

response = client.chat.completions.create(
```

```
,"model="llama-3.3-70b-versatile
]=messages
{"role": "system", "content": "أنت مساعد عر
{"role": "user", "content": "ما هو RAG؟ اشر
,[
,max_tokens=300
(

print(response.choices[0].message.content)
```

API Groq مطابق لـ OpenAI تقريبًا. هذا ليس صدفة — Groq يتعمد التوافق ليسهل الانتقال.

البديل المجاني الثاني: OpenRouter

OpenRouter بوابة موحّدة لعشرات النماذج، فيه طبقة مجانية بأسماء تنتهي بـ `:free`. مثل `meta-llama/llama-3.3-8b-instruct:free`.

```
import requests
import os
from dotenv import load_dotenv
```

```
()load_dotenv

)response = requests.post
"https://openrouter.ai/api/v1/chat/completions"
}=headers
"Bearer {os.environ['OPENROUTER_API_KEY']}"
,"Content-Type": "application/json"
,{
}=json
": "meta-llama/llama-3.3-8b-instruct:free"
] :"messages"
عرّف الذكا" :"role": "user", "content"}
,[
,{
,timeout=30
(

esponse.json()["choices"][0]["message"]["content"])
```

OpenRouter ميزته أنك بتغيير `model` تنتقل بين GPT، Claude، Mistral، Llama، وعشرات النماذج، بمفتاح واحد.

الـ Streaming — استلام الرد كلمةً كلمةً

في الواجهات التفاعلية، تريد أن يظهر الرد للمستخدم تدريجيًا، لا دفعة واحدة. هذا يسمى "streaming":

```
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv()
client = Anthropic()

with client.messages.stream(
    model="claude-haiku-4-5-20251001",
    max_tokens=300,
    messages=[
        {"role": "user", "content": "اكتب قصة قصيرة
    ],
```

```
:as stream (
    :for text in stream.text_stream
print(text, end="", flush=True)

()print
```

كل قطعة من الرد تأتي فور إنتاجها. في تطبيقات الويب، يحوّل المطورون هذا إلى Server-Sent Events ليُعرض في المتصفح.

التحكم في الإبداعية — Temperature

```
)response = client.messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=100
,temperature=0.0          # دقيق، متكرر
messages=[{"role": "user", "content": "ما عاصمة
(
```

temperature رقم من 0 إلى 1 (أو 2 في بعض النماذج). 0 يُنتج إجابات حتمية ومحافظة. 1 يُنتج إجابات مبدعة وغير متوقعة. للأسئلة الواقعية، استخدم 0. للكتابة الإبداعية، استخدم 0.7-1.0.

التعامل الصحيح مع العربية

النماذج الحديثة تجيد العربية بشكل جيد، لكن بعض النصائح:

1. **اطلب اللغة صراحةً**: "أجبني بالعربية الفصحى" أوضح من ترك الأمر للنموذج.
2. **استخدم system prompt**: حدد فيه اللغة وأسلوب الكتابة.
3. **تأكد من ترميز UTF-8** عند الكتابة في ملف. لاحظ أن `print` على بعض إصدارات Windows القديمة قد يفسد العربية. استخدم `sys.stdout.reconfigure(encoding="utf-8")` إن واجهت مشكلة.

```
import sys
sys.stdout.reconfigure(encoding="utf-8")    # على s
```

مقارنة عملية: نفس السؤال على ثلاثة نماذج

```
import os
from dotenv import load_dotenv
from anthropic import Anthropic
from openai import OpenAI
from groq import Groq
```

```
()load_dotenv

prompt = "اشرح بالعربية ما هو الذكاء الاصطناعي في فق

Claude #
()claude = Anthropic
)c = claude.messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=300
messages=[{"role": "user", "content": prompt}]
(

GPT #
()openai = OpenAI
)g = openai.chat.completions.create
,"model="gpt-4o-mini
messages=[{"role": "user", "content": prompt}]
,max_tokens=300
(
```

```
# Llama على Groq
groq = Groq()
l = groq.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}]
    max_tokens=300,
)

print("=== Claude ===")
print(c.content[0].text)
print("\n=== GPT ===")
print(g.choices[0].message.content)
print("\n=== Llama (Groq) ===")
print(l.choices[0].message.content)
```

شغّل هذا بنفسك. ستلاحظ اختلافات في الأسلوب، الدقة، التركيب. كل نموذج له شخصيته. اختياره يعتمد على المهمة.

حساب الكلفة (تقريبيًا)

كل نموذج يُسعّر بالـ "1 مليون token". أمثلة (تقريبية في 2026):

النموذج	الإدخال (لكل مليون)	الإخراج (لكل مليون)
Claude Haiku 4.5	$0.80	$4.00
GPT-4o-mini	$0.15	$0.60
Llama 3.3 (Groq)	مجاني	مجاني

سؤال متوسط (200 token دخول، 300 token خروج) يكلف على Claude Haiku حوالي $0.0014. أي حوالي 0.7 سنت. ألف سؤال = $1.40.

كرر الحساب على نموذجك، وستجد أن الاستخدام التعليمي رخيص جدًا. التكلفة تظهر في الأنظمة الإنتاجية ذات آلاف المستخدمين.

رسائل خطأ شائعة

رسالة الخطأ	المعنى	الحل
401 Unauthorized	المفتاح خاطئ أو منتهٍ	تحقق من `.env`. أنشئ مفتاحًا جديدًا.
429 Too Many Requests	تجاوزت rate limit	انتظر ثوانٍ، أعد المحاولة.
400 invalid_request_error	شيء خاطئ في المعطيات	اقرأ رسالة الخطأ بعناية. غالبًا اسم نموذج خاطئ.

رسالة الخطأ	المعنى	الحل
server_error 500	مشكلة في الخادم	حاول لاحقًا.
insufficient credits	رصيدك انتهى	شحن، أو استخدم Groq.

الجسر إلى كتاب RAG

في كتاب RAG، ستستخدم هذه الـ APIs بطريقتين:

1. **التضمين (Embeddings)**: تحويل النصوص إلى متجهات. تستدعي `client.embeddings.create()` بدلًا من `messages.create()`.
2. **التوليد (Generation)**: نفس ما تعلمته هنا — ترسل سؤال المستخدم + النصوص المسترجَعة، تحصل على إجابة.

كل سطر من هذا الفصل ستراه في كل سطر من ذلك الكتاب. أتقنه الآن.

خلاصة الفصل

أتقنت استدعاء أهم ثلاثة مزودين لنماذج اللغة. تعلمت كيف تتعامل مع العربية، كيف تتحكم في الإبداعية، كيف تحلل التكاليف، كيف تتعامل مع الأخطاء. أنت الآن مستعد لبناء أول تطبيق ذكاء اصطناعي حقيقي. هذا ما سنفعله في الفصل الختامي.

تمارين الفصل الثالث عشر

1. اختر مزودًا (Groq إن كنت بدون بطاقة)، واسأل النموذج 5 أسئلة عن اليمن. احفظ الردود في ملف.
2. اكتب دالة `chat(question)` تستدعي النموذج وتُرجع الجواب. استدعها 3 مرات متتالية للسؤال نفسه بـ `temperature=1.0`. لاحظ التنوع في الإجابات.
3. اكتب برنامج محادثة يحفظ كل الرسائل، فيتذكر النموذج ما قلته من قبل.
4. قارن جودة العربية بين Claude و Llama 3.3 لمهمة "ترجم هذه الجملة الإنجليزية إلى العربية الفصحى".

الفصل الرابع عشر: المشروع الختامي — مُلَخِّص ملف CSV عربي بالذكاء الاصطناعي

> "العلم بلا عمل كالشجر بلا ثمر."

كل ما تعلمته في 13 فصلًا سيلتقي في هذا المشروع. ستبني سكربتًا حقيقيًا يقرأ ملف CSV عربيًا، يستدعي نموذجًا لغويًا لتلخيص كل صف، ويكتب النتائج في ملف CSV جديد. هذا تمامًا ما يفعله مهندس الذكاء الاصطناعي في يومه الأول من عمله. لا تخف، ستفعلها خطوةً خطوةً.

وصف المشروع

المُدخل: ملف `articles.csv` يحوي 10 مقالات قصيرة بالعربية. لكل مقال: عنوان، تاريخ، نص.

المخرَج: ملف `summaries.csv` يحوي نفس البيانات، مضافًا إليها عمود "ملخص" — تلخيص بسيط بـ 2-3 جمل لكل مقال.

الأدوات: بايثون، مكتبة `csv`، مكتبة `dotenv`، مزوّد LLM (Groq إن لم يكن لديك بطاقة، Claude أو OpenAI إن كان لديك).

بنية المشروع

```
/project
├── .env
├── .gitignore
├── articles.csv        ← المدخل
├── summaries.csv       ← المخرج (سنُنشئه)
├── summarize.py        ← السكربت
├── requirements.txt
└── log.txt             ← سجل التشغيل
```

الخطوة 1: إعداد البيئة

أنشئ مجلدًا للمشروع:

```
mkdir csv-summarizer
cd csv-summarizer
python -m venv venv
```

```
source venv/bin/activate          # على Linux/macOS
venv\Scripts\activate             # على Windows
```

أنشئ ملف requirements.txt:

```
python-dotenv
groq
```

(أو anthropic إن كنت ستستخدم Claude، أو openai لـ GPT.)

ثبّت:

```
pip install -r requirements.txt
```

الخطوة 2: إعداد ملف .env

```
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

أنشئ .gitignore:

```
.env
venv/
__pycache__/
```

```
*.pyc
log.txt
```

الخطوة 3: إنشاء ملف الإدخال

أنشئ `articles.csv` بهذا المحتوى (لاحظ ترميز utf-8-sig للتوافق مع Excel):

```
import csv

articles = [
    {
        "id": 1,
        "title": "افتتاح برنامج LLM4Yemen",
        "date": "2026-04-15",
        "text": "افتُتح اليوم برنامج LLM4Yemen في ج
    },{
    }
        "id": 2,
        "title": "إطلاق نموذج Claude الجديد",
        "date": "2026-03-20",
```

"text": "أعلنت شركة Anthropic عن إطلاق نموذج
,{
}
,id": 3"
"title": "ندوة عن الأمن السيبراني",
"date": "2026-04-10",
"text": "نظّمت كلية الحاسوب ندوة عن الأمن الـ
,{
}
,id": 4"
"title": "معرض الإبداع الطلابي",
"date": "2026-04-22",
"text": "احتضنت قاعة الجامعة الكبرى معرض ال
,{
}
,id": 5"
"title": "تعاون أكاديمي مع جامعة دولية",
"date": "2026-02-08",
"text": "وقّعت جامعة عدن مذكرة تعاون مع جامـ
,{
}

,id": 6"
"title": "بطولة برمجة جامعية",
"date": "2026-03-30",
"text": "اختُتمت أمس بطولة البرمجة الجامعية
,{
}
,id": 7"
"title": "ورشة عن الذكاء الاصطناعي التوليدي"
"date": "2026-04-05",
"text": "نظّمت كلية الهندسة ورشة تعريفية عن
,{
}
,id": 8"
"title": "إطلاق مكتبة برمجية مفتوحة",
"date": "2026-04-18",
"text": "أصدر فريق من المطورين اليمنيين مك
,{
}
,id": 9"
"title": "محاضرة عن أخلاقيات التقنية",
"date": "2026-04-12",

```
"text": "ألقى أستاذ زائر محاضرة عن أخلاقيات
,{
}
,id": 10"
"title": "مسابقة لأفضل مشروع تخرج",
,"date": "2026-04-25"
"text": "أعلنت الجامعة عن مسابقة لأفضل مشروع
,{
[

s.csv", "w", encoding="utf-8-sig", newline="") as f
(f, fieldnames=["id", "title", "date", "text"])
()writer.writeheader
writer.writerows(articles)

print(f"تم إنشاء articles.csv بـ {len(articles)} مقـ
```

احفظ هذا في ملف seed_data.py وشغّله مرة واحدة:

```
python seed_data.py
```

الخطوة 4: السكربت الرئيسي

الآن ننشئ summarize.py — قلب المشروع:

```
"""
ملخّص مقالات CSV بالذكاء الاصطناعي.
يقرأ articles.csv، يلخّص كل مقال عبر LLM، يحفظ النتي
"""
import csv
import logging
import os
import time
from pathlib import Path
from dotenv import load_dotenv
from groq import Groq

# 1. الإعداد
load_dotenv()

logging.basicConfig(
    level=logging.INFO,
```

```
rmat="%(asctime)s - %(levelname)s - %(message)s
]=handlers
ng.FileHandler("log.txt", encoding="utf-8")
,()logging.StreamHandler
[
(

INPUT_FILE = Path("articles.csv")
OUTPUT_FILE = Path("summaries.csv")
"MODEL = "llama-3.3-70b-versatile
MAX_RETRIES = 3

# 2. التحقق من المفتاح
api_key = os.environ.get("GROQ_API_KEY")
:if not api_key
("env. في GROQ_API_KEY مفقود")raise SystemExit

client = Groq(api_key=api_key)

def summarize_article(title: str, text: str) -> str
```

"""يطلب من النموذج تلخيصًا مختصرًا للمقال."""

prompt = f"""أنت محرر صحفي يكتب بالعربية الفصحى

لخّص المقال التالي في جملتين فقط، بأسلوب موضوعي مختص

العنوان: {title}

النص:

{text}

الملخص:"""

```
:for attempt in range(1, MAX_RETRIES + 1)
                                    :try
sponse = client.chat.completions.create
                          ,model=MODEL
                            ]=messages
e": "user", "content": prompt"}
                                  ,[
                        ,max_tokens=150
                      ,temperature=0.3
                                    (
```

```
sponse.choices[0].message.content.strip
                    :except Exception as e
فشلة {attempt} محاولة"f)logging.warning
                :if attempt < MAX_RETRIES
            time.sleep(2 ** attempt)

                    return "[فشل التلخيص]"


                    :()def main
                # 3. التحقق من الملف
                :()if not INPUT_FILE.exists
غير م {INPUT_FILE} الملف"f)raise SystemExit

                # 4. القراءة
INPUT_FILE.open("r", encoding="utf-8-sig") as f
                reader = csv.DictReader(f)
                articles = list(reader)

مقالًا {len(articles)} قُرئ"f)logging.info
```

```
# 5. التلخيص
summaries = []
:for i, article in enumerate(articles, 1)
:{len(articles)}/{i} معالجة"f)logging.info

_article(article["title"], article["text"])

})summaries.append
,id": article["id"]"
,title": article["title"]"
,date": article["date"]"
,summary": summary"
({

# احترام rate limits
time.sleep(1)

# 6. الكتابة
pen("w", encoding="utf-8-sig", newline="") as f
)writer = csv.DictWriter
ames=["id", "title", "date", "summary"]
```

```
(
writer.writeheader()
writer.writerows(summaries)

logging.info(f"اكتمل! المخرَج: {OUTPUT_FILE}")
print(f"\nنجح. افتح {OUTPUT_FILE} لرؤية الملخص

if __name__ == "__main__":
main()
```

الخطوة 5: التشغيل

```
python summarize.py
```

سترى في الطرفية:

```
2026-04-29 ... - INFO - قُرئ 10 مقالًا
2026-04-29 ... - INFO - معالجة 1/10: افتتاح برنامج
2026-04-29 ... - INFO - معالجة 2/10: إطلاق نموذج ude
...
2026-04-29 ... - INFO - اكتمل! المخرَج: ummaries.csv
```

افتح `summaries.csv` في Excel — ستجد عمودًا جديدًا "summary" بالملخصات.

الخطوة 6: التحقق من الجودة

ليس كل ملخص سيكون مثاليًا. اقرأ النتائج. إن وجدت ملخصًا ضعيفًا، حاول:

1. تعديل الـ prompt — اجعله أوضح.
2. تجربة نموذج آخر.
3. ضبط `temperature` (0.1 يجعل النموذج أكثر التزامًا، 0.7 يجعله أكثر إبداعًا).

هذه عملية تسمى "هندسة المُحاثة" (prompt engineering)، وستتعلم عنها أكثر في الكتب القادمة.

تحسينات مقترحة

1. التوازي (Parallel Processing)

سكربتك الحالي يُعالج المقالات واحدًا واحدًا. لـ 1000 مقال، سيستغرق ساعة.

يمكنك تسريعه بـ `concurrent.futures`:

```
ent.futures import ThreadPoolExecutor, as_completed

:def process_article(article)
rize_article(article["title"], article["text"])
return {**article, "summary": summary}

:with ThreadPoolExecutor(max_workers=5) as executor
ubmit(process_article, a): a for a in articles}
[] = summaries
:for future in as_completed(futures)
summaries.append(future.result())
```

5 طلبات متوازية — 5 أضعاف السرعة. لكن انتبه لـ rate limits.

2. تخزين مؤقت (Caching)

إن أعدت تشغيل السكربت، فإنه يستدعي API لكل مقال من جديد. أضف cache:

```
import hashlib
import json

CACHE_FILE = Path("cache.json")
```

```
ILE.read_text("utf-8")) if CACHE_FILE.exists() else

:def cached_summarize(title, text)
()key = hashlib.md5(text.encode()).hexdigest
:if key in cache
return cache[key]
summary = summarize_article(title, text)
cache[key] = summary
json.dumps(cache, ensure_ascii=False), "utf-8")
return summary
```

3. تتبع التكلفة

```
total_input_tokens = 0
total_output_tokens = 0

# داخل summarize_article
total_input_tokens += response.usage.prompt_tokens
_output_tokens += response.usage.completion_tokens

# في النهاية
```

```
t_estimate = (total_input_tokens / 1_000_000) * 0.0
* (total_output_tokens / 1_000_000)
_tokens} دخول، {total_input_tokens} :الـرمـوز "f)print
```

(Groq مجاني، لكن إن استخدمت Claude أو OpenAI، احسب بالأسعار الحقيقية).

4. واجهة سطر أوامر

```
import argparse

()parser = argparse.ArgumentParser
ser.add_argument("--input", default="articles.csv")
r.add_argument("--output", default="summaries.csv")
ument("--model", default="llama-3.3-70b-versatile")
()args = parser.parse_args
```

شغّل بـ `python summarize.py --input my_data.csv --output result.csv`.

ماذا تعلمت من هذا المشروع؟

في 100 سطر من بايثون، طبّقت:

- قراءة CSV (الفصل 9)
- استدعاء API (الفصل 13)
- معالجة الأخطاء وretries (الفصل 11)
- متغيرات البيئة (الفصل 12)
- logging (الفصل 11)
- pathlib (الفصل 8)
- f-strings (الفصل 4)
- لغة type hints (`title: str` — تلميحات نوعية)
- بنية المشروع (.gitignore، requirements.txt، فصل seed_data عن الكود الرئيسي)

هذا "خط أنابيب بيانات" (data pipeline) كامل. هذا هو نفس النمط الذي ستبنيه على ملايين النصوص في كتاب RAG. الاختلاف الوحيد: بدلًا من تلخيص كل نص، ستحوّله إلى متجه وتحفظه في قاعدة متجهات.

أفكار لتوسيع المشروع

1. **تصنيف النصوص**: أضف عمودًا يصنّف كل مقال (تعليم، ذكاء اصطناعي، أخلاقيات...).
2. **استخراج الكيانات**: استخراج أسماء الأشخاص والأماكن والمنظمات.
3. **ترجمة**: ترجمة كل مقال إلى الإنجليزية.
4. **سؤال وجواب**: لكل مقال، توليد 3 أسئلة محتملة وإجاباتها.
5. **واجهة ويب**: استخدم Streamlit (`pip install streamlit`) لتحويل السكربت إلى تطبيق ويب صغير.

الجسر إلى كتاب RAG

في كتاب RAG ستبني نظامًا أكثر تعقيدًا، لكن المعمار نفسه:

```
ess each → call AI API → save results → query later
```

استبدل "تلخيص" بـ "تضمين متجهي"، استبدل "حفظ في CSV" بـ "حفظ في قاعدة متجهات"، أضف خطوة بحث، وستحصل على RAG. كل هذا الكتاب كان يُعدّك لتلك الخطوة التالية.

خلاصة الفصل

أنجزت أول مشروع ذكاء اصطناعي عملي في حياتك. كل المهارات التي تعلمتها التقت في 100 سطر تنتج قيمة حقيقية. هذا ما تفعله الشركات التقنية يوميًا. أنت الآن جاهز لكتاب RAG، حيث ستبني أنظمة أكثر تعقيدًا، لكن مبنية على نفس الأسس التي أتقنتها.

تمارين الفصل الرابع عشر

1. شغّل المشروع، وافحص النتائج. اختر أسوأ ملخص، وعدّل الـ prompt لتحسينه.
2. أضف عمودًا "topic" إلى ملف الخرج، يصنف كل مقال إلى أحد الفئات: تعليم، تقنية، مجتمع، أخرى.
3. قارن النتائج بين Llama 3.3 (Groq) و Claude Haiku أو GPT-4o-mini. أيها أفضل في العربية؟
4. أنشئ نسخة "stream" من السكربت، تطبع كل ملخص فور إنتاجه (بدلًا من الانتظار حتى النهاية).

الفصل الخامس عشر: حلول التمارين

> "ولا تنظر إلى الإجابة قبل أن تجاهد ساعةً كاملة."

هذا الفصل يضم حلول التمارين من كل الفصول السابقة. **حذار من الإجابة المباشرة**. إن نظرت قبل المحاولة الجدية، فقد ضيعت أهم فرصة للتعلم. الكفاح هو ما يبني المهارة. التحقق هو ما يصقلها.

اكتب الحل بنفسك. شغّله. صحّح أخطاءه. ثم تعال هنا للمقارنة.

حلول الفصل الثالث: برنامجك الأول

تمرين 1: ترحيب بالاسم والبلد

```
name = input("ما اسمك؟ ")
country = input("ما بلدك؟ ")
print(f"أهلًا {name} من {country}")
```

تمرين 2: مساحة مستطيل

```
((" :الطول")input)length = float
((" :العرض")input)width = float
area = length * width
("{area} :المساحة"f)print
```

تمرين 3: الحاسبة الموسّعة

```
((" :الرقم الأول")input)a = float
((" :الرقم الثاني")input)b = float
("{a + b} :الجمع"f)print
("{a - b} :الطرح"f)print
("{a * b} :الضرب"f)print
:if b != 0
    ("{a / b} :القسمة"f)print
:else
    print("القسمة: غير ممكن (المقام صفر)")
```

تمرين 4: حساب العمر

```
((" :سنة ميلادك")input)birth = int
age = 2026 - birth
("{age} :عمرك تقريبًا"f)print
```

حلول الفصل الرابع: المتغيرات والأنواع

تمرين 1: السعر بعد الخصم

```
((" :السعر")input)price = float
((" :% نسبة الخصم")input)discount = float
new_price = price - (price * discount / 100)
("{new_price:.2f} :السعر بعد الخصم"f)print
```

تمرين 2: تحليل النص

```
(" :اكتب نصًا")text = input
("{len(text)} :عدد الحروف"f)print
("{text[::-1]} :معكوس النص"f)print
("{()text.upper} :بأحرف كبيرة"f)print
```

تمرين 3: متوسط ثلاث درجات

```
((" :الأولى الدرجة")input)g1 = float
((" :الثانية الدرجة")input)g2 = float
((" :الثالثة الدرجة")input)g3 = float
average = (g1 + g2 + g3) / 3
("{average:.2f} :المتوسط")print
```

تمرين 4: الفرق بين العمليات

```
print(10 / 3)   # 3.333 ...قسمة عادية، نتيجة عشرية
print(10 // 3)  # 3  قسمة صحيحة، يقطع الكسر
print(10 % 3)   # 1  الباقي

print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(7 % 2)    # 1

print(15 / 4)   # 3.75
print(15 // 4)  # 3
print(15 % 4)   # 3
```

حلول الفصل الخامس: التحكم في التدفق

تمرين 1: FizzBuzz

```
:for i in range(1, 101)
    :if i % 15 == 0
print("FizzBuzz")
    :elif i % 3 == 0
    print("Fizz")
    :elif i % 5 == 0
    print("Buzz")
                :else
        print(i)
```

تمرين 2: أكبر وأصغر

```
[] = nums
:for i in range(5)
((" :{i+1} الـرقـم "f)input)n = float
nums.append(n)
```

```
print(f"الأكبر: {max(nums)}")
print(f"الأصغر: {min(nums)}")
```

تمرين 3: كلمة سرية بمحاولات محدودة

```
correct = "yemen2026"
for attempt in range(3):
    pwd = input("كلمة السر: ")
    if pwd == correct:
        print("صحيح! مرحبًا بك.")
        break
    else:
        print(f"خطأ. تبقى لك {attempt - 2} محاولة.")
else:
    print("استُنفدت المحاولات.")
```

تمرين 4: مكعّبات مضاعفات الأربعة

```
cubes = [n ** 3 for n in range(1, 21) if n % 4 == 0]
print(cubes)   # [64, 512, 1728, 4096, 8000]
```

حلول الفصل السادس: الدوال

تمرين 1: مئوي إلى فهرنهايت

```
:def temperature_to_fahrenheit(c)
    return c * 9/5 + 32

print(temperature_to_fahrenheit(0))      # 32.0
print(temperature_to_fahrenheit(100))    # 212.0
```

تمرين 2: عدّ الحروف العربية

```
:def arabic_letter_count(text)
    count = 0
    :for char in text
        # نطاق Unicode للحروف العربية
        :"ه" => char => "ـ" if
            count += 1
    return count

print(arabic_letter_count("Hello أهلًا 123")) # 4
```

تمرين 3: قسمة آمنة

```
:def safe_divide(a, b)
        :if b == 0
    return None
        return a / b

print(safe_divide(10, 2))     # 5.0
print(safe_divide(10, 0))     # None
```

تمرين 4: تنسيق الاسم

```
:("السيد"=first, last, title)def format_name
"return f"{title} {first} {last}

((" علي" ,"أحمد")format_name)print
((" الدكتورة" ,"محمد" ,"فاطمة")format_name)print
```

حلول الفصل السابع: هياكل البيانات

تمرين 1: إحصائيات قائمة درجات

```
grades = [85, 92, 78, 65, 88, 71, 95, 58, 80, 77]
average = sum(grades) / len(grades)
highest = max(grades)
lowest = min(grades)
passed = len([g for g in grades if g >= 60])
print(f"المتوسط: {average:.2f}")
print(f"الأعلى: {highest}")
print(f"الأدنى: {lowest}")
print(f"الناجحون: {passed}/{len(grades)}")
```

تمرين 2: أكبر طالب سنًا

```
students = {
    "أحمد": 22,
    "محمد": 25,
    "خالد": 19,
    "فاطمة": 28,
```

```
"علي": 21,
}
oldest = max(students, key=students.get)
print(f"الأكبر سنًا: {oldest} ({students[oldest]}) سنة
```

تمرين 3: عمليات على قائمتين

```
list_a = ["أحمد", "محمد", "خالد", "علي"]
list_b = ["محمد", "فاطمة", "علي", "مريم"]
set_a, set_b = set(list_a), set(list_b)
print("المشتركة:", set_a & set_b)
print("في الأولى فقط:", set_a - set_b)
print("الكل بدون تكرار:", set_a | set_b)
```

تمرين 4: قاموس إلى tuples

```
d = {"أحمد": 88, "محمد": 92, "خالد": 75}
tuples = list(d.items())
print(tuples)  # [('أحمد', 88), ('محمد', 92), ('خ
```

حلول الفصل الثامن: الملفات

تمرين 1: تحليل ملف نصي

```
from pathlib import Path

text = Path("poem.txt").read_text(encoding="utf-8")
lines = text.splitlines()
words = text.split()
longest = max(words, key=len)
print(f"الأسطر: {len(lines)}")
print(f"الكلمات: {len(words)}")
print(f"أطول كلمة: {longest}")
```

تمرين 2: عكس الأسطر

```
from pathlib import Path

text = Path("input.txt").read_text(encoding="utf-8")
reversed_lines = "\n".join(reversed(text.splitlines()))
.txt").write_text(reversed_lines, encoding="utf-8")
```

تمرين 3: البحث في مجلد

```
from pathlib import Path

:def search_in_folder(folder, keyword)
[] = matches
:for file in Path(folder).glob("*.txt")
keyword in file.read_text(encoding="utf-8")
matches.append(file.name)
return matches

((بـايـثـون" ,"documents")search_in_folder)print
```

تمرين 4: ملف سجل

```
from datetime import datetime
from pathlib import Path

:def log(message)
= datetime.now().strftime("%Y-%m-%d %H:%M:%S")
"line = f"[{now}] {message}\n
```

```
ith open("log.txt", "a", encoding="utf-8") as f
                                    f.write(line)

log("بدء البرنامج")
log("نجحت العملية")
```

حلول الفصل التاسع: JSON و CSV

تمرين 1: إنشاء JSON والمتوسط

```
import json
from pathlib import Path

students = [
{"name": "أحمد", "city": "age": 22, "grade": 88}
{"name": "محمد", "city": "age": 25, "grade": 92}
{"name": "خالد", "city": "age": 19, "grade": 75}
{"name": "فاطمة", "city": "age": 21, "grade": 95}
{"name": "مريم", "city": "age": 23, "grade": 80}
]
```

```
)Path("students.json").write_text
n.dumps(students, ensure_ascii=False, indent=2)
"encoding="utf-8
(

(Path("students.json").read_text(encoding="utf-8"))
avg = sum(s["grade"] for s in loaded) / len(loaded)
("{avg:.2f} :المتوسط"f)print
```

تمرين 2: فصل الناجحين والراسبين

```
import csv

pen("students.csv", "r", encoding="utf-8-sig") as f
students = list(csv.DictReader(f))

ed = [s for s in students if int(s["grade"]) >= 60]
ed = [s for s in students if int(s["grade"]) < 60]

n [("passed.csv", passed), ("failed.csv", failed)]
```

```
me, "w", encoding="utf-8-sig", newline="") as f
:if data
ictWriter(f, fieldnames=data[0].keys())
()writer.writeheader
writer.writerows(data)
```

تمرين 3: JSON إلى CSV

```
import json
import csv
from pathlib import Path

ads(Path("books.json").read_text(encoding="utf-8"))

s.csv", "w", encoding="utf-8-sig", newline="") as f
iter(f, fieldnames=["title", "author", "year"])
()writer.writeheader
writer.writerows(books)
```

تمرين 4: pandas لأعلى 10 طلاب

```
import pandas as pd

= pd.read_csv("students.csv", encoding="utf-8-sig")
= df.sort_values("grade", ascending=False).head(10)
print(top10)
```

حلول الفصل العاشر: HTTP

تمرين 1: جدول طقس ثلاث مدن

```
import requests

} = cities
"عدن": (12.78, 45.04),
"تعز": (13.58, 44.02),
"الحديدة": (14.80, 42.95),
{

:()for name, (lat, lon) in cities.items
```

```
)r = requests.get
,"https://api.open-meteo.com/v1/forecast"
longitude": lon, "current_weather": "true"}
timeout=10
(
w = r.json()["current_weather"]
d']} ريـاح ،f"{name}: {w['temperature']}°C)print
```

تمرين 2: مستودعات GitHub

```
import requests

)r = requests.get
'https://api.github.com/users/tensorflow/repos"
,params={"per_page": 5, "sort": "updated"}
timeout=10
(

:()for repo in r.json
repo['name']}: {repo['stargazers_count']})print
```

تمرين 3: safe_get

```
import requests
import time

:def safe_get(url, max_retries=3)
    :for attempt in range(1, max_retries + 1)
        :try
    r = requests.get(url, timeout=10)
        ()r.raise_for_status
        ()return r.json
s.ConnectionError, requests.HTTPError) as e
    ("{e} :{attempt} محاولة"f)print
        :if attempt < max_retries
        time.sleep(2 ** attempt)
return None
```

تمرين 4: ويكيبيديا

```
import requests
from pathlib import Path
```

```
title = "اليمن"
//ar.wikipedia.org/api/rest_v1/page/summary/{title}
r = requests.get(url, timeout=10)
data = r.json()
txt").write_text(data["extract"], encoding="utf-8")
print("تم الحفظ.")
```

حلول الفصل الحادي عشر: الأخطاء

تمرين 1: safe_int

```
def safe_int(text):
    try:
        return int(text)
    except ValueError:
        print(f"تحذير: '{text}' ليس رقمًا، أُرجعت 0")
        return 0
```

تمرين 2: قراءة JSON آمنة

```
import json
from pathlib import Path

:def read_json_safely(path)
:try
ads(Path(path).read_text(encoding="utf-8"))
xcept (FileNotFoundError, json.JSONDecodeError)
{} return
```

تمرين 3: خطأ مخصص

```
:class InsufficientBalanceError(Exception)
pass

:def withdraw(account, amount)
:if amount > account["balance"]
)raise InsufficientBalanceError
f"الرصيد {account['balance']} لا يكفي لـ
(
```

```
    account["balance"] -= amount
    return account["balance"]

acct = {"balance": 100}
try:
withdraw(acct, 200)
except InsufficientBalanceError as e:
print(e)
```

تمرين 4: لعبة تخمين آمنة

```
import random

secret = random.randint(1, 100)
attempts = 0

while attempts < 7:
raw = input("خمّن: ")
try:
guess = int(raw)
except ValueError:
```

```
print("أدخل رقمًا صحيحًا")
continue
attempts += 1
if guess == secret:
print(f"صحيح! في المحاولة {attempts}")
break
print("أكبر" if guess < secret else "أصغر")
else:
print(f"الرقم كان {secret}")
```

حلول الفصل الثاني عشر: مفاتيح API

تمرين 1: مشروع آمن

```
mkdir secure-project
cd secure-project
echo "API_KEY=" > .env.example
echo "API_KEY=test123" > .env
echo ".env" > .gitignore
echo "venv/" >> .gitignore
```

```
git init
git status        # يجب ألا يظهر .env
```

تمرين 2: get_required_env

```
import os

def get_required_env(name):
    value = os.environ.get(name)
    if not value:
        raise SystemExit(f"المتغير المطلوب '{name}'")
    return value
```

تمرين 3: عرض المفاتيح بأمان

```
from dotenv import dotenv_values

values = dotenv_values(".env")
for key, value in values.items():
    if value:
        masked = value[:4] + "*" * (len(value) - 4)
    else:
```

```
"(فـارغ)" = masked
print(f"{key} = {masked}")
```

تمرين 4: git-secrets

> *git-secrets* أداة مفتوحة المصدر تفحص أي *commit* جديد قبل الرفع، بحثًا عن أنماط مفاتيح معروفة (مفاتيح *GitHub tokens*، *AWS*، إلخ). إن وجدت تطابقًا، تمنع الـ *commit*. تُثبَّت بـ `brew install git-secrets` أو من المصدر، وتُفعَّل في كل مستودع بـ `git secrets --install`. تحمي المبرمج من الخطأ الكلاسيكي لرفع مفتاح بالخطأ.

حلول الفصل الثالث عشر: LLM APIs

تمرين 1: 5 أسئلة عن اليمن

```
from groq import Groq
from pathlib import Path
```

```
client = Groq()
questions = [
    "ما عاصمة اليمن؟",
    "ما أبرز المعالم التاريخية في اليمن؟",
    "كم عدد سكان اليمن تقريبًا؟",
    "ما اللغة الرسمية في اليمن؟",
    "ما أهم الموانئ اليمنية؟",
]

answers = []
for q in questions:
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": q}],
        max_tokens=200,
    )
    answers.append(f"س: {q}\nج: {r.choices[0].message.content}")

Path("answers.txt").write_text("\n".join(answers), encoding="utf-8")
```

تمرين 2: Temperature والتنوع

```
from groq import Groq

client = Groq()
question = "اكتب جملة شعرية عن البحر."

for i in range(3):
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": question}],
        max_tokens=80,
        temperature=1.0
    )
    print(f"--- محاولة {i+1} ---")
    print(r.choices[0].message.content)
```

تمرين 3: محادثة مع ذاكرة

```
from groq import Groq
```

```
client = Groq()
history = []

while True:
    user = input("أنت: ")
    if user.lower() in {"exit", "quit", "خروج"}:
        break
    history.append({"role": "user", "content": user})
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=history,
        max_tokens=300,
    )
    assistant = r.choices[0].message.content
    history.append({"role": "assistant", "content": assistant})
    print(f"النموذج: {assistant}")
```

تمرين 4: مقارنة Claude و Llama في الترجمة

```
# يتطلب ANTHROPIC_API_KEY و GROQ_API_KEY
from anthropic import Anthropic
```

```
from groq import Groq

haping every industry, from healthcare to education

)claude = Anthropic().messages.create
,"model="claude-haiku-4-5-20251001
,max_tokens=200
"role": "user", "content": f"ترجم إل}]=messages
content[0].text.(

)llama = Groq().chat.completions.create
,"model="llama-3.3-70b-versatile
"role": "user", "content": f"ترجم إل}]=messages
,max_tokens=200
choices[0].message.content.(

print("Claude:", claude)
()print
print("Llama:", llama)
```

كلمة ختامية

أكملت تمارين الكتاب. عدد المسائل التي حللتها يقاس بالعشرات، وعدد دقائق الـ debugging بالمئات. أنت الآن مبرمج بايثون. ليس مبتدئًا — مبرمج. خذ هذه الثقة معك إلى كتاب RAG التالي.

الفصل السادس عشر: الخاتمة والجسر إلى عالم RAG

> "والعلم في الصدر كالغرس في الأرض، فاسقه بالعمل."

أيها الطالب، أيتها الطالبة:

أنت الآن في صفحة لم تكن تتخيلها قبل أسبوعين. حين فتحت الفصل الأول، لم تكن قد كتبت سطر بايثون واحدًا. والآن، تستطيع أن تقرأ ملفات، تستدعي نماذج الذكاء الاصطناعي، تعالج البيانات، وتبني خطوط أنابيب كاملة. هذه قفزة كبيرة. خذ لحظة لتقدر ما أنجزته.

لكن الرحلة لم تنتهِ. هذا الكتاب هو البوابة الأولى. أمامك بوابات كثيرة. هذا الفصل القصير يخبرك أين تذهب الآن، وكيف تستعد للكتاب التالي في برنامج LLM4Yemen.

ماذا تعرف الآن؟

دعني ألخص ما أصبحت قادرًا على فعله:

في الأساسيات: تعرف كيف تعمل لغة برمجة، وكيف تترجم فكرتك إلى كود ينفّذها بايثون. تستطيع كتابة دالة، تنظيمها في ملف، استيرادها في برنامج آخر. لديك حدس قوي للأنواع: متى تستخدم list، متى dict، متى set.

في التعامل مع البيانات: تستطيع قراءة وكتابة الملفات النصية، CSV، JSON، حتى العربية بترميز UTF-8. تعرف الفروق الدقيقة (مثل `utf-8-sig` لـ Excel، و `ensure_ascii=False` لـ JSON العربي).

في الشبكة: تستطيع طلب أي API على الإنترنت، فهم استجابته، التعامل مع أخطائه، إعادة المحاولة عند فشله.

في الأمن: لن تكتب مفتاح API في الكود مرةً أخرى. تعرف `.env` و `.gitignore` كأصدقاء قدامى.

في الذكاء الاصطناعي: استدعيت نماذج GPT، Claude، و Llama. حصلت على ردود عربية. بنيت مشروع تلخيص حقيقي.

هذه ليست قائمة مهارات هواية. هذه قائمة مهارات مهنية. أي شركة تقنية في العالم اليوم تبحث عن أشخاص يستطيعون فعل ما تستطيع فعله.

ما الفجوة بين هذا الكتاب وكتاب RAG؟

كتاب RAG التالي في برنامج LLM4Yemen يفترض أنك: - تكتب بايثون بطلاقة - تعرف JSON و CSV - تستدعي APIs بثقة - تتعامل مع المفاتيح بأمان - تكتب دوال نقية، توثقها، تختبرها

كل هذه فرضيات صحيحة الآن. أنت جاهز.

ما ستضيفه على معرفتك في كتاب RAG:

مفاهيم جديدة: التضمين (embeddings)، قواعد المتجهات (vector databases)، البحث الدلالي (semantic search)، تقطيع النصوص (chunking)، تقييم النظام (evaluation)، التقطيع المتداخل (overlapping chunks)، إعادة الترتيب (re-ranking).

مكتبات جديدة: Pinecone، ChromaDB، LlamaIndex، LangChain، sentence-transformers، Python client.

أنماط معمارية: خطوط أنابيب الاستيعاب (ingestion pipelines)، خطوط أنابيب الاستعلام (query pipelines)، طبقات التخزين المؤقت، التقييم الآلي، النشر السحابي.

كل واحد منها سيُبنى على ما تعلمته. لا شيء يبدأ من الصفر.

نصائح للأسبوعين القادمين قبل بدء كتاب RAG

أولًا: استمر في الكتابة. التوقف عن كتابة الكود لأسبوعين سيضعف مهاراتك. اكتب 30 دقيقة على الأقل يوميًا. أعد فتح الفصول، حلّ تمارين أخرى، اخترع مسائل صغيرة لنفسك.

ثانيًا: اقرأ كود الآخرين. زر GitHub، ابحث عن مشاريع بايثون صغيرة، اقرأها. لاحظ كيف يكتب المحترفون، كيف ينظمون الملفات، كيف يسمّون المتغيرات. هذه ثقافة تكتسب بالتعرض.

ثالثًا: لا تعتمد كليًا على ChatGPT أو Claude. نعم، يمكنك أن تطلب منهما يكتبا الكود لك. لكن في المراحل المبكرة، هذا يضرّك. اكتب أنت أولًا. ثم اطلب منهما مراجعة كودك واقتراح تحسينات. هكذا تتعلم منهما، لا تعتمد عليهما.

رابعًا: شارك تجاربك. اكتب منشورًا قصيرًا على LinkedIn أو X عن ما تعلمته. علّم زميلًا. الشرح يكشف الثغرات في فهمك ويملؤها.

خامسًا: ابدأ مشروعًا شخصيًا. اختر مشكلة تهمك (مهما كانت بسيطة)، وحاول حلّها بالكود. ربما تطبيق صغير لتنظيم قراءاتك، أو سكربت لتحليل ملفات Excel لعملك. المشروع الشخصي يجبرك على ربط كل المهارات معًا.

ست مكتبات يجب أن تستكشفها

قبل بدء كتاب RAG، أمضِ ساعة مع كل من هذه (لا تتعمق، فقط تعرّف):

1. **pandas** — لتحليل البيانات الجدولية. إن لم تستخدمها بعد، شاهد فيديو 10 دقائق على YouTube عنها.
2. **numpy** — للأرقام والمصفوفات. أساس كل مكتبات التعلم الآلي.
3. **tqdm** — شريط تقدّم للحلقات الطويلة. سطر واحد ينقذ صبرك في خطوط أنابيب طويلة.
4. **rich** — تنسيق جميل للطرفية: جداول، ألوان، شريط تقدّم. يجعل سكربتاتك تبدو احترافية.
5. **typer** — بناء واجهات سطر أوامر بسرعة. أبسط من argparse.
6. **pytest** — اختبار كودك تلقائيًا. حتى إن لم تكتب اختبارات، تعرّف على الفلسفة.

كل واحدة منها ستظهر في كتاب RAG، أو في عملك المهني لاحقًا.

كلمة شخصية أخيرة

عزيزي القارئ:

كتبت هذا الكتاب وأنا أتخيلك. تخيلتك في صنعاء أو عدن أو تعز أو حضرموت. تخيلت حاسوبك القديم، وانقطاع التيار الذي يقاطع تركيزك، والبيانات الباهظة التي تستهلكها على Colab. تخيلت أحلامك أن تكون من بناة المستقبل التقني لليمن.

هذه الأحلام ليست بعيدة. كل تقني عالمي بدأ من حيث أنت الآن: من فصل أول لا يفهم منه شيئًا، من برنامج أول لا يعمل، من خطأ أول يظنه نهاية العالم.

اليمن يستحق جيلًا من المهندسين الذين يبنون لا يستهلكون. الذين يصنعون أدواتهم لا ينتظرون من يصنعها لهم. الذين يربطون التكنولوجيا بمشاكل بلدهم الحقيقية. أنت من هذا الجيل.

استمر. التزم. ثق.

أراك في كتاب RAG.

— د. فضل محمد الأكوع

﴿وَقُل رَّبِّ زِدْنِي عِلْمًا﴾

رسالة مفتوحة للطلاب الذين يدرسون مع زملاء

البرمجة رياضة جماعية. اعقد جلسات أسبوعية مع زملائك:

- **مراجعة الكود**: كل أسبوع، يقدّم طالب مشروعه ويناقشه الباقون.
- **حل تحديات معًا**: اختاروا تمرينًا من الكتاب، وحلّوه على لوح أو شاشة مشتركة.
- **التعلم من الأخطاء**: شاركوا أصعب مشاكل صادفتموها وكيف حللتموها.
- **التحضير لكتاب RAG**: ابدؤوا بقراءة مقدمة كتاب RAG معًا، واطرحوا الأسئلة التي تختمر في أذهانكم.

الجماعة تضاعف معدّل تعلمكم. لا تتعلموا منعزلين.

قائمة فحص نهائية

قبل أن تفتح كتاب RAG، تأكد أنك تستطيع:

- [ ] إنشاء مشروع جديد ببيئة افتراضية وملف requirements.txt
- [ ] قراءة ملف CSV عربي وحفظ نسخة معدلة منه
- [ ] استدعاء أي API يُرجع JSON والتعامل مع أخطاء HTTP
- [ ] تخزين مفتاح API في `.env` واستيراده بـ python-dotenv
- [ ] استدعاء نموذج لغوي والحصول على رد عربي صحيح

- [ ] كتابة دالة تأخذ معطيات، تعالجها، وتُرجع نتيجة
- [ ] التعامل مع `try/except` لمنع انهيار البرنامج
- [ ] استخدام pathlib لإدارة المسارات بشكل عابر للأنظمة
- [ ] فهم الفرق بين list و dict و set و tuple وموعد استخدام كل منها
- [ ] قراءة رسالة خطأ Python وتحديد سببها بسرعة

كل صح في هذه القائمة هو حصن منيع. ضع علامة ✓ على ما تتقنه، وراجع ما لم تتقنه قبل البداية.

شكرًا لقراءتك حتى هذه السطر. شكرًا لاختيارك تعلم الذكاء الاصطناعي. شكرًا لإيمانك بنفسك.

والآن، اذهب وابنِ شيئًا لم يره العالم من قبل.

المراجع

التوثيق الرسمي

- توثيق بايثون 3 الرسمي — https://docs.python.org/3
- دليل بايثون للمبتدئين — https://docs.python.org/tutorial/3
- مرجع المكتبة المعيارية — https://docs.python.org/3/library
- PEP 8: دليل أسلوب كتابة الكود — https://peps.python.org/pep-0008
- PEP 257: اتفاقيات docstring — https://peps.python.org/pep-0257

واجهات النماذج اللغوية

- توثيق Anthropic API — https://docs.anthropic.com

- توثيق **OpenAI API** — https://platform.openai.com/docs
- توثيق **Groq Cloud** — https://console.groq.com/docs
- **OpenRouter** — بوابة موحدة لعشرات النماذج — https://openrouter.ai/docs
- **Hugging Face Inference API** — https://huggingface.co/docs/api-inference

مكتبات أساسية

- **requests** — طلبات **HTTP** — https://requests.readthedocs.io
- **python-dotenv** — تحميل ملفات **.env** — https://github.com/theskumar/python-dotenv
- **pandas** — تحليل البيانات — https://pandas.pydata.org/docs/
- **numpy** — الحوسبة العددية — https://numpy.org/doc/

- **pathlib** — مسارات الملفات — https://
docs.python.org/3/library/pathlib.html
- **json** — معالجة JSON — https://
docs.python.org/3/library/json.html
- **csv** — معالجة CSV — https://
docs.python.org/3/library/csv.html

بيئات التطوير

- **Visual Studio Code** — https://
code.visualstudio.com
- **Google Colab** — https://
colab.research.google.com
- **Jupyter Notebooks** — https://jupyter.org
- **PyCharm Community** — https://
www.jetbrains.com/pycharm

مصادر تعليمية مفتوحة

- **Real Python** — مقالات وتمارين — https://realpython.com
- **Python.org Beginner Guide** — https://www.python.org/about/gettingstarted/
- **Automate the Boring Stuff with Python** (كتاب مجاني) — https://automatetheboringstuff.com
- **Think Python (Allen B. Downey)** (كتاب مجاني) — https://greenteapress.com/wp/think-python-3rd-edition/
- **Composing Programs (UC Berkeley CS61A)** — https://composingprograms.com

مجتمعات ومناقشات

- **Stack Overflow** — أسئلة وإجابات — https://stackoverflow.com/questions/tagged/python

- **Python Discord** — https://pythondiscord.com
- **r/learnpython على Reddit** — https://www.reddit.com/r/learnpython
- **Real Python Newsletter** — https://realpython.com/newsletter

معالجة النصوص العربية

- **CAMeL Tools — أدوات معالجة اللغة العربية** — https://github.com/CAMeL-Lab/camel_tools
- **PyArabic — مكتبة عربية صغيرة** — https://github.com/linuxscout/pyarabic
- **Arabic NLP Resources** — https://github.com/SOMASS/Arabic-NLP-Resources

الأمن والمفاتيح

- **OWASP Secrets Management Cheat Sheet** — https://cheatsheetseries.owasp.org

/cheatsheets

Secrets_Management_Cheat_Sheet.html

- **git-secrets** — https://github.com/awslabs/

git-secrets

- **GitHub Secret Scanning** — https://

docs.github.com/en/code-security/

secret-scanning

أدوات المبرمج اليومية

- **Git** — التحكم في الإصدار — https://git-scm.com/

doc

- **GitHub Skills** — https://skills.github.com
- **Conventional Commits** — https://

www.conventionalcommits.org

- **The Twelve-Factor App** — https://

12factor.net

كتب مرجعية موصى بها

- **Fluent Python (Luciano Ramalho)** — أعمق كتاب لإتقان بايثون.
- **Effective Python (Brett Slatkin)** — 90 طريقة لتحسين كودك.
- **Python Tricks (Dan Bader)** — مقتطفات قصيرة لتحسين أسلوبك.
- **Architecture Patterns with Python (Harry Percival)** — تنظيم المشاريع الكبيرة.
- **Designing Data-Intensive Applications (Martin Kleppmann)** — أساس فهم خطوط أنابيب البيانات.

مصادر متخصصة في الذكاء الاصطناعي

- **Anthropic Cookbook** — `https://github.com/anthropics/anthropic-cookbook`
- **OpenAI Cookbook** — `https://cookbook.openai.com`
- **LangChain Documentation** — `https://python.langchain.com`

- **LlamaIndex Documentation** — https://
docs.llamaindex.ai
- **Hugging Face Course** — https://
huggingface.co/learn

معايير ترميز النصوص

- **Unicode Standard** — https://
www.unicode.org/standard/standard.html
- **UTF-8 Explained** — https://
en.wikipedia.org/wiki/UTF-8
- **W3C Internationalization** — https://
www.w3.org/International/

أدوات تحرير وتنسيق الكود

- **Black** — منسّق كود تلقائي — https://
black.readthedocs.io
- **isort** — ترتيب الـ imports — https://
pycqa.github.io/isort/

- **flake8** — **فحص أسلوب الكود** — https://
  flake8.pycqa.org
- **mypy** — **فحص الأنواع** — https://
  mypy.readthedocs.io
- **ruff** — **أداة فحص حديثة وسريعة** — https://
  docs.astral.sh/ruff

كل المراجع أعلاه مجانية. كثير منها متاح للقراءة دون إنشاء حساب. ابدأ بما يخصّ مشكلتك الحالية، ووسّع تدريجيًا.

القسم الإنجليزي

English Section

Table of Contents

Introduction: Why Python, and Why Now? 1

Chapter 2: Setting Up Your Workspace 2

Chapter 3: Your First Program and How Python Thinks 3

Chapter 4: Variables and Types 4

Chapter 5: Controlling Program Flow 5

Chapter 6: Functions 6

Chapter 7: Data Structures 7

Chapter 8: Reading and Writing Files 8

Chapter 9: Working with JSON and CSV 9

Chapter 10: HTTP Requests and the requests Library 10

Chapter 11: Error Handling and Debugging Strategies 11

Chapter 12: Environment Variables and Safely Storing API Keys 12

Chapter 13: Calling Large Language Models from Python 13

Chapter 14: Capstone Project — Arabic CSV Summarizer with AI 14

Chapter 15: Exercise Solutions 15

Chapter 16: Conclusion and Bridge to the RAG World 16

References 17

Introduction: Why Python, and Why Now?

> *"And say: My Lord, increase me in knowledge."*
>
> *Surat Ta-Ha — Verse 114*

A Word to the Yemeni Student

Dear student of LLM4Yemen Track B,

If you are opening this book having never written a line of code in your life, know that you are in the right place. This book was not written for software engineers in Silicon Valley. It was written for you: the Yemeni student studying on a laptop with 8 GB of RAM, who may lose electricity twice a day, who wants to learn artificial intelligence but first needs the basic tools.

Programming in Python is your gateway into the world of AI. Every large language model you use, and every application powered by AI, depends on Python at its core. Over the next two weeks, you will learn enough Python to build your own system in the next book on Retrieval-Augmented Generation (RAG).

Why Python in Particular?

Imagine you wanted to learn a new language to speak with some people. Would you choose a language spoken by one million people, or one spoken by tens of millions? Python is the official language of the global AI community. When a researcher at Stanford publishes a new paper, or when Anthropic releases a new model, the first code they release is in Python.

Four reasons make Python your first choice:

First: simplicity. Python is the closest programming language to spoken English. The following line actually works:

```
if temperature > 30:
    print("It is very hot")
```

This line needs no long explanation. You read it and you understand: "if the temperature is greater than 30, print: It is very hot."

Second: ready libraries. Almost everything you need in AI is already written in Python. Want to read a CSV file? One line. Want to call a large language model from Anthropic? Five lines. Want to build a vector database for a RAG system? All the libraries are ready.

Third: community. When you face a problem in your code, thousands of programmers have faced it before you. You will find the answer on Stack Overflow and on GitHub. You will find video tutorials on YouTube in Arabic and English.

Fourth: it is free. Python is completely free. You don't need a license, you don't pay annual fees. Everything you will learn in this book you can run without paying a single riyal.

How Does This Book Bridge to the Next RAG Book?

Your next book in the LLM4Yemen program is "RAG Engineering: From Concept to Production." Every skill you learn here you will use there directly. The following table shows the bridge between the two books:

What you learn in this book	How you'll use it in the RAG book
Reading and writing files (Chapter 8)	Loading documents (PDF, text, Word) into a RAG system
Dictionaries and lists (Chapter 7)	Storing metadata for vector chunks
HTTP requests and `requests` library (Chapter 10)	Calling embedding APIs from OpenAI and Voyage
Working with JSON and CSV (Chapter 9)	Receiving search results and formatting them for the LLM

What you learn in this book	How you'll use it in the RAG book
Environment variables and `.env` (Chapter 12)	Hiding API keys for Pinecone and Weaviate
Calling LLM APIs (Chapter 13)	Building the generation layer of a RAG system
Error handling (Chapter 11)	Making your system resilient when any pipeline step fails
Capstone project (Chapter 14)	The complete foundation for building your first experimental RAG system

Study this table. It tells you that every minute you spend learning Python is a direct investment in your ability to build AI systems later.

How to Use This Book

This book is designed for two weeks of intensive study, at the rate of one or two full chapters per day. But speed is not the goal; the goal is for you to **type every line of code yourself**.

Do not copy code from the book and paste it directly. Type it. Type it character by character with your fingers. Your finger will mistype at first, and Python will tell you that you forgot a parenthesis or a comma, and you will learn from this mistake more than you would learn from reading the code already written in front of you. This is the golden rule for beginners.

Each chapter of this book contains: first, an explanation in simple language of a new concept, with examples from your daily life. Second, short code examples, each line of which is explained. Third, exercises at the end of the chapter to test your understanding by yourself. Fourth, the answers to these exercises are in a special chapter at the end of the book — but do not look at them before you have tried for at least a full hour by yourself. Patience in trying is what makes a programmer.

What You Need Before You Start

You don't need any prior programming knowledge. You only need:

A computer running Windows, macOS, or Linux, with at least 4 GB of RAM (this book is specifically designed to work on 8 GB laptops). An internet connection to download tools at the start and to access Google Colab. Patience for two weeks. Passion to be among the first AI builders in Yemen.

If your computer is very weak, or if power outages are frequent, don't worry. We will teach you to use Google Colab, a complete Python environment that runs in a web browser and requires no installation on your computer.

A Final Tip Before You Begin

Programming is not an innate talent. It is a skill acquired through repetition. You will not understand everything from the first reading. This is normal. Your brain on day one will think you will never be able to do this. Then on day five, problems that confused you on day one will dissolve in front of you. This is the journey of every programmer in the world. Trust the process, trust yourself, and start.

"And say: My Lord, increase me in knowledge."

Now go to Chapter 2 and begin setting up your environment.

Chapter 1 Exercises

1. Write a short paragraph (5 lines) about why you want to learn AI. Keep this paper. Read it after two weeks when you finish the book.
2. Search the internet for "What is the difference between Python 2 and Python 3?" and write the answer in five lines.
3. List three products or applications you use daily that depend on AI.

Chapter 2: Setting Up Your Workspace

> *"A scholar without tools is a knight without a horse."*
>
> *Arabic proverb*

Before you write your first line of Python, you need to prepare your workshop. As a carpenter does not begin work before bringing out his saw and hammer, you will need essential tools: the Python interpreter itself, an intelligent text editor that helps you write code, and a backup cloud environment for when the power goes out.

In this chapter we cover three paths: the local path on your computer using VS Code, the cloud path using Google Colab, and organizing virtual environments to prevent conflicts between projects.

First: Installing Python on Your Computer

Begin by visiting the official Python website: `python.org`. Click the "Downloads" button and choose the latest stable version — at the time of this writing it is version 3.12. Avoid versions marked with (a), (b), or (rc) because they are experimental.

When installing Python on Windows, stop at the first installer screen and make sure to enable the option **Add Python to PATH**. This option is critical — without it your system will not recognize Python from the command line, and you will spend hours solving a problem that was a single click. If you forgot this option, uninstall Python and reinstall.

After installation completes, open "Command Prompt" on Windows or "Terminal" on macOS and Linux, and type:

```
python --version
```

It should print a line resembling: `Python 3.12.3`. If this line appears, you succeeded. If you see an error, return to the installation steps and ensure the PATH option was enabled.

Second: Installing the VS Code Editor

Python itself does not include a text editor. You need a tool that helps you write code, color important keywords, and warn you of errors before running the program. The best free choice is Visual Studio Code from Microsoft.

Visit `code.visualstudio.com` and download the version for your system. After installation, open VS Code and go to the Extensions icon in the side bar. Search for "Python" and install the official extension from Microsoft. This extension adds one-click code execution, keyword highlighting, and autocomplete suggestions.

I also recommend installing the following extensions:

Pylance — provides smarter writing assistance and detects errors before runtime.

Jupyter — lets you open interactive notebook files (`.ipynb`) directly inside VS Code.

Arabic Language Pack — if you want VS Code's interface in Arabic. (Personally I recommend keeping it in English; most learning resources use English terminology, and you'll find help online faster.)

Third: Your First Project — Testing the Setup

Create a new folder somewhere convenient on your computer, for example `C:\python-projects` on Windows or `~/python-projects` on macOS. Open VS Code, and from the File menu choose Open Folder, then select this folder.

Inside VS Code, create a new file named `hello.py`. (The `.py` extension is important — it tells VS Code this is a Python file). Paste the following line:

```
print("Hello, world! I am learning Python.")
```

Save with `Ctrl+S`. Now open the Terminal inside VS Code from View → Terminal. Type:

```
python hello.py
```

The line will print in the Terminal:

```
Hello, world! I am learning Python.
```

Congratulations! You ran the first Python program in your life.

Fourth: Google Colab — The Cloud Alternative

Your computer may not always work. Power may cut out, or you may be at the university away from your computer. This is where Google Colab comes in — a complete Python environment that runs through the browser, free, requiring no installation.

Visit `colab.research.google.com`. Sign in with a Gmail account. Click "New Notebook". A page opens with an empty box called a "cell". Type in it:

```
print("Hello from the cloud")
2 + 2
```

Press the play button (right triangle) or `Shift+Enter`. The result will appear directly under the cell. This is the magic of Colab: no separate files, no Terminal — everything in one page.

Additional Colab features: you get a free GPU for training AI models. You get 12 GB of RAM free. Your files are saved to Google Drive automatically. And most important: your work doesn't get lost if the power goes out at home.

Colab disadvantages: needs a constant internet connection. The session ends after 12 hours of inactivity. Not suitable for applications that need graphical windows.

My practical advice: use VS Code for serious long-running projects, and use Colab for quick experiments and for working when the power is out.

Fifth: Virtual Environments

Imagine you have two projects: an old project that uses a specific version of the `requests` library, and a new project that needs a newer version. If you install both versions together in Python, the libraries will conflict and neither project will work. Here comes the idea of a "virtual environment": a separate box for each project, containing its own versions of libraries.

To create a virtual environment, navigate to your project folder in Terminal, then type:

```
python -m venv venv
```

This command creates a new folder called `venv` containing an isolated copy of Python. To activate the environment:

On Windows:

```
venv\Scripts\activate
```

On macOS and Linux:

```
source venv/bin/activate
```

You will notice the word `(venv)` appearing at the start of the Terminal line. This means you are now inside the virtual environment. Any library you install with `pip install` will go to this environment only, and will not affect the rest of your computer.

To install your first library, try:

```
pip install requests
```

The `requests` library (which we will use in Chapter 10) will be installed inside your environment. To verify:

```
pip list
```

This will display all installed libraries. To deactivate the environment when finished:

```
deactivate
```

Golden tip: Create a new virtual environment for every project. Build this habit from now. You will see later — when

you work on a RAG system needing specialized libraries — that the virtual environment protects you from chaos you can't imagine.

Sixth: requirements.txt

When your project ends, you can save the list of all libraries you used in a text file:

```
pip freeze > requirements.txt
```

This file becomes your project's "recipe". When a colleague comes later wanting to run your code, they only type:

```
pip install -r requirements.txt
```

And all libraries install automatically with the same versions you used. This is what programmers call "reproducibility" — the ability of someone else to run your code the same way it works for you.

Chapter Summary

Your workshop is ready. Python installed, VS Code working, Google Colab available as backup, and the virtual environment protecting your projects from conflict. In the next chapter, you will write your first real program and understand how Python thinks.

Chapter 2 Exercises

1. Install Python on your computer and type `python --version` to confirm. Take a screenshot if you wish to share with classmates.
2. Create a folder named `learning-python`, and inside it create a virtual environment, activate it, then install the `requests` library, and type `pip list` to confirm.
3. Sign in to Google Colab, create a new notebook, and write a program that prints your name in Arabic three times.

4. Create a `requirements.txt` file for your project, open it in VS Code, and view its content.

Chapter 3: Your First Program and How Python Thinks

> *"If you sit well with yourself for an hour, you have done two hours of work."*
> *Old wisdom*

In this chapter you will understand how Python reads code, how it executes commands line by line, and how you talk to it through printing (`print`) and through input (`input`). These are the first building blocks of any program, and you will see them in every line you write afterwards.

How Does Python Think?

Python is an interpreter, not a compiler. The important difference for you as a beginner: Python reads your code line

by line, from top to bottom of the file, and executes each line on its own. If it reaches a line containing an error, it stops execution and tells you the error. This means you can write a line, run it, see the result, then write the next line. This magical cycle is called the "interactive loop" (REPL: Read-Eval-Print Loop).

Open Terminal and type the command `python` only, without a file name. The marks >>> will appear waiting for you to type. This is the interactive loop. Try:

```
>>> 2 + 2
4
>>> print("Hi")
Hi
>>> name = "Fadhl"
>>> print("Hello,", name)
Hello, Fadhl
>>> exit()
```

In Colab, each cell itself acts as a small interactive loop. You write code, press `Shift+Enter`, and the result appears immediately.

The `print` Function — How You Talk to the Screen

`print` is the most important function you will learn. Whatever you write between its parentheses will appear on the screen. Examples:

```
print("Hello")                  # prints: Hello
print("My name is Fadhl")       # prints: My name is F
print(2 + 3)                    # prints: 5
print("The result is:", 2 + 3)  # prints: The resul
```

Notice three important things: - Strings are placed inside quotation marks: `"..."` or `'...'`. Either is correct as long as the start and end match. - Numbers are written without quotes. If you put a number in quotes it becomes a string, not a

number. - You can pass several values separated by commas. Python automatically puts a space between them.

Separators and Endings

If you want to change the separator between values, use the `sep` parameter:

```
print("Ahmed", "Ali", "Khalid", sep=" and ")
# prints: Ahmed and Ali and Khalid
```

And if you don't want Python to put a new line at the end, use `end`:

```
print("First part, ", end="")
print("second part")
# prints: First part, second part
```

The `input` Function — How the User Talks to You

`input` lets you ask the user to type something. For example:

```
name = input("What is your name? ")
print("Hello,", name)
```

When you run this program, the question "What is your name?" appears, and Python waits. When you type your name and press Enter, the name is saved in the variable `name`, then used in the next line.

Very important note: Everything the user enters via `input` is a **string**, even if they typed a number. If you ask for a user's age and want to add to a number, you must convert the string to a number:

```
age = input("How old are you? ")
print("Next year you'll be:", int(age) + 1)
```

`int(age)` converts the string to an integer. If you forgot this step and wrote `age + 1` directly, you would get a cryptic error called `TypeError: can only concatenate str (not "int") to str`.

Comments

Anything starting with # Python ignores. These are called "comments" and are used to explain what the code does to whoever reads it later — including yourself two months from now:

```
# This program calculates the area of a circle
radius = 5              # radius in meters
pi = 3.14159            # constant pi
area = pi * radius * radius
print("Area:", area)
```

Train yourself to comment from day one. Code without comments is a tightly woven puzzle, and even its author forgets the keys.

Spaces and Indentation

Python differs from most languages on a critical point: spaces at the start of a line **have meaning**. Python does not use curly

braces `{ }` to define code blocks; it uses indentation — the number of spaces at the start of each line.

You'll see in the next chapter how this works with `if` and `for`. For now just remember the rule: when code asks you to indent, use **4 spaces** or the `Tab` key. Don't mix spaces and Tab — choose one and stick with it throughout the whole file. VS Code will manage this automatically if you enable the Python extension.

A Complete Program: Simple Calculator

Let's apply what we've learned. Create a file called `calculator.py` and write:

```
# Simple calculator for two numbers
print("=== Simple Addition Calculator ===")

first = input("Enter the first number: ")
second = input("Enter the second number: ")
```

```
# Convert strings to numbers
num1 = float(first)
num2 = float(second)

result = num1 + num2

print("The result is:", result)
```

Run with `python calculator.py`. Enter two numbers. Look at the result. Now try entering a word instead of a number. What happens? You get an error message. This is the start of your journey with error handling — and we'll learn in Chapter 11 how to prevent the program from crashing.

`float` converts a string to a decimal number (like 3.14). And `int` converts a string to an integer (like 5). The difference we'll cover in detail in the next chapter.

Error Messages — Your Friend, Not Your Enemy

Many beginners panic when they see an error message in red. But Python error messages are among the clearest in the programming world. Learn to read them. Example:

```
Traceback (most recent call last):
  File "calculator.py", line 7, in <module>
    num1 = float(first)
ValueError: could not convert string to float: 'hel
```

This message tells you three important things: the file name where the error occurred, the line number (7), the type of error (`ValueError`), and why it happened (it cannot convert the string "hello" to a number). All you need to do is go to line 7 and understand why a string that doesn't fit reached it.

Practical Tip: Run the Code Often

Do not wait until you finish writing 50 lines before trying the program. Write 3 lines, run, verify, then add 3 more lines. This is the habit of professional programmers. If you wait too long before running, you'll accumulate 10 errors together and won't know which caused which.

Chapter Summary

You learned the `print` and `input` functions, how to write comments, how to read error messages, and that spaces at the start of a line matter in Python. You can now write simple programs that interact with the user. In the next chapter, we'll dive into variables and types — how Python stores data and how you choose the right type for each piece.

Chapter 3 Exercises

1. Write a program that asks the user for their name and country, then prints a welcome sentence including the

name and country. Example: "Welcome Ahmed from Sana'a".

2. Write a program that calculates the area of a rectangle. Ask the user for length and width, then print the area.
3. Modify the calculator program to also calculate difference (`-`), multiplication (`*`), and division (`/`), printing all four results.
4. Write a program asking for the year of birth and approximately calculating the user's age (assume current year 2026).

Chapter 4: Variables and Types

> *"Words carry meanings; types carry data."*

Every program saves data: a username, an age, a message, a list of products. The way Python stores this data is called "variables", and each variable has a "type" determining what it can hold and what it can do.

What Is a Variable?

Imagine you have boxes on a shelf, each with a labeled name. You put inside the "Ahmed" box a string "Mohammed". You put inside the "age" box the number 25. Whenever you want a value, you call the box's name:

```
name = "Mohammed"
age = 25
```

```
city = "Sana'a"

print(name)          # Mohammed
print(age)           # 25
print(city)          # Sana'a
```

The = sign is not "equals" in the math sense. It means "make the value of the variable on the left equal to what's on the right". This means the line `x = x + 1` is not nonsense — it says "take x's current value, add 1, and resave the result in x".

Variable Naming Rules

A variable name in Python follows simple rules: starts with a letter or underscore _, doesn't start with a digit. Doesn't contain spaces (use _ instead). Case-sensitive (`Name` and `name` are different variables). Cannot use Python's reserved words like `if`, `for`, `class`.

Good examples: `student_name`, `total_score`, `is_active`, `book_count`.

Bad examples: `1name` (starts with digit), `student name` (space), `class` (reserved word), `x` (non-descriptive).

The golden tip: choose descriptive names. A colleague will read your code, or you yourself in two months. The name `total_revenue_2026` is a thousand times clearer than `t`.

Basic Types

Python distinguishes between several data types. The most important:

String: a variable containing letters, words, or sentences. Written between quotes.

```
greeting = "Peace be upon you"
city = 'Sana\'a'
multiline = """text spanning
several
lines"""
```

Integer: a number without fractions.

```
age = 25
year = 2026
temperature = -5
```

Float: a number with fractions.

```
price = 19.99
pi = 3.14159
weight = 75.5
```

Boolean: a value of only two possibilities: `True` or `False`. (Notice the first letter is capital).

```
is_student = True
has_passport = False
```

None: a special value representing "nothing" or "undefined".

```
result = None
```

You'll see `None` often when you call a function that returns nothing.

Knowing a Variable's Type

Use the `type()` function to know any variable's type:

```
print(type("text"))      # <class 'str'>
print(type(25))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type(True))        # <class 'bool'>
print(type(None))        # <class 'NoneType'>
```

This function is a lifesaver during debugging. If your program doesn't behave as expected, print the variable's type and you'll understand.

Converting Between Types

Sometimes you need to convert a value from one type to another. Python provides conversion functions:

```
# string to number
age_text = "25"
age_number = int(age_text)
```

```
print(age_number + 1)        # 26

# number to string
score = 95
message = "Your score: " + str(score)
print(message)               # Your score: 95

# float to int (truncates the fraction, doesn't rou
price = 19.99
whole = int(price)
print(whole)                 # 19

# int to float
count = 5
ratio = float(count)
print(ratio)                 # 5.0
```

Warning: conversion may fail. `int("hello")` will give a `ValueError`. We'll learn in Chapter 11 how to protect our program from these errors.

Operations on Numbers

Python supports all the usual arithmetic operations:

```
a = 10
b = 3

print(a + b)        # 13   (addition)
print(a - b)        # 7    (subtraction)
print(a * b)        # 30   (multiplication)
print(a / b)        # 3.333... (regular division, a
print(a // b)       # 3    (integer division, ignor
print(a % b)        # 1    (remainder)
print(a ** b)       # 1000 (exponent: 10 to the pow
```

Know the difference between `/` and `//` well. `/` always returns a decimal value, even if 4/2 = 2.0. `//` returns an integer by truncating the fraction.

Operations on Strings

Strings in Python are very powerful. You can join them, repeat them, access their characters, change their case:

```
first = "Mohammed"
last = "Ali"

# concatenation
full = first + " " + last
print(full)                    # Mohammed Ali

# repetition
line = "-" * 20
print(line)                    # --------------------

# length
print(len(full))               # 12

# accessing a specific character (starts from 0)
print(full[0])                 # M
```

```
print(full[-1])                # i (last character)

# slicing
print(full[0:8])               # Mohammed

# case change
greeting = "Hello World"
print(greeting.upper())        # HELLO WORLD
print(greeting.lower())        # hello world
print(greeting.title())        # Hello World

# replacement
text = "I love coffee"
new_text = text.replace("coffee", "tea")
print(new_text)                # I love tea

# splitting a string into a list
sentence = "Ahmed,Mohammed,Khalid"
names = sentence.split(",")
print(names)                   # ['Ahmed', 'Mohammed
```

```
# joining a list into a string
joined = " - ".join(names)
print(joined)                    # Ahmed - Mohammed - F
```

Formatted Strings (f-strings)

Among the most important things Python recently learned: f-strings. An elegant way to compose strings with variables:

```
name = "Fadhl"
age = 35

# old way
print("My name is " + name + " and my age is " + st

# f-string way (better)
print(f"My name is {name} and my age is {age}")

# you can even put expressions inside {}
print(f"In 5 years my age will be {age + 5}")
```

```
# specifying decimal places
price = 19.998765
print(f"Price: {price:.2f}")      # Price: 20.00
```

Use f-strings always. They are the officially recommended method in modern Python, and you'll find them in every line of AI code you'll see.

Constants

Python doesn't formally support "constants" like some languages, but the convention is to write the variable name in capital letters to indicate it's constant and shouldn't be modified:

```
PI = 3.14159
MAX_RETRIES = 3
API_BASE_URL = "https://api.anthropic.com/v1"
```

Any programmer reading your code understands these are constant values, not modified during program execution.

Variables in Python Are Flexible (Dynamic Typing)

In Python, you can change a variable's type at any time:

```
x = 10            # x is now int
x = "text"        # x is now str
x = [1, 2, 3]     # x is now list
```

This is both a feature and a flaw. A feature because it's flexible. A flaw because it produces hard-to-detect errors. To avoid problems, agree with yourself that a variable keeps its type throughout the program. If you want to store something new, give it a new name.

Chapter Summary

You learned Python's basic data types: strings, integers and floats, booleans, and `None`. You learned how to convert between types, how to perform operations on numbers and

strings, and how to use f-strings for formatted printing. These building blocks will appear in every Python line you write.

Chapter 4 Exercises

1. Write a program that asks the user for a price and discount percentage, calculating the price after discount. Example: price 100, discount 20%, prints: Price after discount: 80.
2. Write a program that takes text from the user and prints: number of letters, the text reversed, the text in uppercase (if English).
3. Write a program that calculates the average of three grades entered by the user, printing the average to two decimal places using f-string.
4. What is the difference between `/`, `//`, and `%`? Write 3 examples for each.

Chapter 5: Controlling Program Flow

> *"The mind discriminates; logic decides."*

Everything we've written so far runs in a straight line: line after line from top to bottom. But real programs need **decisions** and **repetition**. To say to Python: "If a student's grade is above 90, print Excellent; otherwise print Good." Or: "Repeat this operation for every name in the list."

This is called "control flow", and it's the heart of any program.

The Conditional Statement: if / elif / else

The simplest decision in Python begins with `if`:

```
age = 18

if age >= 18:
    print("Adult, allowed to vote")
```

Notice three things: - The end of the `if` line ends with a colon `:`. - The next line is indented by four spaces (this is necessary — indentation is what makes Python know the line belongs to the condition). - The == sign for comparison (different from = which assigns a value).

Comparison operations in Python: == (equals), != (not equals), < (less than), > (greater than), <= (less than or equal), >= (greater than or equal).

else — What If the Condition Isn't Met?

```
age = 15

if age >= 18:
```

```
    print("Adult")
else:
    print("Minor")
```

`else` doesn't take a condition; it means "in all other cases".

elif — Multiple Conditions

When you have more than two cases, use `elif` (short for `else if`):

```
score = 75

if score >= 90:
    print("Excellent")
elif score >= 80:
    print("Very Good")
elif score >= 70:
    print("Good")
elif score >= 60:
    print("Pass")
```

```
else:
    print("Fail")
```

Python executes the first true condition and ignores the rest. If the first condition is true, Python won't check the following ones.

Combining Conditions: and / or / not

```
age = 25
has_passport = True

if age >= 18 and has_passport:
    print("You can travel")

if age < 18 or not has_passport:
    print("You cannot travel")
```

`and` requires both conditions true. `or` requires at least one. `not` flips the boolean.

Membership Check: in

A very useful operation — tells you if something exists in a string or list:

```
name = "Mohammed"
if "M" in name:
    print("Name contains the letter M")

cities = ["Sana'a", "Aden", "Taiz", "Hodeidah"]
city = input("Enter your city name: ")
if city in cities:
    print("We know your city")
else:
    print("Not in our list")
```

"Falsy" Values

In Python, the following values are considered `False` when checked as a condition: - `False` itself - `None` - Zero (`0` and `0.0`) - Empty string `""` - Empty list `[]` - Empty dict `{}`

Everything else is considered `True`. This greatly simplifies many checks:

```
name = input("Enter your name: ")


if name:
    print(f"Hello, {name}")
else:
    print("You didn't enter a name")
```

`if name:` means "if `name` is not empty".

for Loop — Iterating Over Items

The most common loop you'll use is `for`. It iterates over items of a "collection" (string, list, dictionary...).

```
names = ["Ahmed", "Mohammed", "Khalid"]
for name in names:
    print(f"Hello, {name}")
```

Python reads: "take each item from `names`, name it `name` temporarily, and execute the indented lines below it".

You can iterate over a string too (will iterate each character):

```
for letter in "Python":
    print(letter)
```

range — Repeating a Specific Number of Times

`range(n)` produces numbers from 0 to n-1.

```
for i in range(5):
    print(i)
# prints: 0, 1, 2, 3, 4
```

`range(start, stop)` starts from `start` and ends before `stop`:

```
for i in range(1, 11):
    print(i)
# prints: 1, 2, 3, ..., 10
```

`range(start, stop, step)` takes a specific step:

```
for i in range(0, 21, 5):
    print(i)
# prints: 0, 5, 10, 15, 20
```

while Loop — Repeat as Long as Condition Is True

```
balance = 1000
while balance > 0:
    spend = int(input("How much to spend? "))
    balance = balance - spend
    print(f"Remaining: {balance}")

print("Out of balance!")
```

`while` keeps repeating as long as the condition is true. **Beware:** if the condition doesn't change inside the loop, you get an infinite loop. Press `Ctrl+C` to stop a program stuck in an infinite loop.

break and continue

`break` exits the loop immediately:

```
for number in range(1, 100):
    if number == 7:
        print("Found seven!")
        break
    print(number)
```

`continue` skips the rest of the current iteration and goes to the next:

```
for number in range(1, 11):
    if number % 2 == 0:
        continue            # skip even numbers
    print(number)           # prints: 1, 3, 5, 7, 9
```

else After a Loop

A unique feature in Python: you can add `else` after a `for` or `while` loop. This `else` executes if the loop ended naturally (was not broken with `break`):

```
numbers = [2, 4, 6, 8]
for n in numbers:
    if n % 2 == 1:
        print("Found an odd number")
        break
else:
    print("All numbers are even")
```

List Comprehension

An elegant way to create a list from a loop in one line. For example, to create a list of squares of numbers from 1 to 10:

```
# long way
squares = []
```

```
for i in range(1, 11):
    squares.append(i * i)

# short way
squares = [i * i for i in range(1, 11)]
print(squares)  # [1, 4, 9, 16, 25, 36, 49, 64, 81,
```

You can add a condition:

```
# squares of even numbers only
even_squares = [i * i for i in range(1, 11) if i %
print(even_squares)  # [4, 16, 36, 64, 100]
```

You'll see this often in AI code. Get used to it early.

Nested Loops

You can put a loop inside a loop:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} × {j} = {i * j}")
```

This prints the multiplication table from 1 to 3. The outer loop runs 3 times, the inner runs 3 times each cycle, so the total is 9 prints.

Warning: nested loops slow down a program quickly. Two loops each of 1000 elements produces a million operations. We'll see in Chapter 11 how to measure program performance.

A Complete Example: Guessing Game

```
import random

secret = random.randint(1, 100)
attempts = 0
max_attempts = 7

print("I thought of a number between 1 and 100. You

while attempts < max_attempts:
    guess_text = input("Guess: ")
    guess = int(guess_text)
```

```
    attempts += 1

    if guess == secret:
        print(f"Well done! Guessed in attempt {atte
        break
    elif guess < secret:
        print("The number is larger")
    else:
        print("The number is smaller")
else:
    print(f"Attempts exhausted. The number was {sec
```

Study the program well. It contains `while`, `if/elif/else` condition, use of `break`, and even `else` on the loop. This summarizes the entire chapter in 15 lines.

Chapter Summary

You learned how to make decisions with `if/elif/else`, how to iterate with `for` and `while`, and how to control the loop with `break` and `continue`. These tools are enough to

write most program logic. In the next chapter, we'll learn functions — how to wrap code into reusable units.

Chapter 5 Exercises

1. Write a program printing numbers from 1 to 100, but replacing numbers divisible by 3 with "Fizz", numbers divisible by 5 with "Buzz", and numbers divisible by both with "FizzBuzz" (this is a famous classic exercise).
2. Write a program asking the user for 5 numbers and printing the largest and smallest.
3. Write a program asking the user for a password, allowing them only 3 attempts to enter it correctly (the correct password: "yemen2026").
4. Using list comprehension, create a list of cubes (power 3) of numbers from 1 to 20 that are divisible by 4.

Chapter 6: Functions

> *"The best speech is concise and meaningful. The best code is reused."*

Imagine writing a program that calculates VAT on ten products. Will you write the tax formula ten times? Of course not. You will put the formula in a "function" and call it ten times.

A function is a piece of code with a name, taking inputs and returning a result. This is the cornerstone of any serious program.

Defining a Function

A function begins with the keyword `def`, followed by the function name, parentheses, and a colon:

```
def greet():
    print("Peace be upon you")
    print("Welcome to our program")
```

To call the function:

```
greet()
greet()
greet()
```

This will print the greeting three times. Notice that writing `def greet():` doesn't run the function — it only defines it. Execution happens at `greet()`.

Functions Taking Parameters

When is a function actually useful? When it receives inputs that change each time:

```
def greet(name):
    print(f"Peace be upon you, {name}")

greet("Ahmed")         # Peace be upon you, Ahmed
```

```
greet("Mohammed")      # Peace be upon you, Mohammed
greet("Khalid")        # Peace be upon you, Khalid
```

The word `name` between the parentheses is called a "parameter". And the value "Ahmed" you passed at the call is called an "argument". This difference matters in advanced books, but in practical life people use both terms interchangeably.

Functions Returning a Value

The strongest function is one that **returns** a result, so you can use it in other operations:

```
def calculate_tax(price):
    tax_rate = 0.15
    tax = price * tax_rate
    return tax

product_price = 200
tax_amount = calculate_tax(product_price)
```

```
total = product_price + tax_amount
print(f"Price after tax: {total}")
```

The `return` keyword ends the function's execution and gives the value to whoever called it. Any line after `return` will not execute.

Multiple Parameters

```
def calculate_total(price, quantity, tax_rate):
    subtotal = price * quantity
    tax = subtotal * tax_rate
    return subtotal + tax

result = calculate_total(50, 3, 0.15)
print(result)         # 172.5
```

Argument order matters: price first, then quantity, then tax rate.

Parameters with Default Values

You can give a parameter a default value:

```
def calculate_total(price, quantity, tax_rate=0.15)
    subtotal = price * quantity
    tax = subtotal * tax_rate
    return subtotal + tax

# you can omit tax_rate (will use 0.15)
print(calculate_total(50, 3))         # 172.5

# or modify it
print(calculate_total(50, 3, 0.20))   # 180.0
```

Keyword Arguments

When calling, you can name arguments to increase clarity:

```
total = calculate_total(price=50, quantity=3, tax_r
```

This is much clearer than `calculate_total(50, 3, 0.10)`, especially when the function is large with many parameters. Get used to this in important functions.

Returning Multiple Values

A Python function can return multiple values at once, separated by commas:

```
def stats(numbers):
    return min(numbers), max(numbers), sum(numbers)

scores = [85, 90, 78, 92, 88]
lowest, highest, average = stats(scores)

print(f"Lowest: {lowest}")
print(f"Highest: {highest}")
print(f"Average: {average:.2f}")
```

What the function returns here is a "tuple" (an ordered pair/triple), and you'll learn about it in the next chapter. Unpacking onto multiple variables is called "tuple unpacking".

The docstring — Documenting the Function

Good practice requires explaining each function with a short text at its beginning. This is called a "docstring":

```
def calculate_tax(price, rate=0.15):
    """Calculate the tax amount on a given price.

    Parameters:
        price: The original price.
        rate: Tax rate (default 0.15).

    Returns:
        The tax amount.
    """
    return price * rate
```

Use triple-quote strings `"""`. Any Python editor will display this documentation when you use the function later.

Scope

Variables inside a function are "local" — the rest of the program doesn't see them. Variables outside functions are "global" — all functions see them.

```
total = 100          # global variable

def add(x):
    result = x + total      # can see total
    return result

print(add(5))               # 105
# print(result)             # this would error: res
```

Tip: don't rely on global variables inside functions. Pass everything you need via parameters. This makes your functions "pure" and easy to test.

Functions as Values — A Glimpse of Functional Programming

In Python, functions themselves are values — you can pass them to other functions and save them in variables:

```
def shout(text):
    return text.upper()

def whisper(text):
    return text.lower()

# save the function in a variable
my_function = shout
print(my_function("hello"))      # HELLO

# pass a function to another function
def process(text, transformer):
    return transformer(text)
```

```
print(process("Hello", shout))      # HELLO
print(process("Hello", whisper))    # hello
```

This pattern is fundamental when using AI libraries, where we pass "callbacks" to process each chunk of model response.

Anonymous Functions (Lambda)

Sometimes you need a very small function not worth a full `def`. Use `lambda`:

```
# regular way
def square(x):
    return x * x

# anonymous way
square = lambda x: x * x

print(square(5))      # 25
```

The very common use of lambda is in sorting functions:

```
people = [
    {"name": "Ahmed", "age": 30},
    {"name": "Mohammed", "age": 25},
    {"name": "Khalid", "age": 35},
]

# sort by age
people.sort(key=lambda p: p["age"])
print(people)
```

args and *kwargs

Sometimes you don't know in advance how many arguments. Use `*args` to receive a variable number of positional arguments, and `**kwargs` for named arguments:

```
def sum_all(*numbers):
    total = 0
    for n in numbers:
        total += n
    return total
```

```
print(sum_all(1, 2, 3))            # 6
print(sum_all(1, 2, 3, 4, 5))      # 15

def show_info(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

show_info(name="Ahmed", age=30, city="Sana'a")
```

You'll see them often in library documentation, especially in functions like `requests.get()` accepting dozens of optional arguments.

Importing Functions from Other Files

You don't have to write all functions in one file. Put your important functions in a separate file, then import:

File `math_helpers.py`:

```
def calculate_tax(price, rate=0.15):
    return price * rate

def calculate_total(price, quantity):
    return price * quantity
```

File `main.py`:

```
from math_helpers import calculate_tax, calculate_t

t = calculate_total(50, 3)
tx = calculate_tax(t)
print(t + tx)
```

Or import everything at once:

```
import math_helpers

t = math_helpers.calculate_total(50, 3)
tx = math_helpers.calculate_tax(t)
```

This is what allows building large projects from small reusable pieces.

A Complete Example: Text Helper Library

```
def word_count(text):
    """Count the number of words in a text."""
    return len(text.split())

def is_palindrome(text):
    """Check if the text is a palindrome."""
    cleaned = text.replace(" ", "").lower()
    return cleaned == cleaned[::-1]

def truncate(text, max_length=50):
    """Truncate the text and add ellipsis."""
    if len(text) <= max_length:
        return text
    return text[:max_length] + "..."

# usage
sentence = "Python is an easy and powerful programm
```

```
print(word_count(sentence))          # 7
print(is_palindrome("level"))        # True
print(truncate("very long text " * 10))
```

Chapter Summary

Functions are the cornerstone of reusable code. You learned how to define a function, how to pass parameters to it, how to return results, and how to organize them in separate files. Every serious Python line you'll write will be inside a function. In the next chapter, we'll dive into core data structures — lists, dictionaries, and sets.

Chapter 6 Exercises

1. Write a function `temperature_to_fahrenheit(c)` taking a Celsius temperature and returning the Fahrenheit equivalent (formula: F = C × 9/5 + 32).

2. Write a function `arabic_letter_count(text)` counting only Arabic letters in a text (ignoring numbers, spaces, punctuation).
3. Write a function `safe_divide(a, b)` dividing two numbers and returning `None` if `b` is zero.
4. Write a function `format_name(first, last, title="Mr.")` composing a full name with an optional title.

Chapter 7: Data Structures

> *"Organize your data, and your thinking organizes itself."*

Every serious program deals with collections of data, not single values. A list of students, a dictionary of their grades, a set of countries from different books. This chapter covers four basic structures in Python: list, dictionary, set, and tuple. All of them will appear in every line of AI code you write later.

The List

A list is an ordered collection of items, modifiable. Written between square brackets:

```
fruits = ["apple", "banana", "orange"]
numbers = [10, 20, 30, 40]
```

```
mixed = [1, "text", 3.14, True]       # types can be
empty = []
```

Accessing an Element

Elements are indexed from 0:

```
fruits = ["apple", "banana", "orange"]
print(fruits[0])       # apple
print(fruits[1])       # banana
print(fruits[2])       # orange
print(fruits[-1])      # orange (last element)
print(fruits[-2])      # banana
```

Slicing

```
nums = [10, 20, 30, 40, 50, 60]
print(nums[0:3])       # [10, 20, 30]   from 0 to be
print(nums[2:])        # [30, 40, 50, 60] from 2 to
print(nums[:4])        # [10, 20, 30, 40] from start
print(nums[::2])       # [10, 30, 50] step 2
print(nums[::-1])      # reversed
```

Modifying the List

```
fruits = ["apple", "banana", "orange"]

# change an element
fruits[1] = "mango"
print(fruits)          # ['apple', 'mango', 'orange

# add to the end
fruits.append("grape")
print(fruits)          # ['apple', 'mango', 'orange

# insert at a specific position
fruits.insert(0, "strawberry")
print(fruits)

# delete by value
fruits.remove("mango")

# delete by position
removed = fruits.pop(0)
```

```
# add a whole list
more = ["peach", "cherry"]
fruits.extend(more)
```

Useful Functions on Lists

```
nums = [3, 1, 4, 1, 5, 9, 2, 6]

print(len(nums))       # 8 (number of elements)
print(sum(nums))       # 31
print(min(nums))       # 1
print(max(nums))       # 9
print(sorted(nums))    # sorted copy

nums.sort()            # sort the original list
nums.reverse()         # reverse order

print(nums.count(1))   # 2 (occurrences of 1)
print(nums.index(5))   # position of 5
```

Membership Check

```
fruits = ["apple", "banana", "orange"]
if "apple" in fruits:
    print("yes we have apples")
```

The Dictionary

A dictionary is a collection of "key: value" pairs. Like a real Arabic dictionary: you look up a word (the key) and find its meaning (the value).

```
student = {
    "name": "Ahmed",
    "age": 22,
    "city": "Sana'a",
    "grade": 88.5,
}

# access
print(student["name"])          # Ahmed
print(student["age"])           # 22
```

```
# modification
student["grade"] = 92.0
student["email"] = "ahmed@example.com"     # add ne

# deletion
del student["city"]

# check key
if "name" in student:
    print("we have his name")

# safe access
phone = student.get("phone")               # None i
phone = student.get("phone", "unknown")    # defaul
```

Iterating Over a Dictionary

```
student = {"name": "Ahmed", "age": 22, "city": "Sar

# keys only
```

```
for key in student:
    print(key)

# keys and values
for key, value in student.items():
    print(f"{key}: {value}")

# values only
for value in student.values():
    print(value)
```

Nested Dictionaries

In AI you'll see them often — API responses are usually nested dictionaries:

```
response = {
    "model": "claude-sonnet-4-6",
    "usage": {
        "input_tokens": 150,
        "output_tokens": 320,
    },
```

```
    "content": [
        {"type": "text", "text": "Hello"},
    ],
}

print(response["usage"]["input_tokens"])     # 150
print(response["content"][0]["text"])        # Hello
```

Get used to this nesting. Every LLM API returns a similar structure.

The Set

A set is a collection of **unique** items, unordered. Written between curly braces:

```
fruits = {"apple", "banana", "orange", "apple"}
print(fruits)          # {'apple', 'banana', 'orange

# add and remove
fruits.add("grape")
fruits.remove("banana")
```

```
# check
if "apple" in fruits:
    print("yes")
```

Mathematical Set Operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)          # union: {1, 2, 3, 4, 5, 6}
print(a & b)          # intersection: {3, 4}
print(a - b)          # difference: {1, 2}
print(a ^ b)          # symmetric difference: {1, 2
```

A very common use of set: removing duplicates from a list:

```
words = ["yes", "no", "yes", "maybe", "yes", "no"]
unique_words = list(set(words))
print(unique_words)
```

The Tuple

Similar to a list, but **cannot be modified** after creation. Written with regular parentheses:

```
point = (3, 5)
colors = ("red", "green", "blue")
single = (42,)         # tuple with one element, not
empty = ()
```

You use a tuple when you want unchangeable data, like point coordinates, or RGB color components. Many Python functions also return tuples (like `divmod` returning quotient and remainder together).

```
result = divmod(17, 5)
print(result)          # (3, 2)

# tuple unpacking
quotient, remainder = divmod(17, 5)
print(quotient)        # 3
print(remainder)       # 2
```

Which to Use When?

Choosing the right structure is more important than learning to use it:

Use list when you need an ordered collection of items you can modify (add, delete, change order).

Use dict when you need to associate keys with values (name with age, word with meaning, product number with price).

Use set when you only care about whether an element exists, order doesn't matter, and you want to prevent duplication.

Use tuple when you want fixed data that should not be modified (coordinates, dimensions, dates).

Nesting and Combination

Reality is unordered. You'll often need a list of dictionaries, or a dictionary whose values are lists:

```
# list of dictionaries (each row of data)
students = [
    {"name": "Ahmed", "grade": 88},
    {"name": "Mohammed", "grade": 92},
    {"name": "Khalid", "grade": 75},
]

# iteration
for s in students:
    print(f"{s['name']}: {s['grade']}")

# sort by grade (descending)
sorted_students = sorted(students, key=lambda s: s
print(sorted_students)

# dictionary whose values are lists
courses = {
    "Python": ["Ahmed", "Mohammed"],
    "AI": ["Khalid", "Ahmed"],
    "Databases": ["Fatima"],
}
```

```
# print each course and its students
for course, names in courses.items():
    print(f"{course}: {', '.join(names)}")
```

Dict Comprehension

Just as we have list comprehension, we have dict comprehension:

```
# dictionary mapping number to its square
squares = {n: n * n for n in range(1, 6)}
print(squares)        # {1: 1, 2: 4, 3: 9, 4: 16, 5

# dictionary with string keys and length values
names = ["Ahmed", "Mohammed", "Khalid"]
lengths = {name: len(name) for name in names}
print(lengths)
```

A Complete Example: Simple Statistics Analyzer

```
scores = {
    "Ahmed": 88,
    "Mohammed": 92,
    "Khalid": 75,
    "Fatima": 95,
    "Maryam": 80,
}

# calculate statistics
all_scores = list(scores.values())
average = sum(all_scores) / len(all_scores)
top_student = max(scores, key=scores.get)
passing = {name: s for name, s in scores.items() i

print(f"Number of students: {len(scores)}")
print(f"Average: {average:.2f}")
```

```
print(f"Top: {top_student} with score {scores[top_s
print(f"Passing: {passing}")
```

Chapter Summary

Four core structures form the foundation of every Python application: list for order, dictionary for association, set for uniqueness, tuple for immutability. Mastering them will lift you from beginner to capable. In the next chapter, we'll go beyond program memory to files on disk, learning how to read and write — including Arabic UTF-8 files.

Chapter 7 Exercises

1. Create a list of 10 student grades. Write code that prints: average, highest grade, lowest grade, number passing (>= 60).
2. Create a dictionary of 5 students and their ages. Write code finding the oldest student.

3. Take two lists of students and write code printing: shared names, names in the first only, all names without duplication.
4. Convert the following dictionary to a list of tuples: `{"Ahmed": 88, "Mohammed": 92, "Khalid": 75}`.

Chapter 8: Reading and Writing Files

> *"Memory forgets; files preserve."*

Everything we've learned so far lives in program memory and disappears when the program closes. But real programs deal with files: reading a document, extracting data from it, writing new results. In AI especially, you'll load thousands of text documents, process them, and save the results. This chapter teaches you the basics.

Opening a File and Reading It

The simplest way to read a text file:

```
file = open("students.txt", "r", encoding="utf-8")
content = file.read()
```

```
print(content)
file.close()
```

Explanation of each part: - `open()`: opens the file. - `"students.txt"`: the file name. Can be a full path. - `"r"`: open mode (`r` for read). - `encoding="utf-8"`: file encoding. **Critical** for Arabic texts. Never forget it. - `file.read()`: reads all file content as one string. - `file.close()`: closes the file. Necessary to free resources.

The Recommended Way: with

Forgetting `close()` causes problems. The elegant way is using `with`, which closes the file automatically:

```
with open("students.txt", "r", encoding="utf-8") as
    content = file.read()

print(content)          # the file is already closed
```

Use `with` always. It is the officially recommended way.

File Open Modes

Mode	Meaning
`"r"`	Read only (default). Fails if file doesn't exist.
`"w"`	Write. **Creates a new file or erases the existing one**.
`"a"`	Append. Writes at the end of the file without erasing.
`"r+"`	Read and write.
`"rb"`	Binary read (for images and non-text files).

Warning: `"w"` is dangerous. It destroys the existing file immediately without asking. Always check the file name before using this mode.

Writing a File

```
with open("output.txt", "w", encoding="utf-8") as f
    file.write("First line\n")
    file.write("Second line\n")
    file.write("Third line\n")
```

`\n` is the "new line" symbol. Any text editor will display it as an actual line break.

To write several lines at once, use `writelines`:

```
lines = ["Ahmed\n", "Mohammed\n", "Khalid\n"]
with open("names.txt", "w", encoding="utf-8") as f
    file.writelines(lines)
```

Reading Line by Line

AI files can be huge (a million lines). Loading them all with `read()` consumes massive memory. Better to read line by line:

```
with open("big_file.txt", "r", encoding="utf-8") as
    for line in file:
        print(line.strip())     # strip removes \n
```

This code works even on a 10 GB file, because it loads one line in memory at a time.

Reading into a List

If you want all lines as a list:

```
with open("students.txt", "r", encoding="utf-8") as
    lines = file.readlines()

# lines is now a list, each element a line (with \n
for line in lines:
    print(line.strip())
```

Appending to an Existing File

```
with open("log.txt", "a", encoding="utf-8") as file
    file.write("New event: login\n")
```

This code adds a new line at the end of `log.txt` without erasing what came before. This is a beloved habit for log files.

Arabic Texts and UTF-8 Encoding

Arabic texts need special encoding to be written and read correctly. The standard encoding supporting Arabic and all world languages is **UTF-8**. Every `open` command must contain `encoding="utf-8"`.

What happens if you forget? The code may work on Linux and macOS (which use UTF-8 by default) but fails on Windows (which uses cp1252 by default), and you'll get a horrifying error called `UnicodeDecodeError`. Write `encoding="utf-8"` every time. Make it a fixed habit.

```
# comprehensive example for reading an Arabic file
with open("poem.txt", "r", encoding="utf-8") as fil
    poem = file.read()

print(f"Number of characters: {len(poem)}")
print(f"Number of lines: {poem.count(chr(10)) + 1}"
print(f"Number of words: {len(poem.split())}")
```

Checking File Existence

Before reading, you may want to confirm the file exists:

```
import os

if os.path.exists("data.txt"):
    with open("data.txt", "r", encoding="utf-8") as
        print(f.read())
else:
    print("File doesn't exist")
```

Or the modern way with `pathlib`:

```
from pathlib import Path

file = Path("data.txt")
if file.exists():
    print(file.read_text(encoding="utf-8"))
```

`pathlib` is cleaner and more powerful. You'll see it often in modern code:

```
from pathlib import Path

# read in one line
text = Path("poem.txt").read_text(encoding="utf-8")

# write in one line
Path("output.txt").write_text("New content", encodi

# iterate over every text file in a folder
for file in Path("documents").glob("*.txt"):
    print(file.name)
    print(file.read_text(encoding="utf-8")[:100])
```

Dealing with Paths

Paths differ between Windows and Linux: Windows uses `\`, others `/`. `pathlib` handles this automatically:

```
from pathlib import Path

# you write like this
```

```
path = Path("documents") / "students" / "ahmed.txt"

# pathlib converts to the right system
print(path)
# on Linux: documents/students/ahmed.txt
# on Windows: documents\students\ahmed.txt
```

Creating Folders

```
from pathlib import Path

# create a new folder
Path("output").mkdir(exist_ok=True)

# create a chain of folders
Path("output/2026/january").mkdir(parents=True, exi
```

`exist_ok=True` prevents the error if the folder exists.

`parents=True` creates all parent folders.

Copy, Delete, Move

```
import shutil
from pathlib import Path

# copy
shutil.copy("source.txt", "destination.txt")

# move
shutil.move("old_name.txt", "new_name.txt")

# delete
Path("file_to_delete.txt").unlink()
```

Warning: `unlink` deletes the file immediately, without asking, and without a trash bin. Verify well before running it.

A Complete Example: Cleaning an Arabic Text File

Imagine you have a file with thousands of lines, some empty, some with extra spaces. Write a program to clean it:

```
from pathlib import Path

input_file = Path("messy.txt")
output_file = Path("clean.txt")

# read
text = input_file.read_text(encoding="utf-8")

# clean
lines = text.split("\n")
clean_lines = []
for line in lines:
    line = line.strip()        # remove spaces from
    if line:                   # ignore empty lines
```

```
        clean_lines.append(line)

# write
output_text = "\n".join(clean_lines)
output_file.write_text(output_text, encoding="utf-8

print(f"Removed {len(lines) - len(clean_lines)} emp
print(f"Remaining: {len(clean_lines)} lines")
```

This kind of cleaning is the first step in every RAG pipeline. You need to clean files before chunking them and converting to vectors.

Bridge to the RAG Book

In the next book you'll learn to build "ingestion pipelines". Each pipeline begins with:

1. Reading files from a folder (this chapter).
2. Extracting text (whether it's txt, PDF, Word).
3. Cleaning it (this chapter).
4. Chunking into pieces (chunks).

5. Converting each chunk to a vector via API (Chapter 13).
6. Saving the vectors in a database.

Every line you learned here will be a building block in those pipelines. Get used to `pathlib` and `with open` now, and you'll find them close friends later.

Chapter Summary

You learned to read and write files in two ways: traditional (`open` with `with`) and modern (`pathlib`). You learned the importance of UTF-8 encoding for Arabic texts. You learned how to deal with paths in an OS-neutral way. All these are essential skills for any data work. In the next chapter, we'll dive deeper into JSON and CSV — the two most important formats you'll deal with in AI.

Chapter 8 Exercises

1. Write a program reading an Arabic text file and printing: number of lines, number of words, longest word in it.

2. Write a program taking a text file and creating a copy of it with all lines in reversed order.
3. Write a program searching all `.txt` files in a folder for a specific word, printing the names of files containing it.
4. Write a "logger" program: a function receiving a text and adding it to a `log.txt` file with a timestamp at the start.

Chapter 9: Working with JSON and CSV

> *"Data without form is chaos. Form without data is hollow structure."*

In the world of AI, 90% of the data you'll deal with will be in one of two formats: JSON or CSV. Every LLM API returns JSON. Every data file you download from Kaggle or your university administration is most likely CSV. Mastering these two formats is essential.

What Is JSON?

JSON (JavaScript Object Notation) is a format for writing structured data as text. It greatly resembles Python dictionaries:

```
{
    "name": "Ahmed",
    "age": 25,
    "is_student": true,
    "courses": ["Python", "AI"],
    "address": {
        "city": "Sana'a",
        "street": "Sabain"
    }
}
```

Similarities with a Python dictionary: key-value pairs, curly braces, square brackets for lists.

Important differences: - Keys in JSON must be strings between quotes. - `True/False/None` in Python are written `true/false/null` in JSON (lowercase). - No commas after the last element (Python allows it; JSON doesn't).

The json Library in Python

Python comes with the `json` library ready. You don't need to install it:

```
import json
```

The two most important functions: - `json.loads(text)`: converts JSON text to a Python dictionary. - `json.dumps(obj)`: converts a Python dictionary to JSON text.

```
import json

# JSON text to dictionary
text = '{"name": "Ahmed", "age": 25}'
data = json.loads(text)
print(data["name"])          # Ahmed
print(type(data))            # <class 'dict'>

# dictionary to JSON text
person = {"name": "Mohammed", "city": "Aden", "age"
```

```
text = json.dumps(person)
print(text)
```

Note! By default, `json.dumps` converts Arabic letters to Unicode escapes. To prevent this:

```
text = json.dumps(person, ensure_ascii=False)
print(text)
# {"name": "Mohammed", "city": "Aden", "age": 30}
```

`ensure_ascii=False` tells Python to keep Arabic letters as they are. Get used to adding it in every Arabic JSON.

Reading and Writing JSON Files

```
import json

# write a dictionary to a file
data = {
    "students": [
        {"name": "Ahmed", "grade": 88},
        {"name": "Mohammed", "grade": 92},
```

```
        {"name": "Khalid", "grade": 75},
    ],
    "course": "Python for AI",
    "year": 2026,
}

with open("students.json", "w", encoding="utf-8") a
    json.dump(data, f, ensure_ascii=False, indent=2

# read from file
with open("students.json", "r", encoding="utf-8") a
    loaded = json.load(f)

print(loaded["course"])
for s in loaded["students"]:
    print(f"{s['name']}: {s['grade']}")
```

Important differences: `json.dumps` and `json.loads` deal with **strings**. `json.dump` and `json.load` (without the s) deal with **open files**.

`indent=2` formats the output with nice spaces, making it readable. Without it, the entire JSON becomes one line.

Supported Data Types

JSON supports: strings, numbers, lists, dictionaries, true/false/null. **Doesn't support:** date and time directly, tuple (converts to list), set (gives an error), any custom class.

If you tried to save something unsupported:

```
import json


data = {"date": datetime.now()}
json.dumps(data)              # TypeError!
```

The solution: convert it to text before saving.

```
data = {"date": datetime.now().isoformat()}
json.dumps(data)              # succeeds
```

What Is CSV?

CSV (Comma-Separated Values) is a simple format for tabular data. Each line is a row, and values are comma-separated:

```
name,age,city
Ahmed,25,Sana'a
Mohammed,30,Aden
Khalid,22,Taiz
```

The first line usually contains column names (header). Any Excel file can be saved as CSV.

The csv Library in Python

```
import csv

# read CSV
with open("students.csv", "r", encoding="utf-8") as
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

```
# ['name', 'age', 'city']
# ['Ahmed', '25', "Sana'a"]
```

Each row is a list. The header is the first row.

Reading as Dictionaries (Better)

`DictReader` uses the header line as keys:

```
import csv

with open("students.csv", "r", encoding="utf-8") as
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["age"])
```

Much cleaner. You don't need to remember column order.

Writing CSV

```
import csv
```

```
students = [
    {"name": "Ahmed", "age": 25, "city": "Sana'a"},
    {"name": "Mohammed", "age": 30, "city": "Aden"
    {"name": "Khalid", "age": 22, "city": "Taiz"},
]

with open("output.csv", "w", encoding="utf-8", newl
    writer = csv.DictWriter(f, fieldnames=["name",
    writer.writeheader()
    writer.writerows(students)
```

Two important notes: - `newline=""`: necessary on Windows. Without it, blank lines appear between rows. - `fieldnames`: order of columns in the output.

Dealing with Arabic CSV and Excel

Excel on Windows opens CSV files with Windows-1256 encoding by default, not UTF-8. Arabic text will appear as strange symbols. The solution:

```
# write with BOM so Excel opens the file correctly
with open("output.csv", "w", encoding="utf-8-sig",
    writer = csv.DictWriter(f, fieldnames=["name",
    writer.writeheader()
    writer.writerows(students)
```

`utf-8-sig` adds a "BOM" (Byte Order Mark) at the start of the file, and Excel uses it to know the encoding is UTF-8.

Alternatively, advise the user to open the file via "Get Data → From Text" in Excel and choose UTF-8 manually.

Different Delimiters

Not all "CSV" files use commas. Some use semicolons `;` (common in Europe), some use Tab `\t` (called TSV):

```
with open("data.tsv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter="\t")
    for row in reader:
        print(row)
```

Using pandas (Stronger for Data)

`pandas` is a powerful library for data analysis. Install with `pip install pandas`. Then:

```
import pandas as pd

# read CSV
df = pd.read_csv("students.csv")
print(df.head())              # first 5 rows
print(df.shape)               # (rows, columns)
print(df["age"].mean())       # average age

# filtering
adults = df[df["age"] >= 25]

# saving
adults.to_csv("adults.csv", index=False, encoding='
```

`pandas` is excellent for big data. `pd.read_csv` reads a million-row file in seconds. If your project is serious, use pandas. For simple matters, the `csv` library is enough.

A Complete Example: Read JSON from API, Save as CSV

You'll see this pattern often in your work. APIs return JSON. You extract what matters, and save it as CSV to analyze in Excel.

```
import json
import csv

# imagine this came from an API
api_response = """
{
    "results": [
        {"id": 1, "name": "Ahmed", "score": 88, "c
        {"id": 2, "name": "Mohammed", "score": 92,
        {"id": 3, "name": "Khalid", "score": 75, "c
```

```
    ],
    "total": 3
}
"""

# parse JSON
data = json.loads(api_response)
students = data["results"]

# convert to CSV
with open("api_results.csv", "w", encoding="utf-8-s
    writer = csv.DictWriter(f, fieldnames=["id", "n
    writer.writeheader()
    writer.writerows(students)

print(f"Saved {len(students)} rows.")
```

Reverse Example: Read CSV, Convert to JSON

```
import csv
import json

with open("students.csv", "r", encoding="utf-8") as
    reader = csv.DictReader(f)
    students = list(reader)

# convert numbers (CSV reads everything as strings)
for s in students:
    s["age"] = int(s["age"])

# save as JSON
with open("students.json", "w", encoding="utf-8") a
    json.dump(students, f, ensure_ascii=False, inde
```

Important note: CSV reads everything as strings. You must convert numbers manually.

Bridge to the RAG Book

In a RAG system, JSON is the language of communication between all components. The OpenAI model's response comes as JSON. Pinecone's database response is JSON. The metadata you associate with each text chunk you store as JSON. Reading and writing JSON should be an automatic operation for you.

CSV you'll see more in "system evaluation" — you store test questions and their answers in CSV, read them line by line, send each question to the system, and save the correct and actual answers in a new results CSV.

Chapter Summary

You mastered the two most important data formats in AI. You learned how to read and write JSON correctly (with `ensure_ascii=False` for Arabic), and how to deal with CSV (with `utf-8-sig` for Excel). You learned when to use the `csv` library and when to upgrade to pandas. In the next

chapter, we'll open up to the internet and learn how to request data from APIs.

Chapter 9 Exercises

1. Write a program creating a JSON file containing 5 students (name, age, grade, city), then reading it and printing the average.
2. Write a program reading a CSV file with names and ages, separating those passing (>= 60) into one file and those failing into another.
3. Convert a JSON file containing a list of books (each book: title, author, year) into a CSV file.
4. Read an Arabic CSV file with pandas, sort it descending by grade column, and display the top 10 students.

Chapter 10: HTTP Requests and the requests Library

> *"An isolated program is an island. A connected program is a continent."*

Every AI application talks with remote servers. You send a question to an Anthropic model, you fetch data from Wikipedia, you query a vector database. This communication happens via the HTTP protocol, and the `requests` library in Python is your golden tool for handling it.

What Is HTTP?

HTTP is the language of conversation between computers on the internet. A client (like your program) sends a "request" to a "server". The server replies with a "response".

Each request has: - **URL**: like `https://api.example.com/users/123`. - **method**: GET, POST, PUT, DELETE. - **headers**: extra data like authentication. - **body**: data sent (in POST).

Each response has: - **status code**: 200 means succeeded, 404 means not found, 500 means server error. - **headers**. - **content**: usually JSON.

Installing requests

```
pip install requests
```

Then in code:

```
import requests
```

First GET Request

```
import requests


response = requests.get("https://api.github.com/use
```

```
print(response.status_code)         # 200
print(response.json())              # Python diction
```

Explanation: - `requests.get(url)`: sends a GET request (to fetch data). - `response.status_code`: status code. 200 means succeeded. - `response.json()`: converts the JSON response to a Python dictionary.

Working with Query Parameters

Sometimes you need to pass parameters in the URL. Instead of building it manually, use `params`:

```
import requests

# instead of: https://api.example.com/search?q=pyth
response = requests.get(
    "https://api.example.com/search",
    params={"q": "python", "limit": 10}
)
```

`requests` builds the URL for you and handles encoding for special characters.

Headers

Many APIs require headers for authentication:

```
import requests

headers = {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json",
    "User-Agent": "LLM4Yemen-Student/1.0",
}

response = requests.get(
    "https://api.example.com/data",
    headers=headers
)
```

You'll see this pattern in every AI model API.

POST Request

POST is used to send data (like creating a new record, or sending a question to an LLM):

```
import requests

data = {
    "name": "Ahmed",
    "email": "ahmed@example.com",
    "age": 25,
}

response = requests.post(
    "https://api.example.com/users",
    json=data            # json= sends data as JSON
)

print(response.status_code)
print(response.json())
```

`json=data` converts the dictionary to JSON and adds the appropriate Content-Type automatically.

Checking Response Status

```
import requests

response = requests.get("https://api.example.com/da

if response.status_code == 200:
    data = response.json()
    print(data)
elif response.status_code == 404:
    print("Resource not found")
elif response.status_code == 401:
    print("Unauthorized — check API key")
elif response.status_code == 429:
    print("Exceeded rate limit — wait")
else:
    print(f"Unexpected error: {response.status_code
```

Or in a cleaner way:

```
response = requests.get(url)
response.raise_for_status()        # throws an error
data = response.json()
```

Timeout — Protecting Your Program

If the server doesn't respond, your program will hang forever. Always set a timeout:

```
import requests


try:
    response = requests.get(
        "https://api.example.com/data",
        timeout=10            # 10 seconds
    )
except requests.Timeout:
    print("Server too slow. Try later.")
```

Get used to adding `timeout=` in every request. This is a habit of professional programmers.

Handling Network Errors

```
import requests


try:
    response = requests.get("https://api.example.co
    response.raise_for_status()
    data = response.json()
except requests.Timeout:
    print("Timeout expired")
except requests.ConnectionError:
    print("Cannot connect to server")
except requests.HTTPError as e:
    print(f"HTTP error code: {e}")
except ValueError:
    print("Response is not valid JSON")
```

Practical Example: Free Weather API

`open-meteo` provides a free API without a key:

```
import requests

# Sana'a weather (lat=15.35, lon=44.20)
response = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={
        "latitude": 15.35,
        "longitude": 44.20,
        "current_weather": "true",
    },
    timeout=10
)

if response.status_code == 200:
    data = response.json()
    weather = data["current_weather"]
```

```
    print(f"Temperature: {weather['temperature']}°C
    print(f"Wind speed: {weather['windspeed']} km/h
```

Run this code now. This is the first time in your life that you'll connect a Python program with a real server.

Example: Downloading a File

```
import requests

response = requests.get("https://example.com/docume
with open("document.pdf", "wb") as f:
    f.write(response.content)
```

`response.content` gives the data as bytes. So we opened the file with `"wb"` mode (write binary).

Downloading Large Files (Streaming)

A 1 GB file shouldn't be loaded entirely into memory. Use `stream=True`:

```
import requests

response = requests.get(
    "https://example.com/big_file.zip",
    stream=True,
    timeout=60
)

with open("big_file.zip", "wb") as f:
    for chunk in response.iter_content(chunk_size=8
        f.write(chunk)
```

This code reads the file in 8 KB chunks and writes them to disk immediately, using little memory.

Retries

Networks fail sometimes. Write a retry function:

```
import requests
import time
```

```
def fetch_with_retries(url, max_attempts=3):
    for attempt in range(1, max_attempts + 1):
        try:
            response = requests.get(url, timeout=10
            response.raise_for_status()
            return response.json()
        except (requests.Timeout, requests.Connecti
            print(f"Attempt {attempt} failed: {e}")
            if attempt < max_attempts:
                time.sleep(2 ** attempt)        # k
    raise Exception("All attempts failed")
```

`time.sleep(2 ** attempt)` waits 2 seconds, then 4, then 8. This is "exponential backoff" — a standard pattern when the server fails.

Rate Limits

Every API has a limit on requests per minute. In AI you'll see code 429 ("Too Many Requests") often. The solution:

```
import time

response = requests.get(url)
if response.status_code == 429:
    retry_after = int(response.headers.get("Retry-A
    print(f"Waiting {retry_after} seconds...")
    time.sleep(retry_after)
    response = requests.get(url)         # try agai
```

A Complete Example: Fetch and Save Data

```
import requests
import json
from pathlib import Path

def fetch_user_data(username):
    """Fetch GitHub user data and save it."""
    url = f"https://api.github.com/users/{username}
```

```
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
    except requests.RequestException as e:
        print(f"Failed: {e}")
        return None

    # save to file
    output = Path(f"github_{username}.json")
    output.write_text(
        json.dumps(data, ensure_ascii=False, indent
        encoding="utf-8"
    )
    return data

# usage
user = fetch_user_data("torvalds")
if user:
    print(f"Name: {user['name']}")
```

```
print(f"Followers: {user['followers']}")
print(f"Repositories: {user['public_repos']}")
```

Bridge to the RAG Book

In the RAG book, you'll use `requests` to:

- Call the embedding API to convert text to vectors.
- Call a vector database (Pinecone, Qdrant Cloud) via REST API.
- Call the LLM to get the final answer.

Every line of those calls will use the same patterns you learned here: headers for authentication, `json=` for sending, `response.json()` for reading, `try/except` for handling errors.

Chapter Summary

You mastered `requests` — the most important library in practical AI work. You learned GET and POST, authentication headers, timeouts, error handling, retrying, rate limits. In the

next chapter, we'll dive into error handling and debugging techniques — how to discover and fix errors quickly.

Chapter 10 Exercises

1. Use the open-meteo weather API to fetch weather for Aden, Taiz, and Hodeidah, printing a table containing: city, temperature, wind speed.
2. Call the GitHub API to fetch the latest 5 repositories of user `tensorflow`, printing: name, star count, language.
3. Write a function `safe_get(url)` that tries the request 3 times with increasing wait time, returning JSON or None.
4. Design a program fetching Wikipedia text for a specific article via the API: `https://en.wikipedia.org/api/rest_v1/page/summary/<title>`, saving it in a text file.

Chapter 11: Error Handling and Debugging Strategies

> *"An error is not a defect, but a teacher."*

Every programmer in the world — even the best engineer at Anthropic — spends half their time fixing errors, not writing new code. This is a professional fact. The difference between beginner and expert is not in the number of errors but in the speed of discovering and resolving them. This chapter teaches you the most important skill in your programming life.

Three Types of Errors

Syntax Errors: you write code violating Python's rules. Python can't even run it.

```
print("Hello"          # forgot the closing parenth
# SyntaxError: '(' was never closed
```

Runtime Errors: the code is syntactically correct but fails during execution.

```
x = 10 / 0          # ZeroDivisionError
nums = [1, 2, 3]
print(nums[10])     # IndexError
```

Logic Errors: the code runs without errors but produces a wrong result. These are the hardest errors.

```
def average(nums):
    return sum(nums) / len(nums) + 1     # +1 is a
```

try / except — Avoiding Program Crash

`try/except` lets you "catch" errors and handle them instead of stopping the program:

```
try:
    x = int(input("Enter a number: "))
```

```
    print(f"Square: {x * x}")
except ValueError:
    print("What you entered is not a number!")
```

If the user enters text, control jumps to `except`, and the program avoids crashing.

Catching Multiple Errors

```
try:
    x = int(input("First number: "))
    y = int(input("Second number: "))
    result = x / y
    print(f"Result: {result}")
except ValueError:
    print("Input is not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Catching All Errors

```
try:
    risky_operation()
except Exception as e:
    print(f"An error occurred: {e}")
```

`Exception` catches almost any error. `as e` gives you the error object to read its details.

Warning: avoid catching `Exception` generally. Better to catch only the errors you anticipate, leaving unexpected errors to surface, as they reveal real flaws in your code.

else and finally

```
try:
    file = open("data.txt", "r", encoding="utf-8")
    content = file.read()
except FileNotFoundError:
    print("File doesn't exist")
else:
```

```
    # only executes if no error occurred
    print("Read succeeded")
finally:
    # executes in all cases
    print("Attempt finished")
```

`finally` is useful for "cleanup" — closing files, closing database connections. But in modern Python, `with` handles that automatically.

Raising Errors Yourself

Sometimes you want to raise an explicit error:

```
def calculate_age(birth_year):
    if birth_year < 1900 or birth_year > 2026:
        raise ValueError(f"Unreasonable birth year
    return 2026 - birth_year

try:
    age = calculate_age(1850)
```

```
except ValueError as e:
    print(f"Error: {e}")
```

`raise` throws an error, and any `try/except` above it catches it.

Custom Error Types

You can create error types specific to your project:

```
class APIError(Exception):
    """Error from an API."""
    pass

class RateLimitError(APIError):
    """Allowed limit exceeded."""
    pass

def call_api():
    response = requests.get(url)
    if response.status_code == 429:
        raise RateLimitError("Limit exceeded")
```

```
    if response.status_code != 200:
        raise APIError(f"Error {response.status_coo

try:
    call_api()
except RateLimitError:
    print("Wait a moment")
except APIError as e:
    print(f"API failed: {e}")
```

You'll see this pattern in every serious AI library.

Debugging Strategies — The Five Tools

1. Print Debugging

The simplest and fastest method. Add `print` to know what's happening:

```
def calculate_total(items):
    print(f"DEBUG: number of items = {len(items)}")
    total = 0
```

```
    for item in items:
        print(f"DEBUG: adding {item}")
        total += item
    print(f"DEBUG: total = {total}")
    return total
```

Remove the `prints` after fixing the error. Or use the `logging` library to leave them in and turn them off easily.

2. Read Traceback Carefully

```
Traceback (most recent call last):
  File "main.py", line 15, in <module>
    result = process(data)
  File "main.py", line 8, in process
    return divide(x, y)
  File "main.py", line 3, in divide
    return a / b
ZeroDivisionError: division by zero
```

Read from **bottom**. The last line tells you the error. The lines above tell you the call path — where it started and where it finally failed. This is the "stack trace".

3. Isolate the Error

When an error appears, ask: what is the smallest code that reproduces the error? Delete everything irrelevant. The less code you look at, the faster you find the error.

4. Test One Hypothesis at a Time

Don't change five things together. Change one thing, run, observe the result. If it succeeds, you know the cause. If it fails, you know the cause is elsewhere.

5. Rubber Duck Debugging

Explain your code aloud, as if explaining to a colleague (or a plastic duck). Many errors become clear only when you try to formulate them in words. This is a known technique in the software industry.

The logging Library — Professional print

`print` is for experimenting. For serious projects use `logging`:

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message
)

logging.info("Program started")
logging.warning("Value close to limit")
logging.error("Operation failed")
```

Features of `logging` over `print`: - Different levels (DEBUG, INFO, WARNING, ERROR). - Can save to a file. - Can be turned off without deleting the code. - Includes timestamp automatically.

Detailed Use of logging

```
import logging

logging.basicConfig(
    filename="app.log",
    filemode="w",
    level=logging.DEBUG,
    format="%(asctime)s - %(name)s - %(levelname)s
    encoding="utf-8"
)

def process_user(user_id):
    logging.info(f"Starting to process user {user_i
    try:
        # do something
        logging.debug(f"User {user_id} has 5 record
        return True
    except Exception as e:
        logging.error(f"Failed to process {user_id}
        return False
```

Assertion — Explicit Assumptions

`assert` confirms something is true. If not, it raises an error:

```
def divide(a, b):
    assert b != 0, "Denominator cannot be zero"
    return a / b

divide(10, 2)        # 5.0
divide(10, 0)        # AssertionError: Denominator
```

Use it to verify internal assumptions in your code. Don't use it to verify user input — use `if` and `raise ValueError`.

A Complete Example: Safe API Call

```
import requests
import logging

logging.basicConfig(level=logging.INFO)

def safe_api_call(url, headers=None, max_retries=3)
```

```
    """Calls an API with full error handling."""
    for attempt in range(1, max_retries + 1):
        try:
            response = requests.get(url, headers=he
            response.raise_for_status()
            return response.json()
        except requests.Timeout:
            logging.warning(f"Attempt {attempt}: ti
        except requests.ConnectionError:
            logging.warning(f"Attempt {attempt}: no
        except requests.HTTPError as e:
            if response.status_code == 429:
                logging.warning(f"Attempt {attempt
            else:
                logging.error(f"HTTP {response.stat
                return None
        except ValueError:
            logging.error("Response is not valid JS
            return None

    logging.error(f"All attempts failed for {url}")
```

```
    return None

# usage
data = safe_api_call("https://api.github.com/users/
if data:
    print(data["name"])
else:
    print("Could not fetch data")
```

Bridge to the RAG Book

In a RAG system, dozens of things may fail: API slows, vector database fails, model exceeds context window. Every step in the pipeline must be "fault-tolerant". The patterns of try/except and retries you learned here will appear everywhere.

`logging` is essential for diagnosing problems in production. When a user tells you the system gave them a wrong answer an hour ago, you need a detailed log to know what happened.

Chapter Summary

You learned to handle errors professionally — with try/except, raise, and custom error types. You learned 5 fundamental debugging strategies. You learned when to use `print` and when to upgrade to `logging`. These skills separate the program "that works on your computer" from the program "that works for thousands of users without interruption". In the next chapter, we'll learn how to safely hide API keys.

Chapter 11 Exercises

1. Write a function `safe_int(text)` converting text to a number. If conversion fails, return 0 and print a warning.
2. Write a function `read_json_safely(path)` reading a JSON file and returning its dictionary, or returning `{}` if file not found or JSON invalid.

3. Write a custom error class `InsufficientBalanceError`, and use it in a function `withdraw(account, amount)`.
4. Rewrite the guessing game from Chapter 5, but with full error handling (if user enters text instead of a number, ask again without crashing).

Chapter 12: Environment Variables and Safely Storing API Keys

> *"A secret known by two is no longer a secret."*

Every AI model API requires a "key" (API key). This key is your digital identity, and it's what charges you for usage. If the key falls into the wrong hands, someone may use it in massive quantities, and you'll find a bill for tens or hundreds of dollars.

The most famous beginner's mistake: writing the key directly in the code, then pushing the code to GitHub. Then within hours, scraping bots discover the key, and the account is stolen. This chapter teaches you how to avoid this disaster.

What Are Environment Variables?

Environment variables are values stored at the OS level, that any program can read. For example, `PATH` is a famous environment variable containing folder paths the system searches for programs.

We'll use them to store secret keys, so they don't appear in code.

Reading an Environment Variable in Python

```
import os

api_key = os.environ.get("ANTHROPIC_API_KEY")
print(api_key)         # prints the key if exists, o
```

`os.environ.get(name)` returns the variable's value or `None` if not present. Or you can specify a default value:

```
api_key = os.environ.get("ANTHROPIC_API_KEY", "defa
```

Or fail clearly if not present:

```
api_key = os.environ["ANTHROPIC_API_KEY"]    # KeyE
```

Setting an Environment Variable on Windows

In PowerShell:

```
$env:ANTHROPIC_API_KEY = "sk-ant-..."
```

Then run Python **in the same window**. The variable disappears when the window closes. For permanent storage: 1. Press Windows + R, type `sysdm.cpl`, press Enter. 2. "Advanced" tab → "Environment Variables". 3. Add a new variable.

Setting an Environment Variable on macOS / Linux

In Terminal:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

For permanent storage, add this line to `~/.bashrc` or `~/.zshrc`.

The Better Way: .env File

Managing environment variables manually is tiring. The accepted solution in the software industry is the `.env` file:

Create a file called `.env` in your project's root:

```
ANTHROPIC_API_KEY=sk-ant-api03-...
OPENAI_API_KEY=sk-proj-...
GROQ_API_KEY=gsk_...
DATABASE_URL=postgresql://localhost/mydb
```

(Note: no spaces around =, and no need for quotes).

Then install the `python-dotenv` library:

```
pip install python-dotenv
```

Then in your code:

```
from dotenv import load_dotenv
import os

load_dotenv()                                    # lo
api_key = os.environ.get("ANTHROPIC_API_KEY")
print("I have a key:", api_key is not None)
```

`load_dotenv()` reads the `.env` file and puts its values in `os.environ`. The rest of the code is the same as before.

Protecting the Key from Git — The Most Important Step in Your Life

If you use Git (and you will), you must ensure `.env` is **never** uploaded to the repository.

Create a file called `.gitignore` in your project root, and write in it:

```
# keys and secrets
.env
.env.local
```

```
*.pem
secrets/

# Python
__pycache__/
*.pyc
venv/

# OS
.DS_Store
Thumbs.db

# editor
.vscode/
.idea/
```

`.gitignore` tells Git to ignore these files. They won't appear in `git status`, and won't be uploaded.

Write `.env` in `.gitignore` **before** you create the `.env` file. If you uploaded `.env` even once, it stays in Git history forever, even if you deleted it later.

Alternative for Sharing Settings: .env.example

How will your colleague who clones the project with `git clone` know they need to define `ANTHROPIC_API_KEY`? Create a file called `.env.example`:

```
# copy this file to .env and fill values
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
GROQ_API_KEY=
```

This file gets uploaded to Git (because the values are empty). Anyone who clones the project knows from it which keys they need.

Verifying the Key Is Loaded

Adding a simple check saves you hours of confusion:

```
from dotenv import load_dotenv
import os

load_dotenv()

required_keys = ["ANTHROPIC_API_KEY", "OPENAI_API_K
missing = [k for k in required_keys if not os.envir

if missing:
    print(f"Missing keys in .env: {missing}")
    exit(1)

print("All keys present. Can start.")
```

Never Print the Key

```
# very bad!
print(f"My key: {api_key}")

# also bad — the key may end up in logs
logging.info(f"Used {api_key}")

# good
print(f"My key: {api_key[:8]}...{api_key[-4:]}")
print(f"My key: ***")
```

Showing a small part of the key (so you know it's loaded) is acceptable security. Showing the full key is dangerous.

Advanced Security: Secret Managers

In real production environments, even `.env` is not enough. Companies use:

- **HashiCorp Vault**: a system dedicated to secret management.

- **AWS Secrets Manager**: from Amazon.
- **Google Secret Manager**: from Google.
- **Azure Key Vault**: from Microsoft.

These systems store keys encrypted, give them to your program only at runtime, and log who requested them and when. For now you don't need them — `.env` with `.gitignore` is completely sufficient for learning and personal projects.

Dealing with Keys in Google Colab

Colab doesn't read a `.env` file directly. The best way:

```
from google.colab import userdata


api_key = userdata.get("ANTHROPIC_API_KEY")
```

Go to Colab, the key icon in the side bar, and add a "secret" with the same name. It will be saved in your Google account, and won't appear in the notebook.

Or if you're in a quick experiment only:

```
import getpass

api_key = getpass.getpass("Paste your key: ")
```

`getpass` hides what you type (like a password). Safe from a curious eye on your screen.

A Complete Example: Calling Claude with .env

`.env` file:

```
ANTHROPIC_API_KEY=sk-ant-api03-...
```

`app.py` file:

```
from dotenv import load_dotenv
import os
import requests

load_dotenv()
```

```
api_key = os.environ.get("ANTHROPIC_API_KEY")
if not api_key:
    raise SystemExit("Missing ANTHROPIC_API_KEY in

headers = {
    "x-api-key": api_key,
    "anthropic-version": "2023-06-01",
    "content-type": "application/json",
}

payload = {
    "model": "claude-haiku-4-5-20251001",
    "max_tokens": 100,
    "messages": [
        {"role": "user", "content": "Hello, answer
    ],
}

response = requests.post(
    "https://api.anthropic.com/v1/messages",
    headers=headers,
```

```
    json=payload,
    timeout=30,
)

if response.status_code == 200:
    data = response.json()
    print(data["content"][0]["text"])
else:
    print(f"Error {response.status_code}: {response
```

Run this, and you'll have made your first secure conversation with an LLM. We'll dive deeper into call details in the next chapter.

Pre-commit Checklist

Before you write `git commit`:

1. Is `.env` in `.gitignore`?
2. Does `git status` show `.env`? If yes, something is wrong.

3. Is there any constant key starting with `sk-` or `gsk_` or other in the code?
4. Is there any leftover `print(api_key)`?

Get used to these four questions. They'll save you from great regret.

What If a Key Leaks by Accident?

If you forgot and put a key in code and uploaded it:

1. **Immediately** go to the API control panel (Anthropic Console, OpenAI Platform, etc.).
2. Revoke the leaked key immediately.
3. Create a new key.
4. Put the new one in `.env`.
5. Even if you deleted the line from code, it may remain in Git history. For full cleanup, use `git filter-repo` or even create a new repository.

Speed matters. Scraping bots discover keys in minutes.

Chapter Summary

You learned API key security — a skill that will save you from real losses. You learned environment variables, the `.env` file, and preventing them from being uploaded with `.gitignore`. You learned how to handle keys in Colab. These habits follow production code worldwide. In the next chapter, we'll apply everything we've learned and call real LLMs.

Chapter 12 Exercises

1. Create a new project with proper `.env`, `.env.example`, and `.gitignore`. Confirm with `git status` that `.env` doesn't appear.
2. Write a function `get_required_env(name)` reading an environment variable and raising a clear error if not present.
3. Write a program printing all keys in a `.env` file but showing only the first 4 characters of their value.

4. Search online for the `git-secrets` tool. Write a short paragraph (5 lines) about what it does.

Chapter 13: Calling Large Language Models from Python

> *"Give me a model, I'll give you a world."*

This is the chapter every LLM4Yemen student awaits. Everything we've learned in the previous 12 chapters was preparation for this moment: calling a large language model, conducting a conversation with it, and getting an Arabic response. We'll cover three providers: Anthropic (Claude), OpenAI (GPT), and Groq (free and very fast).

A Note Before Starting: Keys and Access Limits

Yemeni students may face two obstacles using Anthropic and OpenAI:

1. **Credit card**: both require a valid credit card. If you don't have one, don't worry — Groq and OpenRouter offer a free tier sufficient for all this book's exercises.
2. **Sanctions**: some services may not be available in Yemen. Try using a trusted VPN if you face an issue, or rely on the free alternative.

My rule: **know how to call Anthropic and OpenAI theoretically** (to understand all the code you'll see online), and **actually apply on Groq or OpenRouter** (to run exercises without cost).

Calling Claude (Anthropic)

Installing the Official Library

```
pip install anthropic
```

The Simplest Call

```
from dotenv import load_dotenv
import os
from anthropic import Anthropic

load_dotenv()

client = Anthropic(api_key=os.environ.get("ANTHROPI

message = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=200,
    messages=[
        {"role": "user", "content": "Write a short
```

```
    ]
)

print(message.content[0].text)
```

Explanation: - `Anthropic(api_key=...)`: creates the "client" — an object responsible for sending requests. - `model`: model name. `claude-haiku-4-5-20251001` is the cheapest and fastest. - `max_tokens`: maximum number of "tokens" in the response. A token ≈ 4 characters approximately. - `messages`: list of messages. Each message has a role (`user` or `assistant`) and content.

The returned `message.content[0].text` contains the model's text response.

Multi-turn Messages

```
message = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=300,
    messages=[
```

```
        {"role": "user", "content": "What is the ca
        {"role": "assistant", "content": "Sana'a is
        {"role": "user", "content": "And when was i
    ]
)

print(message.content[0].text)
```

The model sees the entire conversation and knows the second question concerns "Sana'a" mentioned just before.

System Prompt

Defines the model's character and style:

```
message = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=200,
    system="You are a Yemeni academic assistant, an
    messages=[
        {"role": "user", "content": "What's the dif
    ]
```

```
)

print(message.content[0].text)
```

Calling via REST Directly (without the anthropic library)

Sometimes you want full control, or work in an environment unable to install `anthropic`. The solution:

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()

response = requests.post(
    "https://api.anthropic.com/v1/messages",
    headers={
        "x-api-key": os.environ["ANTHROPIC_API_KEY"
        "anthropic-version": "2023-06-01",
```

```
        "content-type": "application/json",
    },
    json={
        "model": "claude-haiku-4-5-20251001",
        "max_tokens": 200,
        "messages": [
            {"role": "user", "content": "Give me a
        ],
    },
    timeout=30,
)

if response.status_code == 200:
    data = response.json()
    print(data["content"][0]["text"])
else:
    print(f"Error: {response.status_code}")
    print(response.text)
```

Everything you learned about `requests` applies here.

Calling GPT (OpenAI)

Installation

```
pip install openai
```

The Simplest Call

```
from dotenv import load_dotenv
import os
from openai import OpenAI

load_dotenv()

client = OpenAI(api_key=os.environ.get("OPENAI_API_

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a sm
        {"role": "user", "content": "Explain what R
```

```
    ],
    max_tokens=200,
)

print(response.choices[0].message.content)
```

Notable differences from Anthropic: - In OpenAI, the system message comes within `messages` with role `"system"`. - The result is in `response.choices[0].message.content`. - `gpt-4o-mini` is OpenAI's cheap model.

Via REST

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()

response = requests.post(
```

```
    "https://api.openai.com/v1/chat/completions",
    headers={
        "Authorization": f"Bearer {os.environ['OPEN
        "Content-Type": "application/json",
    },
    json={
        "model": "gpt-4o-mini",
        "messages": [
            {"role": "user", "content": "Write 3 ti
        ],
        "max_tokens": 300,
    },
    timeout=30,
)

print(response.json()["choices"][0]["message"]["con
```

First Free Alternative: Groq

Groq offers Llama and Mixtral models for free, at amazing speed. Doesn't require a card. Go to `console.groq.com`, sign up with a Google account, get a key immediately.

Installation

```
pip install groq
```

The Call

```
from dotenv import load_dotenv
import os
from groq import Groq

load_dotenv()

client = Groq(api_key=os.environ.get("GROQ_API_KEY"

response = client.chat.completions.create(
```

```
    model="llama-3.3-70b-versatile",
    messages=[
        {"role": "system", "content": "You are a he
        {"role": "user", "content": "What is RAG? E
    ],
    max_tokens=300,
)

print(response.choices[0].message.content)
```

The Groq API is almost identical to OpenAI's. This isn't accidental — Groq deliberately maintains compatibility to ease transition.

Second Free Alternative: OpenRouter

OpenRouter is a unified gateway to dozens of models, with a free tier for names ending in `:free`. Like `meta-llama/llama-3.3-8b-instruct:free`.

```
import requests
import os
```

```
from dotenv import load_dotenv

load_dotenv()

response = requests.post(
    "https://openrouter.ai/api/v1/chat/completions"
    headers={
        "Authorization": f"Bearer {os.environ['OPEN
        "Content-Type": "application/json",
    },
    json={
        "model": "meta-llama/llama-3.3-8b-instruct:
        "messages": [
            {"role": "user", "content": "Define AI
        ],
    },
    timeout=30,
)

print(response.json()["choices"][0]["message"]["con
```

OpenRouter's strength is that by changing `model` you switch between Claude, GPT, Llama, Mistral, and dozens of models, with one key.

Streaming — Receiving the Response Word by Word

In interactive interfaces, you want the response to appear gradually to the user, not in one batch. This is called "streaming":

```
from anthropic import Anthropic
from dotenv import load_dotenv
import os

load_dotenv()
client = Anthropic()

with client.messages.stream(
    model="claude-haiku-4-5-20251001",
    max_tokens=300,
```

```
    messages=[
        {"role": "user", "content": "Write a short
    ],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)

print()
```

Each chunk of the response arrives as soon as it's generated. In web applications, developers convert this to Server-Sent Events to display in the browser.

Controlling Creativity — Temperature

```
response = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=100,
    temperature=0.0,         # precise, repetitive
    messages=[{"role": "user", "content": "What is
)
```

`temperature` is a number from 0 to 1 (or 2 in some models). 0 produces deterministic and conservative answers. 1 produces creative and unexpected answers. For factual questions, use 0. For creative writing, use 0.7-1.0.

Properly Handling Arabic

Modern models handle Arabic well, but some tips:

1. **Request the language explicitly**: "Answer me in Modern Standard Arabic" is clearer than leaving it to the model.
2. **Use system prompt**: specify language and writing style.
3. **Ensure UTF-8 encoding** when writing to file. Note that `print` on some old Windows versions may corrupt Arabic. Use `sys.stdout.reconfigure(encoding="utf-8")` if you face an issue.

```
import sys
sys.stdout.reconfigure(encoding="utf-8")     # on ol
```

A Practical Comparison: Same Question on Three Models

```
import os
from dotenv import load_dotenv
from anthropic import Anthropic
from openai import OpenAI
from groq import Groq

load_dotenv()

prompt = "Explain in Arabic what AI is in two parag

# Claude
claude = Anthropic()
c = claude.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=300,
    messages=[{"role": "user", "content": prompt}]
)
```

```
# GPT
openai = OpenAI()
g = openai.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=300,
)

# Llama on Groq
groq = Groq()
l = groq.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=300,
)

print("=== Claude ===")
print(c.content[0].text)
print("\n=== GPT ===")
print(g.choices[0].message.content)
```

```
print("\n=== Llama (Groq) ===")
print(l.choices[0].message.content)
```

Run this yourself. You'll notice differences in style, accuracy, structure. Each model has its personality. Choosing depends on the task.

Cost Calculation (Approximately)

Each model is priced per "1 million tokens". Examples (approximate in 2026):

Model	Input (per million)	Output (per million)
Claude Haiku 4.5	$0.80	$4.00
GPT-4o-mini	$0.15	$0.60
Llama 3.3 (Groq)	Free	Free

A medium question (200 input tokens, 300 output tokens) costs on Claude Haiku approximately $0.0014. So roughly 0.14 cents. A thousand questions = $1.40.

Repeat the calculation for your model and you'll find that educational use is very cheap. Cost shows up in production systems with thousands of users.

Common Error Messages

Error message	Meaning	Solution
401 Unauthorized	Wrong or expired key	Check `.env`. Create a new key.
429 Too Many Requests	Exceeded rate limit	Wait seconds, retry.
400 invalid_request_error	Something wrong in parameters	Read error message carefully. Often wrong model name.
500 server_error	Server problem	Try later.
insufficient credits	Balance ran out	Recharge, or use Groq.

Bridge to the RAG Book

In the RAG book, you'll use these APIs in two ways:

1. **Embeddings**: converting texts to vectors. You call `client.embeddings.create()` instead of `messages.create()`.
2. **Generation**: same as you learned here — you send the user's question + retrieved texts, you get an answer.

Every line of this chapter you'll see in every line of that book. Master it now.

Chapter Summary

You mastered calling the most important three providers of language models. You learned how to handle Arabic, how to control creativity, how to analyze costs, how to handle errors. You are now ready to build your first real AI application. This is what we'll do in the final chapter.

Chapter 13 Exercises

1. Choose a provider (Groq if you have no card), and ask the model 5 questions about Yemen. Save the answers in a file.
2. Write a function `chat(question)` that calls the model and returns the answer. Call it 3 consecutive times for the same question with `temperature=1.0`. Notice the variety in answers.
3. Write a chat program that saves all messages, so the model remembers what you said before.
4. Compare Arabic quality between Claude and Llama 3.3 for the task "Translate this English sentence to Modern Standard Arabic".

Chapter 14: Capstone Project — Arabic CSV Summarizer with AI

> *"Knowledge without action is like a tree without fruit."*

Everything you've learned in 13 chapters will come together in this project. You'll build a real script that reads an Arabic CSV file, calls a language model to summarize each row, and writes the results to a new CSV file. This is exactly what an AI engineer does on their first day at work. Don't be afraid, you'll do it step by step.

Project Description

Input: an `articles.csv` file containing 10 short articles in Arabic. Each article has: title, date, text.

Output: a `summaries.csv` file containing the same data plus a "summary" column — a simple 2-3 sentence summary of each article.

Tools: Python, `csv` library, `dotenv` library, an LLM provider (Groq if you don't have a card, Claude or OpenAI if you do).

Project Structure

```
project/
├── .env
├── .gitignore
├── articles.csv              ← input
├── summaries.csv             ← output (we'll creat
├── summarize.py              ← the script
├── requirements.txt
└── log.txt                   ← run log
```

Step 1: Set Up the Environment

Create a project folder:

```
mkdir csv-summarizer
cd csv-summarizer
python -m venv venv
source venv/bin/activate          # on Linux/macOS
# venv\Scripts\activate           # on Windows
```

Create `requirements.txt`:

```
python-dotenv
groq
```

(or `anthropic` if you'll use Claude, or `openai` for GPT.)

Install:

```
pip install -r requirements.txt
```

Step 2: Set Up the .env File

```
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Create `.gitignore`:

```
.env
venv/
__pycache__/
*.pyc
log.txt
```

Step 3: Create the Input File

Create `articles.csv` with this content (note the utf-8-sig encoding for Excel compatibility):

```
import csv

articles = [
    {
        "id": 1,
        "title": "Launching the LLM4Yemen Program",
        "date": "2026-04-15",
        "text": "Today the LLM4Yemen program was la
    },
    {
```

"id": 2,
"title": "Launching the New Claude Model",
"date": "2026-03-20",
"text": "Anthropic announced the launch of
},
{
"id": 3,
"title": "Cybersecurity Symposium",
"date": "2026-04-10",
"text": "The College of Computing organized
},
{
"id": 4,
"title": "Student Innovation Exhibition",
"date": "2026-04-22",
"text": "The Grand University Hall hosted t
},
{
"id": 5,
"title": "Academic Cooperation with Interna
"date": "2026-02-08",

"text": "Aden University signed a cooperati
},
{
"id": 6,
"title": "University Programming Tournament
"date": "2026-03-30",
"text": "Yesterday concluded the University
},
{
"id": 7,
"title": "Workshop on Generative AI",
"date": "2026-04-05",
"text": "The College of Engineering organiz
},
{
"id": 8,
"title": "Open Source Library Released",
"date": "2026-04-18",
"text": "A team of Yemeni developers releas
},
{

```
        "id": 9,
        "title": "Lecture on Technology Ethics",
        "date": "2026-04-12",
        "text": "A visiting professor delivered a l
    },
    {
        "id": 10,
        "title": "Best Graduation Project Competiti
        "date": "2026-04-25",
        "text": "The university announced a competi
    },
]

with open("articles.csv", "w", encoding="utf-8-sig"
    writer = csv.DictWriter(f, fieldnames=["id", "t
    writer.writeheader()
    writer.writerows(articles)

print(f"Created articles.csv with {len(articles)} a
```

Save this in a file `seed_data.py` and run once:

```
python seed_data.py
```

Step 4: The Main Script

Now we create `summarize.py` — the heart of the project:

```
"""
CSV Article Summarizer with AI.
Reads articles.csv, summarizes each via LLM, saves
"""
import csv
import logging
import os
import time
from pathlib import Path
from dotenv import load_dotenv
from groq import Groq

# 1. Setup
load_dotenv()
```

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message
    handlers=[
        logging.FileHandler("log.txt", encoding="ut
        logging.StreamHandler(),
    ]
)

INPUT_FILE = Path("articles.csv")
OUTPUT_FILE = Path("summaries.csv")
MODEL = "llama-3.3-70b-versatile"
MAX_RETRIES = 3

# 2. Verify the key
api_key = os.environ.get("GROQ_API_KEY")
if not api_key:
    raise SystemExit("Missing GROQ_API_KEY in .env'

client = Groq(api_key=api_key)
```

```
def summarize_article(title: str, text: str) -> str
    """Asks the model for a brief summary of the ar
    prompt = f"""You are a journalistic editor writ
Summarize the following article in two sentences or

Title: {title}

Text:
{text}

Summary:"""

    for attempt in range(1, MAX_RETRIES + 1):
        try:
            response = client.chat.completions.crea
                model=MODEL,
                messages=[
                    {"role": "user", "content": pro
                ],
                max_tokens=150,
```

```
                temperature=0.3,
            )
            return response.choices[0].message.cont
        except Exception as e:
            logging.warning(f"Attempt {attempt} fai
            if attempt < MAX_RETRIES:
                time.sleep(2 ** attempt)

    return "[Summary failed]"


def main():
    # 3. Verify the file
    if not INPUT_FILE.exists():
        raise SystemExit(f"File {INPUT_FILE} doesn

    # 4. Reading
    with INPUT_FILE.open("r", encoding="utf-8-sig")
        reader = csv.DictReader(f)
        articles = list(reader)
```

```
logging.info(f"Read {len(articles)} articles")

# 5. Summarizing
summaries = []
for i, article in enumerate(articles, 1):
    logging.info(f"Processing {i}/{len(articles

    summary = summarize_article(article["title"

    summaries.append({
        "id": article["id"],
        "title": article["title"],
        "date": article["date"],
        "summary": summary,
    })

    # respect rate limits
    time.sleep(1)

# 6. Writing
with OUTPUT_FILE.open("w", encoding="utf-8-sig"
```

```
        writer = csv.DictWriter(
            f, fieldnames=["id", "title", "date", "
        )
        writer.writeheader()
        writer.writerows(summaries)

    logging.info(f"Done! Output: {OUTPUT_FILE}")
    print(f"\nSucceeded. Open {OUTPUT_FILE} to see


if __name__ == "__main__":
    main()
```

Step 5: Run It

```
python summarize.py
```

You'll see in terminal:

```
2026-04-29 ... - INFO - Read 10 articles
2026-04-29 ... - INFO - Processing 1/10: Launching
2026-04-29 ... - INFO - Processing 2/10: Launching
```

```
...
2026-04-29 ... - INFO - Done! Output: summaries.csv
```

Open `summaries.csv` in Excel — you'll find a new "summary" column with summaries.

Step 6: Verify Quality

Not every summary will be perfect. Read the results. If you find a weak one, try:

1. Modifying the prompt — make it clearer.
2. Trying another model.
3. Adjusting `temperature` (0.1 makes the model more committed, 0.7 more creative).

This process is called "prompt engineering", and you'll learn more about it in the upcoming books.

Suggested Improvements

1. Parallelism

Your current script processes articles one by one. For 1000 articles, it'll take an hour. You can speed it up with `concurrent.futures`:

```
from concurrent.futures import ThreadPoolExecutor,

def process_article(article):
    summary = summarize_article(article["title"], a
    return {**article, "summary": summary}

with ThreadPoolExecutor(max_workers=5) as executor:
    futures = {executor.submit(process_article, a)
    summaries = []
    for future in as_completed(futures):
        summaries.append(future.result())
```

5 parallel requests — 5x speed. But watch out for rate limits.

2. Caching

If you rerun the script, it calls the API again for every article.

Add a cache:

```
import hashlib
import json

CACHE_FILE = Path("cache.json")
cache = json.loads(CACHE_FILE.read_text("utf-8")) i

def cached_summarize(title, text):
    key = hashlib.md5(text.encode()).hexdigest()
    if key in cache:
        return cache[key]
    summary = summarize_article(title, text)
    cache[key] = summary
    CACHE_FILE.write_text(json.dumps(cache, ensure_
    return summary
```

3. Cost Tracking

```
total_input_tokens = 0
total_output_tokens = 0

# inside summarize_article
total_input_tokens += response.usage.prompt_tokens
total_output_tokens += response.usage.completion_to

# at the end
print(f"Tokens: {total_input_tokens} input, {total_
```

(Groq is free, but if you used Claude or OpenAI, calculate at real prices.)

4. Command Line Interface

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--input", default="articles.cs
parser.add_argument("--output", default="summaries
```

```
parser.add_argument("--model", default="llama-3.3-7
args = parser.parse_args()
```

Run with `python summarize.py --input my_data.csv --output result.csv`.

What Did You Learn from This Project?

In 100 lines of Python you applied:

- Reading CSV (Chapter 9)
- Calling API (Chapter 13)
- Error handling and retries (Chapter 11)
- Environment variables (Chapter 12)
- logging (Chapter 11)
- pathlib (Chapter 8)
- f-strings (Chapter 4)
- type hints (`title: str` — type hints)
- Project structure (requirements.txt, .gitignore, separating seed_data from main code)

This is a complete data pipeline. This is the same pattern you'll build on millions of texts in the RAG book. The only difference: instead of summarizing each text, you'll convert it to a vector and save it in a vector database.

Ideas for Extending the Project

1. **Text Classification**: add a column classifying each article (education, AI, ethics...).
2. **Entity Extraction**: extracting names of people, places, organizations.
3. **Translation**: translating each article to English.
4. **Q&A**: for each article, generate 3 likely questions and their answers.
5. **Web Interface**: use Streamlit (`pip install streamlit`) to convert the script to a small web app.

Bridge to the RAG Book

In the RAG book you'll build a more complex system, but the same architecture:

```
input documents → process each → call AI API → save
```

Replace "summarize" with "vector embedding", replace "save in CSV" with "save in vector database", add a search step, and you get RAG. This whole book was preparing you for that next step.

Chapter Summary

You completed your first practical AI project. All the skills you learned met in 100 lines producing real value. This is what tech companies do daily. You are now ready for the RAG book, where you'll build more complex systems but built on the same foundations you mastered.

Chapter 14 Exercises

1. Run the project and review the results. Choose the worst summary and modify the prompt to improve it.
2. Add a "topic" column to the output file, classifying each article into one of: education, technology, society, other.

3. Compare results between Llama 3.3 (Groq) and Claude Haiku or GPT-4o-mini. Which is better at Arabic?
4. Create a "stream" version of the script that prints each summary as soon as it's produced (instead of waiting until the end).

Chapter 15: Exercise Solutions

> *"Don't look at the answer before you've struggled for a full hour."*

This chapter contains solutions to exercises from all previous chapters. **Beware the direct answer**. If you look before serious attempt, you've wasted the most important learning opportunity. Struggle is what builds skill. Verification is what polishes it.

Write the solution yourself. Run it. Fix its errors. Then come here for comparison.

Chapter 3 Solutions: Your First Program

Exercise 1: Welcome by name and country

```
name = input("What is your name? ")
country = input("What is your country? ")
print(f"Welcome {name} from {country}")
```

Exercise 2: Rectangle area

```
length = float(input("Length: "))
width = float(input("Width: "))
area = length * width
print(f"Area: {area}")
```

Exercise 3: Extended calculator

```
a = float(input("First number: "))
b = float(input("Second number: "))
print(f"Sum: {a + b}")
```

```
print(f"Difference: {a - b}")
print(f"Product: {a * b}")
if b != 0:
    print(f"Division: {a / b}")
else:
    print("Division: not possible (denominator zero
```

Exercise 4: Calculate age

```
birth = int(input("Year of birth: "))
age = 2026 - birth
print(f"Your age approximately: {age}")
```

Chapter 4 Solutions: Variables and Types

Exercise 1: Price after discount

```
price = float(input("Price: "))
discount = float(input("Discount %: "))
```

```
new_price = price - (price * discount / 100)
print(f"Price after discount: {new_price:.2f}")
```

Exercise 2: Text analysis

```
text = input("Enter text: ")
print(f"Number of letters: {len(text)}")
print(f"Reversed text: {text[::-1]}")
print(f"Uppercase: {text.upper()}")
```

Exercise 3: Average of three grades

```
g1 = float(input("First grade: "))
g2 = float(input("Second grade: "))
g3 = float(input("Third grade: "))
average = (g1 + g2 + g3) / 3
print(f"Average: {average:.2f}")
```

Exercise 4: Difference between operations

```
print(10 / 3)   # 3.333...  regular division, decim
print(10 // 3)  # 3         integer division, trunc
```

```
print(10 % 3)    # 1          remainder

print(7 / 2)     # 3.5
print(7 // 2)    # 3
print(7 % 2)     # 1

print(15 / 4)    # 3.75
print(15 // 4)   # 3
print(15 % 4)    # 3
```

Chapter 5 Solutions: Control Flow

Exercise 1: FizzBuzz

```
for i in range(1, 101):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
```

```
        print("Buzz")
    else:
        print(i)
```

Exercise 2: Largest and smallest

```
nums = []
for i in range(5):
    n = float(input(f"Number {i+1}: "))
    nums.append(n)
print(f"Largest: {max(nums)}")
print(f"Smallest: {min(nums)}")
```

Exercise 3: Password with limited attempts

```
correct = "yemen2026"
for attempt in range(3):
    pwd = input("Password: ")
    if pwd == correct:
        print("Correct! Welcome.")
        break
    else:
```

```
        print(f"Wrong. {2 - attempt} attempts remai
else:
    print("Attempts exhausted.")
```

Exercise 4: Cubes of multiples of four

```
cubes = [n ** 3 for n in range(1, 21) if n % 4 == (
print(cubes)   # [64, 512, 1728, 4096, 8000]
```

Chapter 6 Solutions: Functions

Exercise 1: Celsius to Fahrenheit

```
def temperature_to_fahrenheit(c):
    return c * 9/5 + 32

print(temperature_to_fahrenheit(0))     # 32.0
print(temperature_to_fahrenheit(100))   # 212.0
```

Exercise 2: Counting Arabic letters

```
def arabic_letter_count(text):
    count = 0
    for char in text:
        # Unicode range for Arabic letters
        if "ـ" <= char <= "ه":
            count += 1
    return count

print(arabic_letter_count("Hello 123 مرحبا"))    # 5
```

Exercise 3: Safe divide

```
def safe_divide(a, b):
    if b == 0:
        return None
    return a / b

print(safe_divide(10, 2))     # 5.0
print(safe_divide(10, 0))     # None
```

Exercise 4: Format name

```
def format_name(first, last, title="Mr."):
    return f"{title} {first} {last}"

print(format_name("Ahmed", "Ali"))
print(format_name("Fatima", "Mohammed", "Dr."))
```

Chapter 7 Solutions: Data Structures

Exercise 1: Grades list statistics

```
grades = [85, 92, 78, 65, 88, 71, 95, 58, 80, 77]
average = sum(grades) / len(grades)
highest = max(grades)
lowest = min(grades)
passed = len([g for g in grades if g >= 60])
print(f"Average: {average:.2f}")
print(f"Highest: {highest}")
print(f"Lowest: {lowest}")
print(f"Passed: {passed}/{len(grades)}")
```

Exercise 2: Oldest student

```
students = {
    "Ahmed": 22,
    "Mohammed": 25,
    "Khalid": 19,
    "Fatima": 28,
    "Ali": 21,
}
oldest = max(students, key=students.get)
print(f"Oldest: {oldest} ({students[oldest]} years)
```

Exercise 3: List operations

```
list_a = ["Ahmed", "Mohammed", "Khalid", "Ali"]
list_b = ["Mohammed", "Fatima", "Ali", "Maryam"]
set_a, set_b = set(list_a), set(list_b)
print("Shared:", set_a & set_b)
print("Only in first:", set_a - set_b)
print("All without duplicates:", set_a | set_b)
```

Exercise 4: Dict to tuples

```
d = {"Ahmed": 88, "Mohammed": 92, "Khalid": 75}
tuples = list(d.items())
print(tuples)
```

Chapter 8 Solutions: Files

Exercise 1: Text file analysis

```
from pathlib import Path

text = Path("poem.txt").read_text(encoding="utf-8")
lines = text.splitlines()
words = text.split()
longest = max(words, key=len)
print(f"Lines: {len(lines)}")
print(f"Words: {len(words)}")
print(f"Longest word: {longest}")
```

Exercise 2: Reverse lines

```
from pathlib import Path

text = Path("input.txt").read_text(encoding="utf-8"
reversed_lines = "\n".join(reversed(text.splitlines
Path("reversed.txt").write_text(reversed_lines, enc
```

Exercise 3: Search in folder

```
from pathlib import Path

def search_in_folder(folder, keyword):
    matches = []
    for file in Path(folder).glob("*.txt"):
        if keyword in file.read_text(encoding="utf-
            matches.append(file.name)
    return matches

print(search_in_folder("documents", "Python"))
```

Exercise 4: Logger

```
from datetime import datetime
from pathlib import Path

def log(message):
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%
    line = f"[{now}] {message}\n"
    with open("log.txt", "a", encoding="utf-8") as
        f.write(line)

log("Program started")
log("Operation succeeded")
```

Chapter 9 Solutions: JSON and CSV

Exercise 1: Create JSON and average

```
import json
from pathlib import Path
```

```
students = [
    {"name": "Ahmed", "age": 22, "grade": 88, "city
    {"name": "Mohammed", "age": 25, "grade": 92, "c
    {"name": "Khalid", "age": 19, "grade": 75, "cit
    {"name": "Fatima", "age": 21, "grade": 95, "cit
    {"name": "Maryam", "age": 23, "grade": 80, "cit
]

Path("students.json").write_text(
    json.dumps(students, ensure_ascii=False, indent
    encoding="utf-8"
)

loaded = json.loads(Path("students.json").read_text
avg = sum(s["grade"] for s in loaded) / len(loaded)
print(f"Average: {avg:.2f}")
```

Exercise 2: Separate passed and failed

```
import csv
```

```
with open("students.csv", "r", encoding="utf-8-sig"
    students = list(csv.DictReader(f))

passed = [s for s in students if int(s["grade"]) >=
failed = [s for s in students if int(s["grade"]) <

for name, data in [("passed.csv", passed), ("failed
    with open(name, "w", encoding="utf-8-sig", newl
        if data:
            writer = csv.DictWriter(f, fieldnames=d
            writer.writeheader()
            writer.writerows(data)
```

Exercise 3: JSON to CSV

```
import json
import csv
from pathlib import Path

books = json.loads(Path("books.json").read_text(enc
```

```
with open("books.csv", "w", encoding="utf-8-sig", n
    writer = csv.DictWriter(f, fieldnames=["title",
    writer.writeheader()
    writer.writerows(books)
```

Exercise 4: pandas top 10

```
import pandas as pd

df = pd.read_csv("students.csv", encoding="utf-8-si
top10 = df.sort_values("grade", ascending=False).he
print(top10)
```

Chapter 10 Solutions: HTTP

Exercise 1: Three cities weather table

```
import requests

cities = {
    "Aden": (12.78, 45.04),
```

```
    "Taiz": (13.58, 44.02),
    "Hodeidah": (14.80, 42.95),
}

for name, (lat, lon) in cities.items():
    r = requests.get(
        "https://api.open-meteo.com/v1/forecast",
        params={"latitude": lat, "longitude": lon,
        timeout=10
    )
    w = r.json()["current_weather"]
    print(f"{name}: {w['temperature']}°C, wind {w[
```

Exercise 2: GitHub repos

```
import requests

r = requests.get(
    "https://api.github.com/users/tensorflow/repos'
    params={"per_page": 5, "sort": "updated"},
    timeout=10
```

```
)

for repo in r.json():
    print(f"{repo['name']}: {repo['stargazers_count
```

Exercise 3: safe_get

```
import requests
import time

def safe_get(url, max_retries=3):
    for attempt in range(1, max_retries + 1):
        try:
            r = requests.get(url, timeout=10)
            r.raise_for_status()
            return r.json()
        except (requests.Timeout, requests.Connecti
            print(f"Attempt {attempt}: {e}")
            if attempt < max_retries:
                time.sleep(2 ** attempt)
    return None
```

Exercise 4: Wikipedia

```
import requests
from pathlib import Path

title = "Yemen"
url = f"https://en.wikipedia.org/api/rest_v1/page/s
r = requests.get(url, timeout=10)
data = r.json()
Path(f"{title}.txt").write_text(data["extract"], en
print("Saved.")
```

Chapter 11 Solutions: Errors

Exercise 1: safe_int

```
def safe_int(text):
    try:
        return int(text)
    except ValueError:
```

```
        print(f"Warning: '{text}' is not a number,
        return 0
```

Exercise 2: Safe JSON read

```
import json
from pathlib import Path

def read_json_safely(path):
    try:
        return json.loads(Path(path).read_text(enco
    except (FileNotFoundError, json.JSONDecodeEror
        return {}
```

Exercise 3: Custom error

```
class InsufficientBalanceError(Exception):
    pass

def withdraw(account, amount):
    if amount > account["balance"]:
        raise InsufficientBalanceError(
```

```
            f"Balance {account['balance']} insuffic
        )
    account["balance"] -= amount
    return account["balance"]

acct = {"balance": 100}
try:
    withdraw(acct, 200)
except InsufficientBalanceError as e:
    print(e)
```

Exercise 4: Safe guessing game

```
import random

secret = random.randint(1, 100)
attempts = 0

while attempts < 7:
    raw = input("Guess: ")
    try:
```

```
        guess = int(raw)
    except ValueError:
        print("Enter a valid integer")
        continue
    attempts += 1
    if guess == secret:
        print(f"Correct! In attempt {attempts}")
        break
    print("Larger" if guess < secret else "Smaller"
else:
    print(f"The number was {secret}")
```

Chapter 12 Solutions: API Keys

Exercise 1: Secure project

```
mkdir secure-project
cd secure-project
echo "API_KEY=" > .env.example
echo "API_KEY=test123" > .env
```

```
echo ".env" > .gitignore
echo "venv/" >> .gitignore
git init
git status        # should not show .env
```

Exercise 2: get_required_env

```
import os

def get_required_env(name):
    value = os.environ.get(name)
    if not value:
        raise SystemExit(f"Required variable '{name
    return value
```

Exercise 3: Display keys safely

```
from dotenv import dotenv_values

values = dotenv_values(".env")
for key, value in values.items():
    if value:
```

```
        masked = value[:4] + "*" * (len(value) - 4)
    else:
        masked = "(empty)"
    print(f"{key} = {masked}")
```

Exercise 4: git-secrets

> *git-secrets is an open source tool that scans every new commit before push for known key patterns (AWS keys, GitHub tokens, etc.). If a match is found, it blocks the commit. Installed with `brew install git-secrets` or from source, and activated in each repository with `git secrets --install`. Protects the programmer from the classic mistake of accidentally pushing a key.*

Chapter 13 Solutions: LLM APIs

Exercise 1: 5 questions about Yemen

```
from groq import Groq
from pathlib import Path

client = Groq()
questions = [
    "What is the capital of Yemen?",
    "What are the most prominent historical landmar
    "What is the approximate population of Yemen?",
    "What is the official language of Yemen?",
    "What are the most important Yemeni ports?",
]

answers = []
for q in questions:
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": q}],
```

```
        max_tokens=200,
    )
    answers.append(f"Q: {q}\nA: {r.choices[0].messa

Path("yemen_qa.txt").write_text("\n".join(answers),
```

Exercise 2: Temperature and variety

```
from groq import Groq

client = Groq()
question = "Write a poetic line about the sea."

for i in range(3):
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": quest
        max_tokens=80,
        temperature=1.0,
    )
```

```
    print(f"--- Attempt {i+1} ---")
    print(r.choices[0].message.content)
```

Exercise 3: Conversation with memory

```
from groq import Groq

client = Groq()
history = []

while True:
    user = input("You: ")
    if user.lower() in {"quit", "exit"}:
        break
    history.append({"role": "user", "content": user
    r = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=history,
        max_tokens=300,
    )
    assistant = r.choices[0].message.content
```

```
    history.append({"role": "assistant", "content":
    print(f"Model: {assistant}")
```

Exercise 4: Compare Claude and Llama in translation

```
# requires ANTHROPIC_API_KEY and GROQ_API_KEY
from anthropic import Anthropic
from groq import Groq

text = "Artificial intelligence is reshaping every

claude = Anthropic().messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=200,
    messages=[{"role": "user", "content": f"Transla
).content[0].text

llama = Groq().chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role": "user", "content": f"Transla
```

```
    max_tokens=200,
).choices[0].message.content

print("Claude:", claude)
print()
print("Llama:", llama)
```

Closing Word

You completed the book's exercises. The number of problems you solved is in the dozens, and the minutes of debugging in the hundreds. You are now a Python programmer. Not a beginner — a programmer. Take this confidence with you to the next RAG book.

Chapter 16: Conclusion and Bridge to the RAG World

> *"Knowledge in the chest is like a seedling in the soil; water it with practice."*

Dear student,

You are now on a page you couldn't have imagined two weeks ago. When you opened the first chapter, you hadn't written a line of Python. Now you can read files, call AI models, process data, and build complete pipelines. This is a great leap. Take a moment to appreciate what you've accomplished.

But the journey hasn't ended. This book is the first gateway. There are many gateways ahead. This short chapter tells you where to go now and how to prepare for the next book in the LLM4Yemen program.

What Do You Know Now?

Let me summarize what you've become able to do:

In fundamentals: you know how a programming language works and how to translate your idea into code Python executes. You can write a function, organize it in a file, and import it into another program. You have a strong intuition for types: when to use list, when dict, when set.

In data handling: you can read and write text files, JSON, CSV, even Arabic in UTF-8 encoding. You know subtle differences (like `utf-8-sig` for Excel and `ensure_ascii=False` for Arabic JSON).

In networking: you can request any API on the internet, understand its response, handle its errors, retry on failure.

In security: you won't write an API key in code again. You know `.env` and `.gitignore` like old friends.

In AI: you've called Claude, GPT, and Llama models. You got Arabic responses. You built a real summarization project.

This is not a hobby skill list. This is a professional skill list. Any tech company in the world today is looking for people who can do what you can do.

What's the Gap Between This Book and the RAG Book?

The next RAG book in the LLM4Yemen program assumes you: - write Python fluently - know JSON and CSV - call APIs confidently - handle keys safely - write pure functions, document them, test them

All these assumptions are valid now. You are ready.

What you'll add to your knowledge in the RAG book:

New concepts: embeddings, vector databases, semantic search, chunking, evaluation, overlapping chunks, re-ranking.

New libraries: LangChain, LlamaIndex, ChromaDB, Pinecone Python client, sentence-transformers.

Architectural patterns: ingestion pipelines, query pipelines, caching layers, automated evaluation, cloud deployment.

Each of these will build on what you've learned. Nothing starts from scratch.

Tips for the Next Two Weeks Before Starting the RAG Book

First: keep writing. Stopping coding for two weeks weakens your skills. Write at least 30 minutes daily. Reopen chapters, solve other exercises, invent small problems for yourself.

Second: read others' code. Visit GitHub, find small Python projects, read them. Notice how professionals write, organize files, name variables. This culture is acquired through exposure.

Third: don't rely entirely on ChatGPT or Claude. Yes, you can ask them to write code for you. But in early stages this hurts you. Write it yourself first. Then ask them to review your

code and suggest improvements. This way you learn from them, not depend on them.

Fourth: share your experiences. Write a short post on LinkedIn or X about what you learned. Teach a colleague. Explanation reveals gaps in your understanding and fills them.

Fifth: start a personal project. Choose a problem that interests you (however simple), and try to solve it with code. Maybe a small app to organize your readings, or a script to analyze Excel files for your work. A personal project forces you to link all skills together.

Six Libraries You Should Explore

Before starting the RAG book, spend an hour with each (don't go deep, just acquaint):

1. **pandas** — for tabular data analysis. If you haven't used it yet, watch a 10-minute YouTube video on it.
2. **numpy** — for numbers and arrays. The foundation of all ML libraries.

3. **tqdm** — progress bar for long loops. One line saves your patience in long pipelines.
4. **rich** — beautiful terminal formatting: tables, colors, progress bar. Makes your scripts look professional.
5. **typer** — building CLI interfaces quickly. Simpler than argparse.
6. **pytest** — automatically testing your code. Even if you don't write tests, get familiar with the philosophy.

Each will appear in the RAG book or in your professional work later.

A Final Personal Word

Dear reader:

I wrote this book imagining you. I imagined you in Sana'a or Aden or Taiz or Hadramaut. I imagined your old computer, the power cuts that interrupt your concentration, the expensive data you consume on Colab. I imagined your dreams of being among Yemen's tech-future builders.

These dreams are not far. Every world-class technologist began where you are now: from a first chapter they understood nothing of, from a first program that didn't work, from a first error they thought was the end of the world.

Yemen deserves a generation of engineers who build, not consume. Who make their tools, not wait for someone to make them. Who connect technology with their country's real problems. You are from this generation.

Continue. Commit. Trust.

I'll see you in the RAG book.

— Dr. Fadhl Mohammed Alakwaa

"And say: My Lord, increase me in knowledge."

An Open Letter to Students Studying with Peers

Programming is a team sport. Hold weekly sessions with your peers:

- **Code review**: each week, one student presents their project and the others discuss.
- **Solving challenges together**: pick an exercise from the book and solve it on a board or shared screen.
- **Learning from mistakes**: share the hardest problems you've encountered and how you solved them.
- **Preparing for the RAG book**: start by reading the RAG book's introduction together and posing questions brewing in your minds.

Group multiplies your learning rate. Don't learn in isolation.

A Final Checklist

Before you open the RAG book, make sure you can:

- [ ] Create a new project with a virtual environment and a requirements.txt file
- [ ] Read an Arabic CSV file and save a modified copy of it
- [ ] Call any API returning JSON and handle HTTP errors
- [ ] Store an API key in `.env` and import it with python-dotenv
- [ ] Call an LLM and get a correct Arabic response
- [ ] Write a function taking parameters, processing them, and returning a result
- [ ] Handle `try/except` to prevent program crash
- [ ] Use pathlib to manage paths in a cross-OS way
- [ ] Understand the difference between list, dict, set, and tuple, and when to use each
- [ ] Read a Python error message and quickly determine its cause

Each checkmark in this list is a fortified bastion. Mark ✓ on what you've mastered, and review what you haven't before starting.

Thank you for reading until this line. Thank you for choosing to learn AI. Thank you for believing in yourself.

Now go and build something the world hasn't seen before.

References

Official Documentation

- **Official Python 3 Documentation** — `https://docs.python.org/3/`
- **Python Tutorial for Beginners** — `https://docs.python.org/3/tutorial/`
- **Standard Library Reference** — `https://docs.python.org/3/library/`
- **PEP 8: Style Guide for Python Code** — `https://peps.python.org/pep-0008/`
- **PEP 257: Docstring Conventions** — `https://peps.python.org/pep-0257/`

Large Language Model APIs

- **Anthropic API Documentation** — `https://docs.anthropic.com`

- **OpenAI API Documentation** — https://platform.openai.com/docs
- **Groq Cloud Documentation** — https://console.groq.com/docs
- **OpenRouter — Unified Gateway to Dozens of Models** — https://openrouter.ai/docs
- **Hugging Face Inference API** — https://huggingface.co/docs/api-inference

Essential Libraries

- **requests — HTTP Requests** — https://requests.readthedocs.io
- **python-dotenv — Loading .env Files** — https://github.com/theskumar/python-dotenv
- **pandas — Data Analysis** — https://pandas.pydata.org/docs/
- **numpy — Numerical Computing** — https://numpy.org/doc/

- **pathlib — File Paths** — https://docs.python.org/3/library/pathlib.html
- **json — JSON Processing** — https://docs.python.org/3/library/json.html
- **csv — CSV Processing** — https://docs.python.org/3/library/csv.html

Development Environments

- **Visual Studio Code** — https://code.visualstudio.com
- **Google Colab** — https://colab.research.google.com
- **Jupyter Notebooks** — https://jupyter.org
- **PyCharm Community** — https://www.jetbrains.com/pycharm/

Open Educational Resources

- **Real Python — Articles and Exercises** — `https://realpython.com`
- **Python.org Beginner Guide** — `https://www.python.org/about/gettingstarted/`
- **Automate the Boring Stuff with Python** (free book) — `https://automatetheboringstuff.com`
- **Think Python (Allen B. Downey)** (free book) — `https://greenteapress.com/wp/think-python-3rd-edition/`
- **Composing Programs (UC Berkeley CS61A)** — `https://composingprograms.com`

Communities and Discussions

- **Stack Overflow — Q&A** — `https://stackoverflow.com/questions/tagged/python`

- **Python Discord** — https://pythondiscord.com
- **r/learnpython on Reddit** — https://www.reddit.com/r/learnpython/
- **Real Python Newsletter** — https://realpython.com/newsletter/

Arabic Text Processing

- **CAMeL Tools — Arabic Language Processing Tools** — https://github.com/CAMeL-Lab/camel_tools
- **PyArabic — Small Arabic Library** — https://github.com/linuxscout/pyarabic
- **Arabic NLP Resources** — https://github.com/SOMASS/Arabic-NLP-Resources

Security and Keys

- **OWASP Secrets Management Cheat Sheet** — https://cheatsheetseries.owasp.org/

cheatsheets/
Secrets_Management_Cheat_Sheet.html

- **git-secrets** — https://github.com/awslabs/git-secrets
- **GitHub Secret Scanning** — https://docs.github.com/en/code-security/secret-scanning

Daily Programmer Tools

- **Git — Version Control** — https://git-scm.com/doc
- **GitHub Skills** — https://skills.github.com
- **Conventional Commits** — https://www.conventionalcommits.org
- **The Twelve-Factor App** — https://12factor.net

Recommended Reference Books

- **Fluent Python (Luciano Ramalho)** — the deepest book to master Python.
- **Effective Python (Brett Slatkin)** — 90 ways to improve your code.
- **Python Tricks (Dan Bader)** — short snippets to improve your style.
- **Architecture Patterns with Python (Harry Percival)** — organizing large projects.
- **Designing Data-Intensive Applications (Martin Kleppmann)** — foundation for understanding data pipelines.

AI-Specialized Resources

- **Anthropic Cookbook** — `https://github.com/anthropics/anthropic-cookbook`
- **OpenAI Cookbook** — `https://cookbook.openai.com`

- **LangChain Documentation** — https://python.langchain.com
- **LlamaIndex Documentation** — https://docs.llamaindex.ai
- **Hugging Face Course** — https://huggingface.co/learn

Text Encoding Standards

- **Unicode Standard** — https://www.unicode.org/standard/standard.html
- **UTF-8 Explained** — https://en.wikipedia.org/wiki/UTF-8
- **W3C Internationalization** — https://www.w3.org/International/

Code Editing and Formatting Tools

- **Black — Automatic Code Formatter** — https://black.readthedocs.io

- **isort — Import Sorting** — `https://pycqa.github.io/isort/`
- **flake8 — Code Style Checker** — `https://flake8.pycqa.org`
- **mypy — Type Checker** — `https://mypy.readthedocs.io`
- **ruff — Modern Fast Linter** — `https://docs.astral.sh/ruff/`

All references above are free. Many are available to read without creating an account. Start with what's relevant to your current problem, and expand gradually.

عن الكتاب

كتاب عملي يأخذ القارئ من عدم معرفة البرمجة إلى بناء أول تطبيق ذكاء اصطناعي عربي، في أسبوعين فقط. يغطي بايثون من الأساسيات إلى استدعاء النماذج اللغوية، ويُختتم بمشروع تلخيص ملف CSV عربي عبر الذكاء الاصطناعي. مصمَّم خصيصًا لطلاب مسار B في برنامج LLM4Yemen، ويُمهد الطريق لكتاب RAG التالي في البرنامج.

عن المؤلف

الدكتور فضل محمد الأكوع باحث في البيولوجيا الحاسوبية والمعلوماتية الحيوية والجينوميات المكانية بجامعة ميشيغان في آن أربر. تتمحور أبحاثه حول تطبيق الذكاء الاصطناعي في دراسة الأمراض المزمنة، ومن أبرز أعماله المنشورة دراسات على مثبطات ناقل الصوديوم-غلوكوز 2 (SGLT2) والتغيرات الأنبوبية الكلوية المنشورة في مجلة JCI عام 2023. إلى جانب مسيرته العلمية، يكتب د. فضل باللغتين العربية والإنجليزية في حقول متعددة تشمل الدراسات القرآنية، تاريخ اليمن والسياسة، الذكاء الاصطناعي والتقنية، الأسرة والتربية، ودليل المهاجرين في المهجر.

مؤلفات المؤلف الأخرى

معالجة اللغة العربية مع نماذج الذكاء الاصطناعي: العمل بلغتك الأم (2026)

البناء مع كلود — دليل عملي لواجهة برمجة Anthropic (2026)

بناء معرض أعمالك في مجال الذكاء الاصطناعي: 30 مشروعاً حقيقياً (2026)

هندسة الأوامر — فن التحاور مع الذكاء الاصطناعي (2026)

الذكاء الاصطناعي لصنّاع المحتوى - دليل التصميم والصوت والفيديو (2026)

الأتمتة بدون كود - دليل بناء سير عمل ذكي (2026)

العملاء الأذكياء وبروتوكول MCP - بناء أنظمة ذكاء اصطناعي مستقلة (2026)

التوليد المعزز بالاسترجاع - دليل عملي لبناء أنظمة RAG (2026)

العمل الحر بالذكاء الاصطناعي — دليل المستقل اليمني والعربي (2026)

نوت بوك إل إم — الدليل الشامل (2026)

القانون الجميل الكبير - تشريح إصلاح ترامب الضريبي وتأثيره على أمريكا (2026)

التقارب السني الشيعي (2026)

الذكاء الاصطناعي ومستقبل المهن — أيها يندثر وأيها يصعد (2026)

التأشيرات الاستثمارية في أمريكا — دليل المقارنة والاختيار (2026)

اليمن في ملفات إبستين (2026)

الرئيس إبراهيم الحمدي — حياته ومشروعه الوطني (2026)

الشخصيات العامة التي فقدت وظائفها بسبب موقفها من غزة (2026)

الصوفية — دراسة شاملة (2026)

الطعام والصحة بين العلم الحديث والحديث النبوي (2026)

النظام الضريبي الأمريكي — دليل شامل (2026)

دليل شراء اللحم الحلال في ميشيجان وإنديانا (2026)

سلسلة التوريد — دليل شامل (2026)

ميكانيكا الكم — دراسة شاملة (2026)

أدوات الذكاء الاصطناعي — دليل شامل (2026)

Mythos في مواجهة نماذج Anthropic (2026)

مهارات كلود — كيف يجعلك استخدامها خبيراً في كل شيء (2026)

البوذية والهندوسية — النشأة والانتشار والمقارنة (2026)

الذنوب والمعاصي والأخلاق في الإسلام — مقارنة مع الأديان الأخرى (2026)

الذكاء الاصطناعي وثورة اكتشاف الأدوية (2026)

حرب غزة — كيف فضحت المجتمع الدولي (2026)

مجازر ضد المسلمين عبر العصور (2026)

العلاقات اليمنية السعودية — تاريخ وتحولات عبر الأزمان (2026)

كيف تؤسّس شركتك في أمريكا (2026)

هندسة البيانات — الدليل الشامل (2026)

جماعة الدعوة والتبليغ — دراسة شاملة (2026)

أجيال على المحك — الإسلام في المهجر بين الحفاظ والانسلاخ (2026)

المسلمون في الغرب — الإنجازات والتحديات والاندماج (2026)

الربا في الإسلام والبنوك الإسلامية (2026)

القرآن والإنجيل — دراسة مقارنة في مفاهيم السلام والقتال والتسامح (2026)

نهاية الكون في القرآن الكريم (2026)

مشاكل اليمن عبر الأزمان (2026)

سر التقارب الأمريكي الإسرائيلي — من يقود من؟ (2026)

هل انتهت الوحدة اليمنية؟ (2026)

الرؤساء اليمنيون في مذكرات ويكيليكس (2026)

أسرار قوة التحالف الأمريكي الإسرائيلي (2026)

فن التربية — كيف تصنع قادة من أبنائك (2026)

تأثير النماذج اللغوية الكبيرة على البحث العلمي والتعليم الجامعي (2026)

علامات الساعة (2026)

التخطيط للتقاعد (2026)

العملات الرقمية — دليل شامل للمبتدئين (2026)

وادي السيليكون — تاريخ وابتكار وريادة عالمية (2026)

أشهر المتنزهات الوطنية في أمريكا (2026)

دليل شراء وبيع السيارات المستعملة في أمريكا (2026)

كلود كود — الدليل الشامل (2026)

دليل المهاجر اليمني الجديد إلى أمريكا (2026)

الحقوق الزوجية والحياة الأسرية للمسلمين في الغرب (2026)

مستقبل المعلوماتية الحيوية والبيولوجيا الحاسوبية في عصر الذكاء الاصطناعي (2026)

تصنيف الآيات في القرآن الكريم — دراسة تأملية في البنية الموضوعية (2026)

كيف تعمل النماذج اللغوية الكبيرة — دليل مبسّط (2026)

سر قوة الجامعات الأمريكية (2026)

عتيقة — حكاية جدتي من جبال اليمن (2026)

الخلافة الإسلامية: من السقيفة إلى سقوط غرناطة (2026)

السلفية — دراسة في المفهوم والنشأة والتيارات المعاصرة (2026)

النقد المشروع وحدود معاداة السامية (2026)

كيف تنجح أكاديمياً في الجامعات الأمريكية (2026)

الزواج الناجح — دليل عملي بمنظور إسلامي (2026)

الإعجاز العلمي في القرآن — من منظور باحث في الجينوميات (2026)

أمريكا — صعود القوة وبوادر الأفول (2026)

About this Book

A practical book taking the reader from zero programming knowledge to building their first Arabic AI application in just two weeks. Covers Python from fundamentals to calling LLM APIs, concluding with a capstone project summarizing an Arabic CSV via AI. Designed specifically for Track B students of the LLM4Yemen program, paving the way for the upcoming RAG textbook.

About the Author

Dr. Fadhl Mohammed Alakwaa is a researcher in computational biology, bioinformatics, and spatial genomics at the University of Michigan in Ann Arbor. His research focuses on applying AI to the study of chronic diseases; among his notable publications are studies on SGLT2 inhibitors and kidney tubular changes published in JCI in 2023. Alongside his scientific career, Dr. Alakwaa writes in both Arabic and English across fields including Quranic studies, Yemeni history and politics, AI and technology, family and parenting, and guides for diaspora life.